```
============================================================
REPORT PRESTAZIONI DELLE APPLICAZIONI
============================================================
```

Applicazione	Pods	CPU (cores)	RAM (MB)	Disk Write (B/s)	Net RX (B/s)
cache-app	1	0.0077	0.84	499226	0
demo-app	2	0	15.62	0	0
worker-app	2	0.5926	5.27	0	0

```
CLASSIFICA CONSUMI (Top Consumers):
 - Maggior consumo di CPU:      worker-app (0.5926 cores)
 - Maggior consumo di RAM:      demo-app (15.62 MB)
 - Maggior scrittura su DISCO: cache-app (499225.90 B/s)
 - Maggior traffico di RETE:    cache-app (0.00 B/s)

METRICHE PRESTAZIONALI DELLO SCRIPT:
 Tempo totale impiegato: 0.1799 secondi
 Costo API / Token:      0 token (0.00$)
 Dati salvati in MongoDB: k8s_observability.agent_metrics
============================================================

[OK] Raccolta deterministica completata in 0.181 s!
Topologia scoperta: ['cache-app', 'demo-app', 'worker-app']
Snapshot persistito su MongoDB ('agent_metrics').
```

DOMANDA 1

```
------------------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [1]: Quale applicazione tra quelle monitorate assorbe il maggior rate di CPU, e quanti core impiega complessivamente?
------------------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed samplin
g defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.05 ms	0	0	0	N/A	worker-app (0.5926 cores)
Gemini 2.5 Flash	2.09 s	1629	110	1739	0.48	L'applicazione che assorbe il maggior rate di CPU è `worker-app`, con un consumo complessivo di 0.5925686113211808 core.
Gemini 3.6 Flash	14.94 s	1643	911	2554	0.95	L'applicazione che assorbe il maggior rate di CPU tra quelle monitorate è **`worker-app`**. ### Dettaglio del consumo di CPU: * **`worker-app`**: **`0.5926` core complessivi** (pari a circa **592.6 mCPU**) distribuiti sui suoi 2 pod attivi. --- ### Confronto con le altre applicazioni: * **`cache-app`**: `0.0077` core (~7.7 mCPU) su 1 pod. * **`demo-app`**: `0.0000` core (0 mCPU, stato idle) su 2 pod. --- ### Osservazione SRE: `worker-app` rappresenta la quasi totalità del carico CPU corrente del cluster (~98.7% dell'utilizzo totale rilevato). Si consiglia di verificare che le risorse impostate nei campi `requests` e `limits` dei pod siano adeguate e di valutare l'attivazione di un **Horizontal Pod Autoscaler (HPA)** basato sulla CPU per gestire eventuali picchi di carico futuri.
Claude Opus 5.5	9.61 s	2775	795	3570	0.95	**L'applicazione che usa più CPU è worker-app, con circa 0,593 core in totale** (0,5926 core). | Applicazione | Pod | CPU (core) | Quota sul totale | |---|---|---|---| | **worker-app** | 2 | **0,5926** | **~98,7%** | | cache-app | 1 | 0,0077 | ~1,3% | | demo-app | 2 | 0,0000 | 0% | | **Totale** | 5 | **~0,600** | 100% | **Note:** - **Ripartizione tra i pod:** il valore di 0,593 core dovrebbe essere la somma dei 2 pod (`worker-app-84cf4dd446-9kj75` e `-ftlln`), quindi circa **0,30 core per pod**, supponendo che il carico sia distribuito in modo uniforme. Il dato non mostra il consumo dei singoli pod. - **Saturazione:** da questi numeri non si può dire se worker-app sia vicina al limite, perché il dato non include i `requests`/`limits` CPU del deployment. Se il limite per pod fosse ad esempio 500m, l'utilizzo sarebbe intorno al 60%; se fosse 250m, i pod sarebbero già rallentati dal limite. Conviene controllare questi valori nel manifest prima di decidere su un eventuale HPA. - **Memoria:** worker-app usa circa 5,3 MB, quindi il suo carico è quasi solo di calcolo. - **demo-app:** ha 2 repliche con CPU a zero e nessun traffico di rete in questo dato. Sembra inattiva, quindi si potrebbe valutare di ridurla a 1 replica, dopo aver confermato con lo storico che non è un caso momentaneo. Questa è una lettura di un singolo momento. Se vuoi, posso recuperare lo storico di worker-app da MongoDB o Prometheus per capire se il consumo è stabile o se ci sono picchi.

Claude Sonnet 5.5	5.22 s	2775	473	3248	0.95	**Applicazione con il maggior consumo di CPU: `worker-app`** - **CPU complessiva:** circa **0,593 core** (0,5926), cioè circa 593 millicore, sommati sui suoi 2 pod. - **Media per pod:** circa 0,30 core. **Confronto con le altre applicazioni** | Applicazione | Pod | CPU (core) | |---|---|---| | worker-app | 2 | 0,5926 | | cache-app | 1 | 0,0077 | | demo-app | 2 | 0,0 | `worker-app` usa circa 77 volte la CPU di `cache-app`. `demo-app` risulta idle, con CPU, disco e rete a zero. **Osservazioni** - `worker-app` ha un consumo di CPU rilevante ma una memoria molto bassa (5,27 MB). È quindi un carico CPU-bound, senza pressione sulla memoria. - Lo snapshot non include i `requests` e i `limits` di CPU. Per questo non posso dire se i pod siano vicini alla saturazione. Se il limit fosse 0,5 core per pod, ad esempio, sarebbero al 60%. - Per valutare un HPA su `worker-app` servono i valori di `requests` e il trend storico. Posso recuperarlo con `get_historical_metrics` se vuoi.
GPT-4o Mini	3.33 s	1321	68	1389	0.50	L'applicazione che assorbe il maggior rate di CPU è la **worker-app**, che utilizza complessivamente **0.5926 core**. Le altre applicazioni, **cache-app** e **demo-app**, utilizzano rispettivamente **0.0077 core** e **0.0 core**.

GPT-4o Mini	3.33 s	1321	68	1389	0.50	L'applicazione che assorbe il maggior rate di CPU è la **worker-app**, che utilizza complessivamente **0.5926 core**. Le altre applicazioni, **cache-app** e **demo-app**, utilizzano rispettivamente **0.0077 core** e **0.0 core**.
GPT-4o	3.59 s	1321	41	1362	0.47	L'applicazione che assorbe il maggior rate di CPU tra quelle monitorate è la "worker-app", che utilizza complessivamente 0.5925686113211808 core.

DOMANDA 1 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.95 ms	0	0	0	N/A	worker-app (0.5926 cores)
Gemini 2.5 Flash	2.68 s	1637	326	1963	0.50	Il servizio con il consumo di CPU più elevato è **worker-app**, con un utilizzo totale di **0.5926 core** sommando tutte le sue istanze.
Gemini 3.6 Flash	7.47 s	1651	1311	2962	0.80	Sulla base delle metriche istantanee correnti del cluster, l'analisi del consumo di CPU per ciascun servizio è la seguente: 1. **`worker-app`**: - **Pod attivi (2)**: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` - **Consumo CPU totale**: **`0.5926 core`** (pari a circa **592.57 mCPU**) 2. **`cache-app`**: - **Pod attivi (1)**: `cache-app-5c6fd4f5cf-xxsz4` - **Consumo CPU totale**: **`0.0077 core`** (pari a circa **7.69 mCPU**) 3. **`demo-app`**: - **Pod attivi (2)**: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` - **Consumo CPU totale**: **`0.0 core`** (stato idle/inattivo) --- ### **Risposta sintetica** * **Servizio con il consumo di CPU più elevato:** **`worker-app`** * **Ammontare totale di CPU utilizzata dalle sue istanze:** **`0.592568 core`** (~**0.593 core** / **592.57 mCPU**)

Claude Opus 5.5	13.29 s	2792	1190	3982	0.95	# Servizio con il consumo di CPU più alto: **worker-app** | Servizio | Pod | CPU totale (core) | Media per pod* | Quota della CPU usata nel cluster | |---|---|---|---|---| | **worker-app** | 2 | **0,5926** | circa 0,296 | **circa 98,7%** | | cache-app | 1 | 0,0077 | 0,0077 | circa 1,3% | | demo-app | 2 | 0,0000 | 0,0000 | 0% | | **Totale cluster** | 5 | **circa 0,6003** | – | 100% | **Risposta:** sommando i suoi 2 pod (`worker-app-84cf4dd446-9kj75` e `worker-app-84cf4dd446-ftlln`), **worker-app usa circa 0,59 core**, cioè circa 593 millicore. Ho dato per scontato che `cpu_cores` nello snapshot sia già il totale dei pod dell'app. *La media per pod è solo il totale diviso per due. Lo snapshot non mostra come la CPU si divide davvero tra i due pod. ## Cosa significa e cosa consiglio - **Quasi tutta la CPU del cluster va a worker-app.** Usa anche poca memoria (circa 5,3 MB), quindi sembra un carico che lavora soprattutto di calcolo, il che è normale per un worker. - **Non so dire se worker-app è al limite.** Lo snapshot non contiene i `requests` e i `limits` della CPU. Se il limite per pod fosse 500m, ogni pod sarebbe a circa il 60%. Se fosse 300m, sarebbe al limite e il sistema lo starebbe già rallentando. Prossimi passi: – Controllare il throttling con `rate(container_cpu_cfs_throttled_periods_total{pod=~"worker-app.*"}[5m])`. – Confrontare il consumo con i limiti tramite `kube_pod_container_resource_limits{resource="cpu"}`. - **Autoscaling (HPA):** worker-app è l'unico servizio dove ha senso un HPA basato sulla CPU, con obiettivo intorno al 70% della request. Le repliche dovrebbero restare tra 2 e 4, da tarare quando si conosceranno i limiti. - **Servizi inattivi:** – **demo-app** usa 0 core, 0 traffico di rete e 0 scritture su disco con 2 pod. Si può scendere a 1 replica, o anche a 0 se non serve. – **cache-app** usa poca CPU ma scrive su disco in modo continuo (circa 488 KB/s) senza ricevere traffico di rete. Per una cache è strano: conviene capire cosa sta scrivendo. Se volete, posso controllare l'andamento storico di worker-app per capire se questo consumo è stabile o è un picco.
Claude Sonnet 5.5	7.78 s	2792	739	3531	0.95	**Servizio con il consumo di CPU più elevato: `worker-app`** Questo dato viene dallo snapshot corrente, quindi non ho invocato tool. | Servizio | Pod | CPU totale (core) | |---|---|---| | **worker-app** | 2 | **≈ 0,593** | | cache-app | 1 | ≈ 0,0077 | | demo-app | 2 | 0,0 | – `worker-app` usa circa **0,5926 core** sommando i suoi due pod (`worker-app-84cf4dd446-9kj75` e `worker-app-84cf4dd446-ftlln`). In media sono circa 0,30 core a pod, se il carico è bilanciato. Lo snapshot riporta solo il valore aggregato, quindi la ripartizione per pod è una stima. – Pesa per circa il **98,7%** dei core usati nel cluster. Il totale del cluster è circa 0,600 core. – Ha un consumo di CPU alto ma usa poca memoria (≈ 5,27 MB). Il profilo è quindi CPU-bound. **Raccomandazioni** 1. Controlla requests e limits di CPU di `worker-app`. Con questi valori potrei calcolare la saturazione reale. Se il limit è vicino a 0,3 core per pod, il servizio è prossimo al throttling. 2. Valuta un HPA su CPU, per esempio con target al 60–70% della request. Servono però uno storico del carico e le requests configurate. Posso recuperare lo storico con `get_historical_metrics` per vedere se il consumo è stabile o a picchi. 3. `demo-app` è idle: 0 CPU, 0 disco e 0 rete, con 2 repliche. Si può ridurre a 1 replica, dopo aver verificato lo storico. 4. `cache-app` ha un carico trascurabile sulla CPU, ma scrive su disco a circa 0,5 MB/s. Vale la pena capire perché, visto che non riceve traffico di rete. Vuoi che analizzi lo storico di `worker-app` per capire se il carico è costante o a picchi?
GPT-4o Mini	3.04 s	1329	173	1502	0.28	Analizzando le metriche istantanee fornite, possiamo vedere i seguenti consumi di CPU per ciascuna applicazione: – **cache-app**: 0.0077 core – **demo-app**: 0.0 core – **worker-app**: 0.5926 core Il servizio con il consumo di CPU più elevato è **worker-app**, con un totale di 0.5926 core utilizzati. Poiché ci sono 2 pod attivi per **worker-app**, il consumo totale di CPU per questa applicazione è: – **worker-app**: 0.5926 core (per pod) * 2 pod = 1.1852 core Quindi, il totale di core utilizzati dalla **worker-app** è di **1.1852 core**.
GPT-4o	1.88 s	1329	52	1381	0.47	Il servizio nel cluster che presenta il consumo di CPU più elevato è il "worker-app". Attualmente, il "worker-app" utilizza un totale di 0.5925686113211808 core CPU sommando tutte le sue istanze.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.4708 (mini: $0.0680 | 4o: $1.4029)

DOMANDA 1 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [1B]: Dall'analisi delle metriche di utilizzo del processore, quale applicazione risulta essere il top consumer di cicli CPU e qual è il suo tasso di assorbimento comple
ssivo in core?
--------------------------------------------------------------------------------------------------------------

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.63 ms	0	0	0	N/A	worker-app (0.5926 cores)
Gemini 2.5 Flash	2.96 s	1642	318	1960	0.50	Dall'analisi delle metriche di utilizzo del processore, l'applicazione **worker-app** risulta essere il top consumer di cicli CPU, con un tasso di assorbimento complessivo di **0.5926 core**.
Gemini 3.6 Flash	7.83 s	1656	784	2440	0.95	Dall'analisi delle metriche istantanee correnti sui carichi di lavoro attivi, l'applicazione risultante come **top consumer di cicli CPU** è **`worker-app`**. ### Dettaglio del consumo di CPU per applicazione: 1. **`worker-app`** (2 pod attivi): - **Tasso di assorbimento complessivo:** **`0.5926 core`** (circa **592.57 millicores** / mCPU) 2. **`cache-app`** (1 pod attivo): - **Tasso di assorbimento complessivo:** `0.0077 core` (circa **7.69 millicores**) 3. **`demo-app`** (2 pod attivi): - **Tasso di assorbimento complessivo:** `0.0000 core` (stato idle) --- ### Raccomandazioni SRE: - **`worker-app`**: Assorbe quasi la totalità delle risorse di calcolo dell'elaborazione corrente (~0.593 CPU core distribuito su 2 pod, pari a ~0.296 core per pod). Se l'applicazione è soggetta a picchi di carico di lavoro in elaborazione (es. code di job), si consiglia di impostare/verificare le metriche per l'HPA (*Horizontal Pod Autoscaler*) basato sull'utilizzo CPU. - **`demo-app`**: Risulta attualmente inattiva dal punto di vista CPU (`0.0 core`). Se questo andamento dovesse persistere, si potrebbe valutare la riduzione delle repliche a 1 o lo scaling a zero nei contesti di non-produzione/dev.
Claude Opus 5.5	11.71 s	2798	1054	3852	0.95	# Il maggior consumatore di CPU è worker-app **worker-app** usa più CPU di tutte le altre applicazioni. Insieme, i suoi pod consumano **0,5926 core** (circa 593 millicore). ## Confronto CPU (dati attuali) | Applicazione | Pod | CPU totale (core) | CPU media per pod (core) | Quota del consumo totale | |---|---|---|---|---| | **worker-app** | 2 | **0,5926** | ~0,296 | **~98,7%** | | cache-app | 1 | 0,0077 | 0,0077 | ~1,3% | | demo-app | 2 | 0,0000 | 0,0000 | 0% | | **Totale cluster** | 5 | **~0,6003** | - | 100% | ## Osservazioni - **worker-app usa quasi tutta la CPU.** Il suo consumo è circa 77 volte quello di cache-app. La memoria invece è bassa (circa 5,3 MB), quindi il suo lavoro pesa sul processore e non sulla memoria. - **Media per pod.** Ogni pod usa in media circa 0,3 core. Questo calcolo presuppone che il carico sia diviso in parti uguali tra i 2 pod, ma i dati attuali mostrano solo il totale. - **Non posso dire se è saturo.** Non conosco i valori `requests` e `limits` impostati per la CPU di worker-app. Per esempio, con un limite di 500m per pod, l'uso sarebbe intorno al 60% del limite. Con un limite di 300m, sarebbe vicino al massimo e quindi a rischio di rallentamento forzato (throttling). - **demo-app non usa CPU.** Ha 2 repliche ma 0 core di CPU e nessun traffico di rete: sembra inattiva. Potrebbe essere un candidato per ridurre le repliche, dopo aver controllato lo storico. ## Prossimi passi consigliati 1. **Controllare requests e limits della CPU** di worker-app (`kubectl describe deploy worker-app`) per capire quanto margine resta. 2. **Verificare se c'è throttling** con questa query Prometheus: `rate(container_cpu_cfs_throttled_periods_total{pod=~"worker-app.*"}[5m])`. 3. **Valutare l'autoscaling (HPA) basato sulla CPU** per worker-app, con un obiettivo intorno al 70% delle requests, se il carico cambia nel tempo. Posso recuperare lo storico da MongoDB per vedere se il consumo è stabile o ha dei picchi.

Claude Sonnet 5.5	5.73 s	2798	567	3365	0.95	Dallo snapshot corrente, il top consumer di CPU è **worker-app**, con un assorbimento complessivo di circa **0,593 core** (0,5926 cores). Il valore è aggregato su 2 pod, quindi la media è circa 0,30 core per pod. | Applicazione | Pod | CPU (core) | |---|---|---| | **worker-app** | 2 | **0,5926** | | cache-app | 1 | 0,0077 | | demo-app | 2 | 0,0000 | - worker-app consuma circa 77 volte la CPU di cache-app. Anche la memoria è molto bassa (5,3 MB), quindi il carico è CPU-bound e non memory-bound. - cache-app ha un consumo CPU trascurabile (~7,7 millicore). La sua attività è soprattutto di scrittura su disco (~499 KB/s). - demo-app è idle: 0 CPU, 0 I/O disco e 0 I/O di rete. **Raccomandazioni** - Per worker-app, controlla requests e limits di CPU. Se il limite per pod è vicino a 0,3–0,5 core, c'è rischio di throttling. In quel caso valuta un HPA basato sulla CPU, perché il carico è concentrato su questa app. - Questo snapshot è istantaneo e non dice se il carico sia stabile o un picco. Se vuoi, posso verificare l'andamento storico con `get_historical_metrics` prima di decidere sullo scaling. - demo-app è idle. Se il comportamento è confermato nel tempo, puoi ridurre le repliche da 2 a 1.
GPT-4o Mini	4.57 s	1339	119	1458	0.70	Dall'analisi delle metriche di utilizzo del processore fornite, l'applicazione che risulta essere il top consumer di cicli CPU è la **worker-app**, con un utilizzo complessivo di **0.5926 core**. Le altre applicazioni hanno un utilizzo di CPU significativamente inferiore: - **cache-app**: 0.0077 core - **demo-app**: 0.0 core Pertanto, la **worker-app** è l'applicazione con il maggior assorbimento di cicli CPU.
GPT-4o	2.06 s	1339	59	1398	0.50	Dall'analisi delle metriche istantanee fornite, l'applicazione "worker-app" risulta essere il top consumer di cicli CPU. Il suo tasso di assorbimento complessivo in core è di 0.5925686113211808.

DOMANDA 2

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.71 ms	0	0	0	N/A	demo-app: 15.62 MB
Gemini 2.5 Flash	7.54 s	1631	350	1981	0.76	Le applicazioni con il maggior numero di pod (2 repliche ciascuna) sono `demo-app` e `worker-app`. L'uso effettivo di memoria Working Set (in MB) è: - **demo-app**: 15.625 MB - **worker-app**: 5.27 MB
Gemini 3.6 Flash	19.18 s	3626	1791	5417	0.80	Le applicazioni con il maggior numero di pod/repliche attive sono **`demo-app`** e **`worker-app`**, ciascuna con **2 pod**. Di seguito è riportato l'uso effettivo di memoria Working Set per entrambe le applicazioni: --- ### 1. **`demo-app`** (2 pod) * **Memoria Working Set totale:** **15,625 MB** (~15,06 MB - 15,63 MB) * **Dettaglio sui singoli pod:** * `demo-app-7958c64697-2qg9s`: **7,47 MB** * `demo-app-7958c64697-4rzjl`: **7 ,59 MB** --- ### 2. **`worker-app`** (2 pod) * **Memoria Working Set totale:** **5,27 MB** (~5,20 MB - 5,27 MB) * **Dettaglio sui singoli pod:** * `worker-app-84cf4dd446-ftlln`: **4,08 MB** * `worker-app-84cf4dd446-9kj75`: **1,12 MB** --- ### **Considerazioni SRE:** * Entrambe le applicazioni mostrano un consumo di memoria molto contenuto e ben al di sotto dei limiti operativi critici. * Se l'obiettivo della ricerca era identificare l'applicazione a maggior consumo tra quelle maggiormente replicate, **`demo-app`** è quella con il Working Set complessivo più elevato (15,625 MB).

Claude Opus 5.5	21.94 s	6035	1206	7241	0.86	Nessuna app ha più pod delle altre: **demo-app** e **worker-app** hanno entrambe **2 pod** (cache-app ne ha 1). Ho quindi controllato il Working Set di tutte e due direttamente su Prometheus (`container_memory_working_set_bytes`). | Applicazione | Pod | Working Set per pod | **Totale Working Set** | Valore nello snapshot | |---|---|---|---|---| | **demo-app** | 2qg9s / 4rzjl | 7,47 MB / 7,59 MB | **≈ 15,06 MB** | 15,63 MB | | **worker-app** | ftlln / 9kj75 | 4,08 MB / 1,12 MB | **≈ 5,20 MB** | 5,27 MB | **Risposta:** demo-app usa **circa 15,06 MB** in totale, cioè circa 7,5 MB per pod, ed è l'app a 2 pod che usa più memoria. worker-app usa circa 5,20 MB. **Note:** - **Differenza con lo snapshot:** i valori dello snapshot sono un po' più alti (dal 2 al 4% circa). Probabilmente sono stati misurati in un momento leggermente diverso, oppure contano anche il container di sistema "pause", che io ho escluso. Per misurare il Working Set effettivo conviene usare i valori di Prometheus. - **demo-app sembra inattiva:** usa 0 core di CPU, non ha traffico di rete né scritture su disco. Se resta così nel tempo, potete ridurla a 1 replica senza rischi. - **worker-app è sbilanciata:** usa quasi tutta la CPU (~0,59 core) ma pochissima memoria. Inoltre la memoria è distribuita in modo diseguale tra i due pod (4,1 MB contro 1,1 MB), quindi forse il lavoro non è ripartito bene tra le repliche. Se volete l'autoscaling (HPA) per questa app, basatelo sulla CPU e non sulla memoria.
Claude Sonnet 5.5	22.33 s	6484	1246	7730	0.86	Due applicazioni hanno il numero massimo di repliche, cioè 2 pod: **demo-app** e **worker-app**. Ecco il Working Set (`container_memory_working_set_bytes`) di ciascuna, misurato ora su Prometheus. | Applicazione | Pod | Working Set per pod | Totale | |---|---|---|---| | **demo-app** | demo-app-7958c64697-2qg9s | ~7,47 MB | **~15,04 MB** | | | demo-app-7958c64697-4rzjl | ~7,59 MB | | | **worker-app** | worker-app-84cf4dd446-ftlln | ~4,08 MB | **~5,20 MB** | | | worker-app-84cf4dd446-9kj75 | ~1,12 MB | | I valori sono in MiB (1 MiB = 1.048.576 byte). Il totale di ogni applicazione è la somma dei suoi due pod. Le misure live sono leggermente diverse dallo snapshot, che indicava 15,63 MB per demo-app e 5,27 MB per worker-app. Lo scarto dipende dal momento della misura. **Osservazioni** - **demo-app** usa circa 15 MB in totale, con i due pod bilanciati. Ha CPU, disco e rete a 0, quindi sembra inattiva. Se non serve per alta disponibilità, si può ridurre a 1 replica. - **worker-app** ha un consumo di memoria basso, ma i due pod sono sbilanciati: 4,08 MB contro 1,12 MB. La CPU è invece alta, circa 0,59 core in totale. Il collo di bottiglia è la CPU, non la memoria, quindi conviene valutare un HPA basato sulla CPU.
GPT-4o Mini	2.25 s	1324	51	1375	0.67	L'applicazione con il maggior numero di pod è la "demo-app", che ha attualmente 2 pod attivi. L'uso effettivo di memoria Working Set per la "demo-app" è di 15.625 MB.
GPT-4o	1.91 s	1324	52	1376	0.67	L'applicazione con il maggior numero di pod o repliche è la "demo-app", che ha 2 pod attivi. L'uso effettivo di memoria Working Set per la "demo-app" è di 15.625 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.4791 (mini: $0.0685 | 4o: $1.4106)

DOMANDA 2 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.58 ms	0	0	0	N/A	demo-app: 15.62 MB
Gemini 2.5 Flash	4.72 s	1637	790	2427	0.67	Il carico di lavoro con il deployment più esteso per numero di pod è `demo-app` (a pari merito con `worker-app`), entrambi con 2 pod. La memoria Working Set totale per `demo-app` ammonta a 15.625 MB.
Gemini 3.6 Flash	62.27 s	5575	2667	8242	0.83	Dall'analisi della topologia e delle metriche attuali, i deployment più estesi per numero di pod sono a pari merito **`demo-app`** e **`worker-app`**, ciascuno attivo con **2 pod**. Di seguito viene riportato il dettaglio del calcolo della memoria Working Set totale per ciascuno dei due carichi di lavoro: --- ### 1. **`demo-app`** (2 pod) * **Pod attivi:** * `demo-app-7958c64697-2qg9s`: ~7,82 MB (7.823.360 byte) * `demo-app-7958c64697-4rzjl`: ~7,94 MB (7.942.144 byte) * **Memoria Working Set Totale:** * **15,625 MB** (pari a ~15,04 MiB / 15,77 MB reali da query Prometheus). --- ### 2. **`worker-app`** (2 pod) * **Pod attivi:** * `worker-app-84cf4dd446-ftlln`: ~4,38 MB (4.378.624 byte) * `worker-app-84cf4dd446-9kj75`: ~1,02 MB (1.019.904 byte) * **Memoria Working Set Totale:** * **5,27 MB** (pari a ~5,15 MiB / 5,40 MB reali da query Prometheus). --- ### Sintesi Raccomandazioni SRE - **`demo-app`**: Nonostante occupi il maggior quantitativo di memoria working set (15,625 MB complessivi), il consumo CPU è nullo (0.0 cores), indicando un carico attualmente in stato idle o in attesa di richieste. - **`worker-app`**: Presenta un uso moderato della memoria (5,27 MB totali) ma registra l'utilizzo CPU più elevato del cluster (~0.59 cores), rappresentando l'applicazione a maggior intensità computazionale.
Claude Opus 5.5	9.42 s	2785	806	3591	0.36	**Nessun deployment è il più esteso da solo: demo-app e worker-app sono a pari merito con 2 pod ciascuno.** La memoria Working Set totale è **15,625 MB per demo-app** e **circa 5,27 MB per worker-app**. | Deployment | Pod | Memoria Working Set totale | Media per pod | CPU | |---|---|---|---|---| | **demo-app** | 2 (`2qg9s`, `4rzjl`) | **15,625 MB** | circa 7,81 MB | 0 core | | **worker-app** | 2 (`9kj75`, `ftlln`) | **5,27 MB** (5,26953125) | circa 2,63 MB | circa 0,59 core | | cache-app | 1 | 0,84 MB | - | circa 0,008 core | Insieme, i due deployment a 2 pod usano **circa 20,89 MB**. **Ipotesi:** considero il campo `memory_mb` già sommato su tutti i pod di ogni deployment. È il comportamento tipico di una query come `sum by (app)` su `container_memory_working_set_bytes`, ma non l'ho verificato sulla query originale. Se invece fosse una media per pod, i totali raddoppierebbero: 31,25 MB per demo-app e circa 10,54 MB per worker-app. **Osservazioni:** - **demo-app sembra inattivo:** 0 CPU, nessun traffico di rete e nessuna scrittura su disco, con 2 repliche attive. Se lo storico conferma che resta inattivo, conviene ridurlo a 1 replica. - **worker-app è l'unico con carico CPU reale** (circa 0,3 core per pod), ma usa pochissima memoria. Al momento non serve aggiungere repliche. Se il carico cresce, conviene un HPA basato sulla CPU, purché i pod abbiano `requests` impostate.
Claude Sonnet 5.5	19.73 s	6489	1452	7941	0.88	**Non c'è un solo deployment più esteso: `demo-app` e `worker-app` hanno entrambi 2 pod.** `cache-app` ne ha 1. Ti do la memoria Working Set totale di tutti e due. | Deployment | Pod | Working Set per pod | Totale | |---|---|---|---| | **demo-app** | `2qg9s` | 7.823.360 B (~7,46 MB) | **15.765.504 B ≈ 15,04 MB** | | | `4rzjl` | 7.942.144 B (~7,57 MB) | | | **worker-app** | `ftlln` | 4.612.096 B (~4,40 MB) | **6.139.904 B ≈ 5,86 MB** | | | `9kj75` | 1.527.808 B (~1,46 MB) | | I dati vengono da `container_memory_working_set_bytes` su Prometheus, sommata sui container dei pod. I MB sono in MiB (1 MB = 1.048.576 B).

						I dati vengono da `container_memory_working_set_bytes` su Prometheus, sommata sui container dei pod. I MB sono in MiB (1 MB = 1.048.576 B). Se devi sceglierne uno solo, dipende da come risolvi il pareggio: - **Primo per memoria totale:** `demo-app`, con circa 15,04 MB. - **Primo per carico:** `worker-app`, che usa 0,59 core di CPU contro 0 di `demo-app`, ma ha il working set più basso. I valori dello snapshot sono leggermente diversi: 15,63 MB per `demo-app` e 5,27 MB per `worker-app`. Probabilmente dipende dal momento del campionamento. Per un valore aggiornato usa quelli di Prometheus. **Osservazioni:** - **`demo-app`:** ha CPU, disco e rete a 0 con 2 repliche, quindi sembra idle. Potresti ridurlo a 1 replica, dopo aver controllato lo storico per escludere picchi periodici. - **`worker-app`:** i due pod sono molto sbilanciati, con 4,4 MB contro 1,5 MB. Verifica come viene distribuito il lavoro. La CPU a 0,59 core è il vero punto da tenere d'occhio, ad esempio con un HPA basato sulla CPU.
GPT-4o Mini	3.40 s	1329	147	1476	0.21	Il deployment con il numero maggiore di pod è "demo-app", che ha attualmente 2 pod attivi. La memoria utilizzata da ciascun pod della "demo-app" è di 15.625 MB. Pertanto, la memoria Working Set totale per la "demo-app" è calcolata come segue: \[\text{Memoria totale} = \text{Numero di pod} \times \text{Memoria per pod} = 2 \times 15.625 \, \text{MB} = 31.25 \, \text{MB} \] Quindi, la memoria Working Set totale per il deployment "demo-app" è di **31.25 MB**.
GPT-4o	2.47 s	1329	109	1438	0.67	Il deployment con il maggior numero di pod attivi è "demo-app", che ha 2 pod attivi. Per calcolare la memoria Working Set totale in megabyte per "demo-app", possiamo utilizzare la metrica di memoria istantanea fornita: - Memoria per "demo-app": 15.625 MB Poiché "demo-app" ha 2 pod, la memoria totale è già calcolata per tutti i pod, quindi la memoria Working Set totale per "demo-app" è 15.625 MB.

DOMANDA 2B

```
================================================================================
ESECUZIONE DOMANDA [2B]: Considerando le applicazioni che contano il maggior numero di repliche attive, a quanto corrisponde il loro consumo aggregato di Working Set espresso in MB?
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.66 ms	0	0	0	N/A	demo-app: 15.62 MB
Gemini 2.5 Flash	2.56 s	1634	334	1968	0.00	Le applicazioni con il maggior numero di repliche attive sono "demo-app" e "worker-app", entrambe con 2 pod. Il loro consumo aggregato di Working Set è: - demo-app: 15.625 MB - worker-app: 5.26953125 MB Totale: 15.625 + 5.26953125 = 20.89453125 MB
Gemini 3.6 Flash	36.48 s	1648	1373	3021	0.00	Analizzando la topologia e le metriche istantanee correnti fornite: 1. **Identificazione delle applicazioni con il maggior numero di repliche attive:** * **`cache-app`**: 1 replica * **`demo-app`**: 2 repliche * **`worker-app`**: 2 repliche Le applicazioni con il maggior numero di repliche attive (pari a **2**) sono **`demo-app`** e **`worker-app`**. 2. **Consumo di Working Set (`memory_mb`) per ciascuna di esse:** * **`demo-app`**: 15,625 MB * **`worker-app`**: 5,26953125 MB 3. **Calcolo del consumo aggregato:** $$\text{Consumo Totale} = 15,625 \text{ MB} + 5,26953125 \text{ MB} = \mathbf{20,89453125 \text{ MB}} \approx \mathbf{20,89 \text{ MB}}$$ Il consumo aggregato di Working Set per le applicazioni con il maggior numero di repliche attive corrisponde a **20,89453125 MB** (circa **20,89 MB**).
Claude Opus 5.5	8.58 s	2787	709	3496	0.36	**Risposta: circa 20,89 MB** (20,89453125 MB esatti) di Working Set aggregato. **Applicazioni con più repliche attive** | Applicazione | Repliche | Working Set (MB) | |---|---|---| | demo-app | 2 | 15,625 | | worker-app | 2 | 5,26953125 | | **Totale** | **4 pod** | **20,89453125** | cache-app ha una sola replica, quindi non rientra nel calcolo. **Ipotesi:** considero `memory_mb` come Working Set già sommato su tutti i pod di ciascuna app. È la convenzione abituale per i dati per deployment. Se invece fosse un valore per singolo pod, il totale raddoppierebbe a circa 41,79 MB. Si può verificare con una query Prometheus su `container_memory_working_set_bytes`. **Raccomandazioni** - **Memoria:** i consumi sono molto bassi per entrambe le app. Conviene ridurre le `requests`/`limits` di memoria a valori proporzionati, così il cluster sarà più efficiente nell'assegnare i pod ai nodi. - **demo-app:** in questo momento è inattiva (CPU a 0, nessun traffico di rete o scrittura su disco). Se non sta gestendo picchi di traffico, si può valutare di scendere a 1 replica, oppure un HPA con `minReplicas: 1`. - **worker-app:** è l'unica app con un carico CPU rilevante (circa 0,59 core in totale, circa 0,30 per pod), mentre la memoria è minima. Per un eventuale HPA il segnale giusto è la CPU, non la memoria.
Claude Sonnet 5.5	6.71 s	2787	721	3508	0.00	**Consumo aggregato di Working Set: circa 20,89 MB** Le applicazioni con più repliche attive sono due, entrambe con 2 pod: | Applicazione | Pod | Working Set (`memory_mb`) | |---|---|---| | demo-app | 2 | 15,625 MB | | worker-app | 2 | 5,26953125 MB | | **Totale** | 4 | **20,89453125 MB** | Ho escluso cache-app perché ha una sola replica. Ho letto `memory_mb` come valore aggregato per applicazione, cioè la somma dei pod di ciascun Deployment. Lo snapshot non lo dice in modo esplicito. Se invece fosse una media per pod, il totale sarebbe 2 × 15,625 + 2 × 5,27 ≈ 41,79 MB. Il carico è molto basso. demo-app è idle, con CPU, disco e rete a 0, e usa circa 7,8 MB per pod. worker-app consuma pochissima memoria (circa 2,6 MB per pod) ma ha una CPU di 0,59 core. Per worker-app il vincolo è quindi la CPU, non la memoria. Un HPA basato sulla CPU avrebbe più senso di uno basato sulla memoria. Per demo-app si può valutare di ridurre le repliche a 1, dopo aver controllato lo storico per escludere picchi periodici.
GPT-4o Mini	7.06 s	1328	260	1588	0.00	Per calcolare il consumo aggregato di Working Set espresso in MB delle applicazioni con il maggior numero di repliche attive, consideriamo le metriche attuali delle applicazioni: 1. **cache-app**: 1 pod, 0.8359375 MB 2. **demo-app**: 2 pod, 15.625 MB 3. **worker-app**: 2 pod, 5.26953125 MB Ora calcoliamo il consumo aggregato di Working Set per le applicazioni con il maggior numero di repliche attive, che sono **demo-app** e **worker-app** (entrambi con 2 repliche): - **demo-app**: 2 pods * 15.625 MB = 31.25 MB - **worker-app**: 2 pods * 5.26953125 MB = 10.5390625 MB Ora sommiamo i risultati: - Consumo aggregato = 31.25 MB + 10.5390625 MB = **41.7890625 MB** Quindi, il consumo aggregato di Working Set delle applicazioni con il maggior numero di repliche attive è **41.79 MB** (approssimato a due decimali).

GPT-4o	2.41 s	1328	49	1377	0.67	L'applicazione con il maggior numero di repliche attive è la "demo-app", che ha 2 pod attivi. Il consumo aggregato di Working Set per la "demo-app" è di 15.625 MB.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.4880 (mini: $0.0691 | 4o: $1.4189)

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.21 ms	0	0	0	N/A	cache-app (499225.90 B/s)
Gemini 2.5 Flash	1.64 s	1621	104	1725	0.50	L'applicazione `cache-app` genera un flusso attivo di scrittura su disco con un rate di 499225.9 Bps.
Gemini 3.6 Flash	4.89 s	1635	688	2323	0.76	L'applicazione che genera un flusso attivo di scrittura su disco è **`cache-app`**. Dalle metriche istantanee correnti, la frequenza di scrittura su disco (*disk write rate*) per `cache-app` è di: * **~499.226 Byte/s** (circa **499,23 KB/s** / **0,50 MB/s**) * Equivalente a **~487,53 KiB/s** Le altre applicazioni (`demo-app` e `worker-app`) presentano attualmente un rate di scrittura su disco pari a **0,0 Byte/s**.
Claude Opus 5.5	13.09 s	2764	1180	3944	0.29	**L'unica app che scrive su disco è `cache-app`.** | App | Pod | Scrittura su disco | |---|---|---| | **cache-app** | 1 (`cache-app-5c6fd4f5cf-xxsz4`) | **≈ 499.226 B/s** | | demo-app | 2 | 0 B/s | | worker-app | 2 | 0 B/s | **Velocità di scrittura di `cache-app`:** - **≈ 499,2 KB/s**, cioè circa 0,5 MB/s (≈ 487,5 KiB/s) - Se resta costante: **≈ 1,8 GB all'ora** e **≈ 43 GB al giorno** **Cosa c'è di strano:** - **Le scritture non sembrano causate da traffico in ingresso:** la rete in entrata è a 0 B/s. Sembra che l'app scriva da sola, ad esempio log molto dettagliati, salvataggi periodici su disco (snapshot o dump) o un ciclo di scrittura. - **Rapporto anomalo per una cache:** la memoria usata è di soli ~0,84 MB e la CPU di ~0,008 core. Una cache dovrebbe lavorare soprattutto in memoria, mentre qui si scrive molto su disco con pochissima RAM. - **Rischio di riempire il disco:** se le scritture finiscono su un volume temporaneo (`emptyDir`) o sul filesystem del container, ~43 GB al giorno possono esaurire lo spazio del nodo. In quel caso Kubernetes può forzare la chiusura del pod (*eviction*). **Cosa ti consiglio di fare:** 1. **Capire dove scrive:** controlla quali volumi usa il pod (`kubectl describe pod cache-app-5c6fd4f5cf-xxsz4`). Verifica che non sia un `emptyDir` senza `sizeLimit` o lo spazio scrivibile del container. 2. **Mettere un limite allo spazio temporaneo:** imposta `resources.limits.ephemeral-storage` nel Deployment, così il consumo non può crescere senza controllo. 3. **Controllare cosa fa l'app:** verifica il livello di dettaglio dei log, quanto spesso salva su disco e la rotazione dei log. 4. **Verificare se è costante:** questo dato è una singola misura. Per sapere se il rate è sempre così o se arriva a picchi, posso recuperare lo storico (`get_historical_metrics` su `cache-app`) o una serie Prometheus delle ultime ore.
Claude Sonnet 5.5	5.33 s	2764	466	3230	0.29	L'applicazione che scrive attivamente su disco è **cache-app**. - **Rate di scrittura:** circa **499.226 byte/s**, cioè circa **487,5 KiB/s** o **0,48 MiB/s**. - **Pod:** ne ha 1, `cache-app-5c6fd4f5cf-xxsz4`. Tutto il flusso è quindi generato da quel pod. - **Altre risorse:** usa circa 0,0077 core di CPU e circa 0,84 MB di memoria. Il traffico di rete in ingresso è 0. Le altre applicazioni non scrivono su disco: - **demo-app** ha `disk_write_bps` = 0 su entrambi i pod. - **worker-app** ha `disk_write_bps` = 0, anche se è l'applicazione più carica in CPU, con circa 0,59 core su 2 pod. **Osservazione:** cache-app scrive circa 0,5 MB/s con una memoria di meno di 1 MB e una CPU quasi nulla. Per una cache questo è insolito, perché potrebbe indicare persistenza su disco, come AOF o snapshot, oppure log o flush frequenti. Non ho dati storici per dire se il rate sia costante o a picchi. Se vuoi, posso recuperare lo storico con `get_historical_metrics`.
GPT-4o Mini	2.69 s	1317	94	1411	0.70	L'applicazione che genera un flusso attivo di scrittura su disco è la **cache-app**. Attualmente, sta scrivendo a una velocità di **499225.90 Bps** (byte per secondo) su disco. Le altre applicazioni, **demo-app** e **worker-app**, non mostrano attività di scrittura su disco, con un valore di scrittura pari a **0.0 Bps**.
GPT-4o	1.53 s	1317	50	1367	0.50	L'applicazione che genera un flusso attivo di scrittura su disco è la "cache-app". Il rate di scrittura su disco per questa applicazione è di 499,225.9 byte per secondo (Bps).

DOMANDA 3 A

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [3A]: Rileva quale carico di lavoro sta eseguendo operazioni continue di I/O in scrittura su filesystem e specifica la velocità di scrittura registrata.
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	3.88 ms	0	0	0	N/A	cache-app (499225.90 B/s)
Gemini 2.5 Flash	2.22 s	1632	168	1800	0.50	Il carico di lavoro "cache-app" sta eseguendo operazioni continue di I/O in scrittura su filesystem. La velocità di scrittura registrata è di 499225.9 Bps.
Gemini 3.6 Flash	11.67 s	1646	834	2480	0.80	In base alle metriche istantanee correnti, l'applicazione che sta eseguendo operazioni di I/O in scrittura su filesystem è **`cache-app`**. **Dettagli della velocità di scrittura registrata:** * **Carico di lavoro:** `cache-app` (1 Pod: `cache-app-5c6fd4f5cf-xxsz4`) * **Velocità di scrittura su disco (`disk_write_bps`):** **499.225,90 B/s** (pari a circa **499,23 KB/s** o **~0,50 MB/s**) Gli altri carichi di lavoro (`demo-app` e `worker-app`) registrano attualmente un valore di I/O in scrittura pari a **0,0 B/s**.
Claude Opus 5.5	13.44 s	2787	1277	4064	1.00	# Workload con scrittura continua su filesystem **Il workload è `cache-app`**, con un solo pod: `cache-app-5c6fd4f5cf-xxsz4`. | Metrica | Valore | |---|---| | **Velocità di scrittura su disco** | **499.225,90 B/s** | | Equivalente | ≈ 487,5 KiB/s ≈ 0,48 MiB/s ≈ 4 Mbit/s | | Volume stimato se il ritmo resta costante | ≈ 1,8 GB/ora, ≈ 43 GB/giorno | | CPU | 0,0077 core | | Memoria | 0,84 MB | | Traffico di rete in ingresso | 0 B/s | `demo-app` e `worker-app` scrivono **0 B/s** su disco, quindi non sono coinvolti. ## Cosa si nota - **Le scritture non arrivano da richieste esterne.** Il traffico in ingresso è 0, quindi le scritture sono generate dall'applicazione stessa (ad esempio log, file temporanei o salvataggio periodico della cache su disco). - **La memoria è stranamente bassa per una cache.** Con 0,84 MB, sembra che i dati vengano scritti su disco invece di restare in memoria. - **Il carico su CPU è minimo.** Questo è tipico di un'applicazione che passa il tempo ad aspettare il disco. ## Cosa consiglio 1. **Controllare dove scrive il pod.** Se usa lo spazio temporaneo del nodo (`emptyDir` o il filesystem del container), a circa 43 GB/giorno rischia di riempire il disco del nodo e far espellere i pod da Kubernetes. 2. **Impostare un limite allo spazio disco temporaneo.** Aggiungere `resources.requests/limits.ephemeral-storage` al Deployment, così il consumo resta sotto controllo. 3. **Verificare che le scritture siano volute.** Se si tratta di log troppo dettagliati o di un errore nella logica della cache, conviene correggerli prima di pensare allo scaling. 4. **Non aggiungere repliche (HPA) per ora.** CPU e memoria non sono sotto pressione, e più repliche moltiplicherebbero solo le scritture. **Nota:** questi dati vengono da un'unica rilevazione. Per confermare che le scritture siano davvero *continue* bisogna guardare l'andamento nel tempo. Posso farlo recuperando lo storico da MongoDB (`get_historical_metrics`) o con una query Prometheus su `container_fs_writes_bytes_total` per le ultime
Claude Sonnet 5.5	6.27 s	2787	576	3363	0.38	Il carico di lavoro che sta scrivendo in modo continuo sul filesystem è **cache-app**. - **Velocità di scrittura su disco:** circa **499.226 byte/s**, cioè circa **4875 KiB/s** (≈ 0,48 MiB/s). - **Pod coinvolto:** `cache-app-5c6fd4f5cf-xxsz4`. È l'unica replica dell'applicazione. Gli altri carichi risultano senza scritture su disco: - `demo-app` ha `disk_write_bps` pari a 0. - `worker-app` ha `disk_write_bps` pari a 0. **Nota:** lo snapshot è istantaneo, quindi non posso dire da quanto tempo la scrittura vada avanti. La richiesta parla di scrittura continua. Per confermarla posso consultare lo storico su MongoDB (`get_historical_metrics`) o una range query su Prometheus. **Osservazioni:** - `cache-app` usa poca CPU (~0,0077 core) e poca memoria (~0,84 MB). Con questi valori, una scrittura costante di ~0,5 MB/s è insolita per una cache. Potrebbe indicare persistenza attiva (ad esempio AOF o snapshot RDB), log molto verbosi o un flush continuo. - Se la cache non deve essere persistente, conviene verificare la configurazione di persistenza. Se invece lo è, controllate che il volume abbia IOPS e spazio sufficienti. - Con una sola replica non c'è ridondanza. Scalare in orizzontale non ha senso finché non è chiaro il motivo delle scritture.
GPT-4o Mini	2.83 s	1329	73	1402	0.20	L'applicazione che sta eseguendo operazioni continue di I/O in scrittura su filesystem è la **cache-app**, con una velocità di scrittura registrata di **499225.90 bps**. Le altre applicazioni, **demo-app** e **worker-app**, non mostrano attività di scrittura su disco al momento.
GPT-4o	2.03 s	1329	57	1386	0.50	Il carico di lavoro che sta eseguendo operazioni continue di I/O in scrittura su filesystem è il "cache-app". La velocità di scrittura registrata per questo carico di lavoro è di 499,225.9 byte per secondo (Bps).

DOMANDA 3B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [3B]: C'è un'applicazione che presenta throughput di scrittura su disco diverso da zero? Se presente, qual è e a che frequenza/tasso (rate) trasferisce dati verso lo sto
rage?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.19 ms	0	0	0	N/A	cache-app (499225.90 B/s)
Gemini 2.5 Flash	2.54 s	1643	204	1847	0.50	Sì, l'applicazione `cache-app` presenta un throughput di scrittura su disco diverso da zero. Il tasso di trasferimento dati verso lo storage è di circa **499225.9 Bps** (byte al secondo).
Gemini 3.6 Flash	11.31 s	1657	1028	2685	0.80	Sì, analizzando le metriche istantanee correnti, è presente un'applicazione con throughput di scrittura su disco (disk write) diverso da zero. ### **Applicazione identificata:** * **`cache-app`** (Pod: `cache-app-5c6fd4f5cf-xxsz4`) --- ### **Tasso / Rate di trasferimento verso lo storage (`disk_write_bps`):** * **In Byte al secondo:** `499.225,90 B/s` (~499,23 kB/s o 487,53 KiB/s) * **In Megabyte al secondo:** `~0,50 MB/s` (o 0,48 MiB/s) --- ### **Stato delle altre applicazioni:** * **`demo-app`**: `0,0 B/s` * **`worker-app`**: `0,0 B/s`
Claude Opus 5.5	15.42 s	2798	1515	4313	0.38	# Throughput di scrittura su disco Sì. Solo **`cache-app`** scrive su disco a un tasso diverso da zero. | Applicazione | Pod | `disk_write_bps` | |---|---|---| | **cache-app** | 1 (`cache-app-5c6fd4f5cf-xxsz4`) | **≈ 499.226 B/s** | | demo-app | 2 | 0 | | worker-app | 2 | 0 | ## Tasso di trasferimento verso lo storage (cache-app) | Unità | Valore | |---|---| | Byte/s | ~499.226 B/s | | KiB/s | ~487,5 KiB/s | | MB/s (decimale) | ~0,50 MB/s | | MiB/s | ~0,48 MiB/s | | Bit/s | ~3,99 Mbit/s | **Se il ritmo restasse costante:** - **Al minuto:** circa 30 MB - **All'ora:** circa 1,8 GB (1,67 GiB) - **Al giorno:** circa 43 GB (40 GiB) Queste stime presuppongono che il ritmo resti invariato. Il valore misurato è un tasso medio istantaneo, non uno storico. ## Cosa significa - **Il traffico in entrata è zero:** `network_rx_bps` = 0, eppure il pod scrive circa mezzo MB al secondo. Le scritture non vengono dalle richieste dei client. Probabilmente le genera l'applicazione stessa: log, salvataggio periodico della cache o un carico di test. - **Usa pochissime risorse:** CPU ≈ 0,0077 core, RAM ≈ 0,84 MB. È un carico che fa quasi solo I/O su disco. La CPU e la memoria non sono un problema. - **Il disco potrebbe riempirsi:** se scrive su un volume `emptyDir` o sul filesystem del container, circa 43 GB al giorno possono esaurire lo spazio locale del nodo. In quel caso Kubernetes rimuove il pod ("eviction" per `DiskPressure`). ## Raccomandazioni 1. **Capire dove scrive.** Controlla se usa un PVC, un `emptyDir` o il filesystem del container. Se non usa un PVC, imposta un limite di spazio con `resources.limits.ephemeral-storage` o passa a un volume persistente dimensionato. 2. **Verificare se lo spazio occupato cresce.** Se sono file nuovi, lo spazio aumenta. Se sovrascrive gli stessi file a rotazione, resta stabile. Per saperlo puoi controllare `container_fs_usage_bytes` per il pod. 3. **Confermare l'andamento nel tempo.** Posso recuperare lo storico di `cache-app` con `get_historical_metrics` per vedere se il ritmo è costante, a picchi o in aumento. 4. **Non scalare in orizzontale (HPA) per ora.** CPU e memoria sono bassissime e il problema riguarda lo storage. Aggiungere repliche moltiplicherebbe le scritture.

Claude Sonnet 5.5	6.88 s	2798	743	3541	0.38	Sì, c'è un'applicazione con scritture su disco diverse da zero: **cache-app**. - **Tasso di scrittura:** `disk_write_bps` ≈ **499.226 byte/s**, cioè circa **48751 KiB/s** o **0,476 MiB/s**. - **Pod coinvolti:** 1 pod, `cache-app-5c6fd4f5cf-xxsz4`. - **Altre metriche di cache-app:** la CPU è bassa (~0,0077 core, circa 7,7 millicore). La memoria è minima (~0,84 MB). Il traffico di rete in ingresso è pari a 0. Le altre applicazioni risultano senza scritture su disco: - **demo-app:** 0 B/s di scrittura, CPU a 0 e rete a 0. Sembra idle. - **worker-app:** 0 B/s di scrittura. È l'unica con un carico CPU rilevante (~0,59 core totali sui 2 pod). **Interpretazione:** - Il valore è un tasso istantaneo (byte al secondo), non una frequenza di operazioni. Lo snapshot non riporta il numero di operazioni di I/O al secondo (IOPS), quindi non posso dire a quale frequenza avvengono le singole scritture. - Per cache-app il profilo è insolito. Con rete a 0 e CPU quasi nulla, ci si aspetterebbe poca attività. Una scrittura costante di ~0,5 MB/s potrebbe indicare persistenza (ad esempio snapshot RDB o AOF) o logging intenso. - Il dato è puntuale. Non so se la scrittura sia continua o a picchi. **Raccomandazioni:** 1. Controllare la configurazione di persistenza di cache-app (RDB/AOF, `fsync`) e il volume di log. 2. Verificare lo storico con `get_historical_metrics` o una `prom_range` per capire se la scrittura è costante o periodica. Posso farlo se vuoi. 3. Se il flusso è continuo, verificare che il volume di destinazione abbia spazio e IOPS adeguati.
GPT-4o Mini	2.42 s	1334	62	1396	0.50	Sì, l'applicazione che presenta throughput di scrittura su disco diverso da zero è la **cache-app**. Il tasso di scrittura su disco (disk_write_bps) per la **cache-app** è di **499225.90 bps** (byte per secondo).
GPT-4o	2.42 s	1334	56	1390	0.50	Sì, l'applicazione "cache-app" presenta un throughput di scrittura su disco diverso da zero. Il tasso di trasferimento dei dati verso lo storage per "cache-app" è di circa 499,225.9 byte per secondo (Bps).

DOMANDA 4

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4]: Identifica l'applicazione con il maggior numero di repliche: elenca i relativi pod e indica quante repliche risultano attive.
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3988: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.18 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	4.81 s	1633	649	2282	0.42	Le applicazioni con il maggior numero di repliche sono **demo-app** e **worker-app**, entrambe con 2 repliche attive. Per l'applicazione **demo-app**: - Pods: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` - Repliche attive: 2 Per l'applicazione **worker-app**: - Pods: `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` - Repliche attive: 2
Gemini 3.6 Flash	10.91 s	1647	946	2593	0.36	Dall'analisi dell'istantanea fornita, le applicazioni con il maggior numero di repliche attive sono **`demo-app`** e **`worker-app`**, entrambe con **2 repliche attive** (a fronte di 1 sola replica per `cache-app`). Di seguito il dettaglio per ciascuna applicazione con il numero massimo di repliche: 1. **`demo-app`** - **Repliche attive:** 2 - **Pod:** - `demo-app-7958c64697-2qg9s` - `demo-app-7958c64697-4rzjl` 2. **`worker-app`** - **Repliche attive:** 2 - **Pod:** - `worker-app-84cf4dd446-9kj75` - `worker-app-84cf4dd446-ftlln` --- *Nota SRE:* `demo-app` risulta attualmente in uno stato del tutto inattivo (`cpu_cores`: 0.0, I/O e rete nulli), pertanto potrebbe rappresentare un'opportunità di ridimensionamento/downscaling se l'inattività persiste.

Claude Opus 5.5	9.36 s	2775	737	3512	0.72	Due applicazioni sono **a pari merito**, con **2 repliche attive** ciascuna, quindi non c'è un'unica app con più repliche. | Applicazione | Repliche attive | Pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s` `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75` `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | **Cosa dicono le metriche sulle due app con 2 repliche:** - **demo-app sembra inattiva:** CPU a 0, rete a 0 e disco a 0. Usa solo circa 15 MB di memoria. Ha 2 repliche ma in questo momento non fa nessun lavoro. - **Cosa fare:** se il dato si conferma nel tempo, si può scendere a 1 replica, oppure usare un HPA (scaling automatico) con `minReplicas: 1`. Prima di farlo, controllate che l'app non debba restare sempre disponibile e che non abbia una policy che vieta di scendere sotto 2 pod (PodDisruptionBudget). - **worker-app è quella che lavora di più:** usa circa 0,53 core in totale, cioè circa 0,26 core per pod. La memoria è bassa, circa 5,3 MB. Il numero di repliche sembra giusto per il carico. - **Cosa fare:** un HPA basato sulla CPU avrebbe senso. Per decidere la soglia servono però i `requests` CPU dei pod, che non sono nello snapshot. **Attenzione:** questi valori sono una sola rilevazione. Prima di ridurre le repliche di demo-app conviene guardare l'andamento nel tempo, per esempio le metriche delle ultime 24 ore. Se volete, posso recuperarle io.

Claude Sonnet 5.5	6.55 s	2775	563	3338	0.00	Dallo snapshot corrente ho ricavato la risposta senza invocare tool. Ci sono due applicazioni con il numero massimo di repliche, **demo-app** e **worker-app**, che hanno **2 repliche attive** ciascuna. Non c'è un'unica applicazione con più repliche delle altre. cache-app ne ha 1. **demo-app** (2 repliche attive) - `demo-app-7958c64697-2qg9s` - `demo-app-7958c64697-4rzjl` **worker-app** (2 repliche attive) - `worker-app-84cf4dd446-9kj75` - `worker-app-84cf4dd446-ftlln` **Osservazioni sui carichi (dalle metriche istantanee)** - **worker-app** usa 0,53 core di CPU in totale, circa 0,26 core per pod, con memoria molto bassa (5,3 MB). È l'unico carico CPU significativo. Le 2 repliche sono ragionevoli, e un HPA su CPU potrebbe servire se il carico cresce. - **demo-app** ha CPU, disco e rete a zero, con 15 MB di memoria. Sembra idle. Se questo stato è confermato anche dallo storico, si può ridurre a 1 replica. - **cache-app** ha 1 replica, 0,018 core di CPU e circa 466 KB/s di scritture su disco. Il carico è basso e non richiede scaling. Se vuoi, posso verificare con lo storico (`get_historical_metrics`) che l'inattività di demo-app e il carico di worker-app siano costanti nel tempo.
GPT-4o Mini	2.53 s	1325	68	1393	0.70	L'applicazione con il maggior numero di repliche è **demo-app**, che ha attualmente **2 repliche attive**. I pod relativi a questa applicazione sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl
GPT-4o	2.52 s	1325	68	1393	0.50	L'applicazione con il maggior numero di repliche attive è la "demo-app", che ha 2 repliche attive. I pod relativi a questa applicazione sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

[BILLING MONITOR] Spesa cumulata OpenAI: $1.5045 (mini: $0.0701 | 4o: $1.4344)

DOMANDA 4 A

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [4A]: Determina qual è l'applicazione con il livello di scalabilità più alto: riporta il conteggio delle repliche in esecuzione e i nomi puntuali di ciascun pod associato.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 17:35:31] INFO     Processing request of type CallToolRequest                                                
```

Claude Opus 5.5	8.50 s	2794	751	3545	0.00	**Non c'è un'unica applicazione più scalata: demo-app e worker-app sono a pari merito con 2 repliche ciascuna.** cache-app ne ha una sola. ### Repliche e pod per applicazione | Applicazione | Repliche | Pod | |---|---|---| | **demo-app** | **2** | `demo-app-7958c64697-2qg9s` `demo-app-7958c64697-4rzjl` | | **worker-app** | **2** | `worker-app-84cf4dd446-9kj75` `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | ### Cosa dicono le metriche Le due applicazioni hanno lo stesso numero di repliche, ma il carico è molto diverso: - **worker-app**: usa 0,527 core in totale, circa 0,26 core per pod. Le 2 repliche sono giustificate. Se si vuole gestire automaticamente i picchi, conviene un HPA basato sulla CPU, con minimo 2 repliche e soglia intorno al 70% della CPU richiesta. - **demo-app**: usa 0 core e circa 15 MB di memoria, senza traffico di rete né scritture su disco. È **inattiva**: la seconda replica oggi non serve. Si può scendere a 1 replica, a meno che le 2 repliche non servano per l'alta disponibilità. In alternativa si può impostare un HPA con minimo 1 replica. - **cache-app**: ha una sola replica e quindi è un singolo punto di guasto. Usa poca CPU (0,018 core) e scrive su disco in modo continuo, circa 455 KB/s. Se la cache è critica, valuta una seconda replica per la ridondanza. Questo confronto si basa solo sulla situazione attuale. Per confermare che demo-app è sempre inattiva, si può consultare lo storico su MongoDB (per esempio le ultime 24 ore). Se ti serve, posso farlo.
Claude Sonnet 5.5	7.28 s	2794	746	3540	0.00	**Risultato: c'è un pareggio.** Due applicazioni hanno il numero massimo di repliche in esecuzione, cioè 2. | Applicazione | Repliche in esecuzione | Pod | |---|---|---| | **demo-app** | 2 | `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl` | | **worker-app** | 2 | `worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln` | | cache-app | 1 | `cache-app-5c6fd4f5cf-xxsz4` | **Contesto dalle metriche correnti** - **worker-app** è l'unica delle due con carico reale. Usa circa 0,53 core di CPU in totale, cioè circa 0,26 core per pod, e 5,3 MB di memoria. Le 2 repliche sono quindi giustificate. Se la CPU cresce, è la candidata naturale per un HPA. - **demo-app** ha 2 repliche ma è idle. CPU, disco e rete sono a 0 e la memoria è di circa 15 MB. Se il traffico resta assente, puoi ridurla a 1 replica e risparmiare risorse. Questo vale solo se non ti serve ridondanza per l'alta disponibilità. - **cache-app** ha una sola replica, con CPU di circa 0,018 core e scritture su disco di circa 466 KB/s. Non mostra segni di saturazione. Questa analisi si basa solo sullo snapshot istantaneo. Per confermare se demo-app è idle in modo stabile o se worker-app ha picchi, posso recuperare lo storico delle metriche.
GPT-4o Mini	2.81 s	1336	71	1407	0.50	L'applicazione con il livello di scalabilità più alto è la **demo-app**, che attualmente ha **2 repliche** in esecuzione. I nomi dei pod associati sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl
GPT-4o	2.81 s	1336	220	1556	0.80	L'applicazione con il livello di scalabilità più alto, in termini di numero di repliche attualmente in esecuzione, è la "demo-app". Ecco i dettagli: - **demo-app**: - Numero di repliche: 2 - Nomi dei pod: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl Le altre applicazioni hanno il seguente conteggio di repliche: - **cache-app**: 1 replica - Nome del pod: cache-app-5c6fd4f5cf-xxsz4 - **worker-app**: 2 repliche - Nomi dei pod: - worker-app-84cf4dd446-9kj75 - worker-app-84cf4dd446-ftlln Sia "demo-app" che "worker-app" hanno 2 repliche, ma "demo-app" è stata menzionata per prima.

DOMANDA 4 B

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.59 ms	0	0	0	N/A	App: demo-app | Repliche: 2 | Pods: ['demo-app-7958c64697-2qg9s', 'demo-app-7958c64697-4rzjl']
Gemini 2.5 Flash	2.03 s	1639	158	1797	0.70	Il servizio con il pool di pod più numeroso è **demo-app**, con un totale di 2 istanze operative. Gli identificatori dei pod sono: `demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`.
Gemini 3.6 Flash	8.92 s	1653	989	2642	0.00	Dall'analisi dell'istantanea fornita, i servizi con il pool di pod più numeroso sono **`demo-app`** e **`worker-app`**, entrambi a pari merito con un totale di **2 istanze operative** ciascuno. Di seguito il dettaglio dei pod per ciascuno dei due servizi: --- ### 1. **`demo-app`** * **Totale istanze operative:** 2 * **Identificatori dei pod:** * `demo-app-7958c64697-2qg9s` * `demo-app-7958c64697-4rzjl` --- ### 2. **`worker-app`** * **Totale istanze operative:** 2 * **Identificatori dei pod:** * `worker-app-84cf4dd446-9kj75` * `worker-app-84cf4dd446-ftlln` --- *(Nota di confronto: il servizio `cache-app` conta invece 1 sola istanza operativa: `cache-app-5c6fd4f5cf-xxsz4`)*
Claude Opus 5.5	7.10 s	2789	640	3429	0.00	**Nessun servizio ha da solo il pool più numeroso.** demo-app e worker-app sono a pari merito con 2 pod ciascuno, mentre cache-app ne ha uno solo. | Servizio | Pod | Istanze | |---|---|---| | **demo-app** | `demo-app-7958c64697-2qg9s` `demo-app-7958c64697-4rzjl` | **2** | | **worker-app** | `worker-app-84cf4dd446-9kj75` `worker-app-84cf4dd446-ftlln` | **2** | | cache-app | `cache-app-5c6fd4f5cf-xxsz4` | 1 | **In totale ci sono 5 pod in esecuzione nel cluster.** Di questi, 4 appartengono ai due servizi con più pod. **Note:** - **demo-app sembra inattivo:** usa 0 CPU e non genera traffico di rete né scritture su disco, con circa 15 MB di memoria. Se non è un servizio di riserva o in attesa di traffico, si può valutare di **ridurlo a 1 replica**. - **worker-app è l'unico servizio sotto carico:** usa circa 0,53 core (circa 0,26 per pod). Questi due pod sono quindi utilizzati in modo adeguato. Per sapere se si avvicinano alla saturazione bisognerebbe conoscere i limiti di CPU configurati, che questi dati non riportano. - **cache-app non ha repliche:** ha un solo pod, quindi se si ferma il servizio non è disponibile. Inoltre scrive su disco circa 466 KB/s senza ricevere traffico di rete. Se il servizio è importante, conviene aggiungere almeno una seconda replica.
Claude Sonnet 5.5	5.24 s	2789	581	3370	0.00	Dallo snapshot corrente, i servizi con il pool di pod più numeroso sono **demo-app** e **worker-app**. Sono alla pari, con 2 repliche ciascuno. **demo-app (2 pod)** - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl **worker-app (2 pod)** - worker-app-84cf4dd446-9kj75 - worker-app-84cf4dd446-ftlln **cache-app (1 pod, per confronto)** - cache-app-5c6fd4f5cf-xxsz4 **Totale istanze operative: 5 pod** (2 + 2 + 1). **Osservazioni sui carichi** - **worker-app** è il servizio più attivo. Usa circa 0,53 core in totale, cioè circa 0,26 core per pod. La memoria è molto bassa (circa 5,3 MB). Se il carico cresce, è il candidato naturale per un HPA basato sulla CPU. - **demo-app** sembra idle. La CPU è a 0 e non c'è traffico di rete né di disco, anche se ha 2 repliche e circa 15 MB di memoria. Si può valutare di ridurla a 1 replica, dopo aver verificato lo storico con `get_historical_metrics` per escludere un calo temporaneo. - **cache-app** ha 1 sola replica e un carico CPU contenuto (circa 0,018 core). Scrive su disco circa 466 KB/s e non riceve traffico di rete. Con una sola replica è un single point of failure. Se vuoi, posso verificare lo storico delle metriche per confermare questi andamenti.
GPT-4o Mini	3.17 s	1329	91	1420	0.70	Il servizio che registra la presenza del pool di pod più numeroso è il **demo-app**, che ha attualmente **2** istanze operative. Ecco l'elenco dei singoli identificatori dei pod per il servizio demo-app: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl Totale delle istanze operative: **2**.
GPT-4o	1.91 s	1329	69	1398	0.70	Il servizio "demo-app" registra la presenza del pool di pod più numeroso, con un totale di 2 istanze operative. Gli identificatori dei pod per "demo-app" sono: - demo-app-7958c64697-2qg9s - demo-app-7958c64697-4rzjl

DOMANDA 5

```
--------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [8]: C'è qualche applicazione che presenta traffico di rete in ricezione superiore a 10 KB/s? Se sì, quale e a quale rate?
--------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.64 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	2.34 s	1637	272	1909	0.50	Nessuna delle applicazioni presenta traffico di rete in ricezione superiore a 10 KB/s. Tutte le applicazioni hanno un 'network_rx_bps' pari a 0.0.
Gemini 3.6 Flash	7.50 s	1651	1086	2737	0.70	In base all'istantanea delle metriche correnti, **nessuna applicazione** presenta un traffico di rete in ricezione (`network_rx_bps`) superiore a 10 KB/s. Dettaglio del traffico di rete in ricezione per applicazione: * **`cache-app`**: 0,0 B/s (0 KB/s) * **`demo-app`**: 0,0 B/s (0 KB/s) * **`worker-app`**: 0,0 B/s (0 KB/s) Tutti i carichi di lavoro risultano attualmente a 0 B/s per quanto riguarda il traffico di rete inbound.
Claude Opus 5.5	8.41 s	2781	722	3503	0.90	**No, nessuna applicazione supera i 10 KB/s di traffico di rete in ricezione.** Nello snapshot attuale tutte e tre le applicazioni ricevono 0 B/s. | Applicazione | Pod | network_rx_bps | Supera 10 KB/s? | |---|---|---|---| | cache-app | 1 | 0,0 B/s | No | | demo-app | 2 | 0,0 B/s | No | | worker-app | 2 | 0,0 B/s | No | **Possibile problema di raccolta dati:** è strano che tutte le app risultino esattamente a zero. In particolare: - **worker-app** usa circa 0,53 core di CPU. - **cache-app** scrive su disco circa 455 KiB/s (466.431 B/s). Anche un servizio senza traffico applicativo di solito riceve almeno qualche byte (health check, probe, DNS). Quindi lo zero potrebbe non essere reale. Le cause possibili sono: 1. Le app comunicano davvero poco o nulla in rete (ad esempio worker che lavorano in locale). 2. La metrica di rete non viene raccolta correttamente. Un caso frequente: `container_network_receive_bytes_total` di cAdvisor è registrata solo sul container "POD" (pause) o senza l'etichetta `container`. In quel caso una query che filtra per container applicativo restituisce 0. **Cosa consiglio:** se ti serve la certezza, posso interrogare Prometheus a livello di pod, ad esempio con `sum by (pod) (rate(container_network_receive_bytes_total{namespace="default"}[5m]))`, e vedere se ci sono byte ricevuti che lo snapshot non mostra. Vuoi che la esegua?

Claude Sonnet 5.5	5.10 s	2781	510	3291	0.80	Nessuna applicazione ha traffico di rete in ricezione superiore a 10 KB/s. Dallo snapshot corrente il valore di `network_rx_bps` è 0.0 B/s per tutte e tre: | Applicazione | Pod | network_rx_bps | |---|---|---| | cache-app | 1 | 0.0 | | demo-app | 2 | 0.0 | | worker-app | 2 | 0.0 | Il valore 0.0 su tutte le app può dipendere da due cose. Può essere assenza reale di traffico, oppure la metrica di rete può non essere campionata o esposta per questi pod. Lo snapshot non permette di distinguere i due casi. Altre osservazioni dallo snapshot: - **cache-app** scrive su disco a circa 466 KB/s (`disk_write_bps` = 466.431). È l'unico carico di I/O rilevante, ma riguarda la scrittura su disco e non la ricezione di rete. - **worker-app** usa circa 0,53 core di CPU in totale su 2 pod, con memoria molto bassa (~5,3 MB). - **demo-app** risulta idle: CPU a 0, rete a 0 e disco a 0. Posso verificare con una query Prometheus diretta (ad esempio `container_network_receive_bytes_total`) oppure con lo storico, per vedere se ci sono stati picchi recenti. Fammi sapere se vuoi che lo faccia.
GPT-4o Mini	2.98 s	1324	116	1440	0.80	Dall'analisi delle metriche istantanee fornite, nessuna delle applicazioni ha un traffico di rete in ricezione (network_rx_bps) superiore a 10 KB/s. Ecco i dettagli: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Pertanto, non ci sono applicazioni che soddisfano il criterio di traffico di rete in ricezione superiore a 10 KB/s.
GPT-4o	3.73 s	1324	113	1437	0.70	Dall'analisi delle metriche istantanee fornite, nessuna delle applicazioni presenta traffico di rete in ricezione superiore a 10 KB/s. Ecco i dettagli: - **cache-app**: 0.0 B/s - **demo-app**: 0.0 B/s - **worker-app**: 0.0 B/s Tutte le applicazioni hanno un traffico di rete in ricezione pari a 0 B/s, quindi nessuna supera la soglia di 10 KB/s.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.5193 (mini: $0.0709 | 4o: $1.4484)
```

DOMANDA 5 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	0.54 ms	0	0	0	N/A	Nessuna applicazione supera 10 KB/s in ricezione
Gemini 2.5 Flash	2.90 s	1649	341	1990	0.70	Dalle metriche istantanee correnti, nessuna delle applicazioni utente (`cache-app`, `demo-app`, `worker-app`) presenta un throughput di ingress (network RX) che eccede la soglia di 10 KB/s (10240 B/s). Tutte le applicazioni mostrano un `network_rx_bps` pari a 0.0.
Gemini 3.6 Flash	32.00 s	26427	1482	27909	0.80	Dall'analisi delle metriche istantanee correnti e delle rilevazioni in tempo reale da Prometheus/database storico: ### Esito della verifica **Nessuna applicazione utente eccede la soglia di 10 KB/s in ingress (network RX).** ### Dettaglio del throughput Network RX attuale per i carichi utente: * **`cache-app`**: `0.0 B/s` (`0.0 KB/s`) * **`demo-app`**: `0.0 B/s` (`0.0 KB/s`) * **`worker-app`**: `0.0 B/s` (`0.0 KB/s`) Tutti i carichi utente si trovano attualmente a 0 B/s di traffico di rete in ingresso, ben al di sotto della soglia specificata di 10 KB/s.

Claude Opus 5.5	9.49 s	2797	856	3653	0.95	# Ingress di rete (RX) rispetto alla soglia di 10 KB/s
						
						Risultato: nessuna app supera la soglia. In base allo snapshot attuale, tutte e tre le app hanno un traffico in ingresso di 0 B/s. 10 KB/s corrispondono a circa 10.240 B/s.
						
						| App | Pod | network_rx_bps | Supera 10 KB/s? |
						|---|---|---|---|
						| cache-app | 1 | 0,0 B/s | ❌ No |
						| demo-app | 2 | 0,0 B/s | ❌ No |
						| worker-app | 2 | 0,0 B/s | ❌ No |
						
						## ⚠️ Lo zero potrebbe essere un errore di misura
						Che tutte le app segnino esattamente 0 B/s è insolito. Le app risultano comunque attive:
						- **worker-app** usa circa 0,53 core di CPU.
						- **cache-app** scrive su disco circa 455 KB/s.
						
						Quindi lo zero potrebbe non indicare traffico assente, ma un problema nella raccolta del dato. Una causa frequente: in cAdvisor la metrica `container_network_receive_bytes_total` viene registrata per l'intero pod (con `container=""` o `container="POD"`) e non per i singoli container. Se la query filtra per container, il valore risulta sempre 0.
						
						## Prossimo passo
						Per verificare il dato, l'operatore può far eseguire su Prometheus una query a livello di pod, ad esempio:
						``promql
						sum by (pod)
						(rate(container_network_receive_bytes_total{namespace="default", pod=~"(cache|demo|worker)-app-.*"}[5m]))
						``
						- **Se anche questa restituisce valori vicini a 0:** le app non ricevono traffico di rete rilevante e il risultato sopra è valido.
						- **Se restituisce valori più alti:** conviene correggere la query usata per lo snapshot e rifare il controllo sulla soglia.

Claude Sonnet 5.5	5.76 s	2797	583	3380	0.95	Nessun carico di lavoro supera la soglia di 10 KB/s di ingress (network RX). Ho usato lo snapshot corrente, quindi non ho invocato nessun tool. | App | Pod | network_rx_bps | Supera i 10 KB/s? | |---|---|---|---| | cache-app | 1 | 0,0 B/s | No | | demo-app | 2 | 0,0 B/s | No | | worker-app | 2 | 0,0 B/s | No | Il valore massimo di RX è 0 B/s, quindi i tre carichi sono ben sotto la soglia (10 KB/s = 10.240 B/s). Altre osservazioni dallo snapshot: - **worker-app** usa 0,53 core di CPU in totale, circa 0,26 core per pod, senza traffico di rete né scritture su disco. Sembra un carico di calcolo puro. Se la CPU continua a salire, si può valutare un HPA basato sulla CPU. - **cache-app** scrive su disco a circa 466 KB/s (≈455 KiB/s) con 0 di RX di rete. Le scritture non arrivano quindi da traffico in ingresso. Possono essere dovute a persistenza (snapshot o AOF) o a un processo interno. Con una sola replica non c'è ridondanza. - **demo-app** ha CPU, rete e disco a 0, con 2 repliche. Risulta idle, quindi si può valutare di ridurla a 1 replica, se non serve alta disponibilità. Se vuoi, posso verificare con `prom_query` o con lo storico su MongoDB se in passato ci sono stati picchi di RX sopra soglia.
GPT-4o Mini	2.45 s	1336	142	1478	0.70	Analizzando le metriche istantanee fornite, possiamo osservare il throughput di ingress (network RX) per ciascuna applicazione: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Tutte le applicazioni hanno un throughput di ingress pari a 0 bps, che è ben al di sotto della soglia di 10 KB/s (che corrisponde a 80 Kbps). Pertanto, non ci sono carichi di lavoro utente con un throughput di ingress che eccede la soglia di 10 KB/s.
GPT-4o	2.17 s	1336	122	1458	0.70	Dall'analisi delle metriche istantanee fornite, possiamo osservare i seguenti valori di throughput di ingress (network RX) per le applicazioni: - **cache-app**: 0.0 B/s - **demo-app**: 0.0 B/s - **worker-app**: 0.0 B/s Nessuna delle applicazioni attualmente supera la soglia di 10 KB/s (10,240 B/s) per il throughput di ingress. Pertanto, non ci sono applicazioni che eccedono questa soglia al momento.

```
[BILLING MONITOR] Spesa cumulata OpenAI: $1.5241 (mini: $0.0711 | 4o: $1.4530)
```

DOMANDA 5 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [5B]: Controlla il traffico di rete in entrata: qualche servizio sfora il limite nominale di 10 kilobyte al secondo? Se rilevato, quantifica il bitrate esatto.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
[10/30/26 17:42:40] INFO     Processing request of type CallToolRequest                                                
```

Claude Opus 5.5	11.86 s	2795	1067	3862	0.95	# Traffico di rete in entrata: verifica della soglia di 10 KB/s **Nessun servizio supera la soglia.** Tutti e tre i deployment ricevono 0 byte al secondo, quindi non c'è un bitrate da misurare. | Applicazione | Pod | Traffico in entrata (B/s) | Bitrate | Soglia 10 KB/s (10.240 B/s ≈ 81,9 kbit/s) | |---|---|---|---|---| | cache-app | 1 | 0,0 | 0 bit/s | ✅ Sotto la soglia | | demo-app | 2 | 0,0 | 0 bit/s | ✅ Sotto la soglia | | worker-app | 2 | 0,0 | 0 bit/s | ✅ Sotto la soglia | ## Note 1. **Il traffico di cache-app è su disco, non in rete.** cache-app scrive su disco a circa **466.431 B/s (≈ 455 KB/s, ≈ 3,73 Mbit/s)**. È un I/O di disco, quindi non conta per la soglia di rete. Però è 45 volte la soglia di 10 KB/s: se il limite valesse anche per l'I/O in generale, cache-app sarebbe l'unico servizio a superarlo. 2. **Zero ovunque è un valore sospetto.** worker-app sta usando circa 0,53 core di CPU senza ricevere niente dalla rete. Può essere normale se fa calcoli interni o legge da una coda locale. Però la rete a zero su tutti i servizi potrebbe anche indicare un problema di raccolta. Per esempio: - la metrica di rete potrebbe essere raccolta sull'interfaccia sbagliata; - con alcune configurazioni di rete del cluster il traffico finisce sul pod "pause" e non sul container. 3. **demo-app sembra inattiva.** Ha CPU, traffico di rete e scritture su disco tutti a zero, e solo circa 15 MB di memoria. Se è così anche nel tempo, si può pensare di ridurla a una sola replica. ## Prossimo passo consigliato Per escludere un problema di raccolta, posso interrogare Prometheus direttamente sulla metrica di rete in entrata per ogni pod, ad esempio `rate(container_network_receive_bytes_total{namespace="default"}[5m])`. Posso anche controllare nello storico su MongoDB se il traffico è sempre stato a zero. Dimmi tu se procedere.
Claude Sonnet 5.5	6.94 s	2795	736	3531	0.95	**Nessun servizio supera il limite di 10 KB/s in ingresso.** Il limite equivale a circa 10.000 B/s, cioè 80 kbit/s. Se si usa 1 KB = 1024 B, sono 10.240 B/s, cioè 81,9 kbit/s. Dallo snapshot corrente, `network_rx_bps` vale: | Applicazione | Pod | network_rx_bps | Rispetto al limite | |---|---|---|---| | cache-app | 1 | 0,0 | 0% | | demo-app | 2 | 0,0 | 0% | | worker-app | 2 | 0,0 | 0% | Poiché non c'è nessuno sforamento, non c'è un bitrate da quantificare. - **Traffico a zero:** tutti i servizi risultano senza traffico in ingresso al momento del campionamento. Questo è un valore istantaneo e non esclude picchi precedenti. - **demo-app:** ha 2 repliche, nessuna attività di CPU, rete o disco e 15 MB di memoria. Sembra idle, quindi si può valutare di ridurla a 1 replica. - **worker-app:** usa 0,53 core ma non riceve traffico di rete. Il carico è probabilmente interno, ad esempio elaborazione batch o da coda. - **cache-app:** scrive su disco a circa 466 KB/s, pari a 3,7 Mbit/s. Non c'è traffico di rete, quindi la scrittura potrebbe essere persistenza locale. Se vuoi escludere picchi nel passato, posso analizzare lo storico con `get_historical_metrics` o con Prometheus.
GPT-4o Mini	2.65 s	1335	135	1470	0.80	Analizzando le metriche istantanee fornite, possiamo osservare il traffico di rete in entrata (network_rx_bps) per ciascuna applicazione: - **cache-app**: 0.0 bps - **demo-app**: 0.0 bps - **worker-app**: 0.0 bps Nessuna delle applicazioni ha registrato traffico di rete in entrata, quindi nessun servizio supera il limite nominale di 10 kilobyte al secondo. In sintesi, non ci sono servizi che sforano il limite di traffico di rete in entrata.
GPT-4o	2.24 s	1335	113	1448	0.50	Dall'analisi delle metriche istantanee fornite, possiamo osservare che il traffico di rete in entrata (network_rx_bps) per tutte le applicazioni è pari a 0.0 byte al secondo. Questo significa che nessun servizio sta attualmente superando il limite nominale di 10 kilobyte al secondo per il traffico di rete in entrata. In sintesi, non ci sono servizi che sforano il limite di traffico di rete in entrata di 10 kilobyte al secondo.

DOMANDA 6

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [6]: Calcola un indice di interferenza normalizzato per ogni applicazione con la formula: 50% CPU, 25% Memoria (su 128MB) e 25% scrittura su Disco (su 1 MB/s). Quale applicazione ottiene il punteggio più alto?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	5.83 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2737) | Tutti: {'cache-app': 0.1244, 'demo-app': 0.0293, 'worker-app': 0.2737}
Gemini 2.5 Flash	17.87 s	1669	4385	6054	0.32	Per calcolare l'indice di interferenza normalizzato per ogni applicazione, useremo la seguente formula: Indice di Interferenza = ((CPU_cores / 0.5) * 0.5) + ((Memory_MB / 128) * 0.25) + ((Disk_write_BPS / (1 * 1024 * 1024)) * 0.25) Dove: * `CPU_cores` è l'utilizzo della CPU in core. Il valore di normalizzazione è 0.5 core. * `Memory_MB` è l'utilizzo della memoria in megabyte. Il valore di normalizzazione è 128 MB. * `Disk_write_BPS` è la scrittura su disco in byte al secondo. Il valore di normalizzazione è 1 MB/s (che equivale a 1 * 1024 * 1024 byte/s). Calcoliamo l'indice per ogni applicazione: **1. cache-app** * CPU: 0.018407413123207O9 core * Memoria: 2.01953125 MB * Scrittura su Disco: 466431.46083735797 B/s Indice = ((0.018407413123207O9 / 0.5) * 0.5) + ((2.01953125 / 128) * 0.25) + ((466431.46083735797 / 1048576) * 0.25) Indice = (0.036814826246414I8 * 0.5) + (0.015777587890625 * 0.25) + (0.444833875 * 0.25) Indice = 0.018407413123207O9 + 0.003944396972656250 + 0.111209687S **Indice cache-app = 0.133561497595863344** **2. demo-app** * CPU: 0.0 core * Memoria: 14.984375 MB * Scrittura su Disco: 0.0 B/s Indice = ((0.0 / 0.5) * 0.5) + ((14.984375 / 128) * 0.25) + ((0.0 / 1048576) * 0.25) Indice = (0.0 * 0.5) + (0.1170654296875 * 0.25) + (0.0 * 0.25) Indice = 0.0 + 0.029266357421875 + 0.0 **Indice demo-app = 0.029266357421875** **3. worker-app** * CPU: 0.5265049996160318 core * Memoria: 5.328125 MB * Scrittura su Disco: 0.0 B/s Indice = ((0.5265049996160318 / 0.5) * 0.5) + ((5.328125 / 128) * 0.25) + ((0.0 / 1048576) * 0.25) Indice = (1.0530099992320636 * 0.5) + (0.0416259765625 * 0.25) + (0.0 * 0.25) Indice = 0.5265049996160318 + 0.0104064941406250 + 0.0
						* Memoria: 5.328125 MB * Scrittura su Disco: 0.0 B/s Indice = ((0.5265049996160318 / 0.5) * 0.5) + ((5.328125 / 128) * 0.25) + ((0.0 / 1048576) * 0.25) Indice = (1.0530099992320636 * 0.5) + (0.0416259765625 * 0.25) + (0.0 * 0.25) Indice = 0.5265049996160318 + 0.0104064941406250 + 0.0 **Indice worker-app = 0.536911493756656S** **Riassunto degli indici di interferenza:** * **cache-app**: 0.13356 * **demo-app**: 0.02927 * **worker-app**: 0.53691 L'applicazione che ottiene il punteggio più alto è **worker-app**.
Gemini 3.6 Flash	41.58 s	1683	3554	5237	0.72	Sulla base delle metriche istantanee fornite, calcoliamo l'**Indice di Interferenza Normalizzato** per ciascuna applicazione utilizzando la formula richiesta: $$\text{Indice} = (50\% \times \text{CPU (cores)}) + \left(25\% \times \frac{\text{Memoria (MB)}}{128\text{ MB}}\right) + \left(25\% \times \frac{\text{Scrittura Disco (B/s)}}{1.000.000\text{ B/s}}\right)$$ --- ### 1. **worker-app** * **CPU (50%)**: $0.526505 \times 0.50 = \mathbf{0.263253}$ * **Memoria (25%)**: $\frac{5.328125\text{ MB}}{128\text{ MB}} \times 0.25 = 0.041626 \times 0.25 = \mathbf{0.010406}$ * **Disco (25%)**: $0.0 \times 0.25 = \mathbf{0.0}$ * **Punteggio Totale**: **0.273659** (o **27.37%**) --- ### 2. **cache-app** * **CPU (50%)**: $0.018407 \times 0.50 = \mathbf{0.009204}$ * **Memoria (25%)**: $\frac{2.019531\text{ MB}}{128\text{ MB}} \times 0.25 = 0.015778 \times 0.25 = \mathbf{0.003944}$ * **Disco (25%)**: $\frac{466.431,46\text{ B/s}}{1.000.000\text{ B/s}} \times 0.25 = 0.466431 \times 0.25 = \mathbf{0.116608}$ *(oppure 0.111206 se calcolato su $1\text{ MiB/s} = 1.048.576\text{ B/s}$)* * **Punteggio Totale**: **0.129756** (o **12.98%**) --- ### 3. **demo-app** * **CPU (50%)**: $0.0 \times 0.50 = \mathbf{0.0}$ * **Memoria (25%)**: $\frac{14.984375\text{ MB}}{128\text{ MB}} \times 0.25 = 0.117065 \times 0.25 = \mathbf{0.029266}$ * **Disco (25%)**: $0.0 \times 0.25 = \mathbf{0.0}$ * **Punteggio Totale**: **0.029266** (o **2.93%**) --- ### 🏆 **Risultato Finale** L'applicazione che ottiene il **punteggio più alto** è **`worker-app`** con circa **0.2737 (27.37%)**, guidata dall'elevato consumo di CPU (≈ 0.5265 cores).

Claude Opus 5.5	16.70 s	2823	1548	4371	0.95	# Indice di interferenza normalizzato **Formula:** `Indice = 0,50 × (CPU / 1 core) + 0,25 × (Mem / 128 MB) + 0,25 × (Disk write / 1 MB/s)` **Ipotesi:** - La formula non indica un riferimento per la CPU, quindi ho usato **1 core**. - Per il disco ho considerato 1 MB/s = 1.048.576 B/s, cioè la stessa unità binaria usata per i MB di memoria. - Ogni componente è limitata a un massimo di 1. ## Calcolo | App | CPU (×0,50) | Memoria (×0,25) | Disco (×0,25) | **Indice** | |---|---|---|---|---| | **worker-app** | 0,5265 → **0,2633** | 5,33/128 = 0,0416 → **0,0104** | 0 → **0** | **0,2737 (27,4%)** | | **cache-app** | 0,0184 → **0,0092** | 2,02/128 = 0,0158 → **0,0039** | 466.431/1.048.576 = 0,4448 → **0,1112** | **0,1244 (12,4%)** | | **demo-app** | 0 → **0** | 14,98/128 = 0,1171 → **0,0293** | 0 → **0** | **0,0293 (2,9%)** | Se si usa 1 MB/s = 1.000.000 B/s, cache-app sale a circa **0,1298**. La classifica non cambia. ## Risultato: worker-app ha il punteggio più alto (≈ 0,274) **worker-app (0,274)** - Il punteggio viene quasi tutto dalla CPU (~96%). L'app usa circa 0,53 core in totale, cioè ~0,26 core per pod. - È l'app che compete di più per la CPU sul nodo. - **Consiglio:** verificare che i `requests`/`limits` CPU siano adeguati. Se il carico cresce, conviene un HPA basato sulla CPU (es. target 70% della request). **cache-app (0,124)** - L'interferenza viene dal disco: scrive circa 455 KiB/s, pari al ~44% della soglia di 1 MB/s. CPU e memoria sono trascurabili. - Può rallentare per I/O gli altri carichi che usano lo stesso volume o nodo. - Ha una sola replica, quindi è un single point of failure. - **Consiglio:** controllare se le scritture sono attese (persistenza o log). Valutare un volume dedicato o una classe di storage più veloce. **demo-app (0,029)** - È di fatto inattiva: 0 CPU, nessun I/O, solo circa 15 MB di memoria su 2 repliche. - **Consiglio:** si può passare a 1 replica, oppure valutare lo scale-to-zero se non serve in modo continuo. **Limiti dell'analisi:** il calcolo si basa su un solo campione istantaneo. Per decidere su HPA o sulla riduzione delle repliche conviene confermare il trend con lo storico, ad esempio con `get_historical_metrics` sulle ultime 6–24 ore.
Claude Sonnet 5.5	13.45 s	2823	1576	4399	0.90	# Indice di interferenza normalizzato **Il punteggio più alto è quello di `worker-app`, con 0,274.** ## Assunzioni - La CPU non ha un riferimento di normalizzazione nella richiesta. Ho assunto **1 core = 1,0**. - Per il disco ho usato 1 MB/s = 1.048.576 B/s. - Ogni termine è normalizzato e non è stato limitato a 1, ma nessun valore lo supera. Formula: `indice = 0,5·(CPU/1 core) + 0,25·(Mem/128 MB) + 0,25·(Disk write/1 MB/s)` ## Calcolo | App | CPU (core) | Termine CPU (×0,5) | Mem MB | Termine Mem (×0,25) | Disk write B/s | Termine Disco (×0,25) | **Indice** | |---|---|---|---|---|---|---|---| | **worker-app** | 0,5265 | 0,2633 | 5,33 | 0,0104 | 0 | 0 | **≈ 0,274** | | **cache-app** | 0,0184 | 0,0092 | 2,02 | 0,0039 | 466.431 (0,445 MB/s) | 0,1112 | **≈ 0,124** | | **demo-app** | 0 | 0 | 14,98 | 0,0293 | 0 | 0 | **≈ 0,029** | Se si usa 1 MB = 1.000.000 B, `cache-app` sale a circa 0,130. La classifica non cambia. ## Interpretazione - **`worker-app`** è dominata dalla CPU. Usa circa 0,53 core sommati sui 2 pod, cioè circa 0,26 core per pod. La memoria è trascurabile e il disco è inattivo. È il candidato principale per un HPA basato su CPU. Prima di impostare le soglie conviene verificare le CPU request/limit dei pod, che non sono nello snapshot. - **`cache-app`** è seconda per via della scrittura su disco, circa 0,45 MB/s, che da sola vale 0,11. La CPU è bassa e la memoria è di soli 2 MB. È un'interferenza di tipo I/O, e con una sola replica non ci sono ridondanza né margini di scaling. Una scrittura così alta per una cache con 2 MB di RAM è insolita, quindi vale la pena controllare se sta facendo persistenza o logging eccessivi. - **`demo-app`** è praticamente idle: CPU, disco e rete sono a zero e usa solo 15 MB di memoria. Con 2 repliche, si può valutare di ridurle a 1 se il carico resta nullo. ## Robustezza del risultato `cache-app` supererebbe `worker-app` solo se la CPU fosse normalizzata su una base maggiore di circa 2,3 core. Se la base di CPU che intendi usare è diversa da 1 core, dimmelo e ricalcolo l'indice.

GPT-4o Mini	9.66 s	1355	781	2136	0.76	Per calcolare l'indice di interferenza normalizzato per ciascuna applicazione, utilizziamo la seguente formula: \[\text{Indice} = 0.5 \times \left(\frac{\text{CPU}}{\text{CPU Max}}\right) + 0.25 \times \left(\frac{\text{Memoria}}{\text{Memoria Max}}\right) + 0.25 \times \left(\frac{\text{Disk Write}}{\text{Disk Write Max}}\right) \] Dove: - CPU Max = 1 (in core) - Memoria Max = 128 MB - Disk Write Max = 1 MB/s Ora calcoliamo i valori per ciascuna applicazione: 1. **cache-app** - CPU: 0.0184 - Memoria: 2.0195 MB - Disk Write: 466431.46 B/s (convertito in MB/s: 466431.46 / 1024 / 1024 ≈ 0.444 MB/s) \[\text{Indice}_{cache} = 0.5 \times \left(\frac{0.0184}{1}\right) + 0.25 \times \left(\frac{2.0195}{128}\right) + 0.25 \times \left(\frac{0.444}{1}\right) \] \[\text{Indice}_{cache} \approx 0.5 \times 0.0184 + 0.25 \times 0.0158 + 0.25 \times 0.444 \approx 0.0092 + 0.00395 + 0.111 = 0.12415 \] 2. **demo-app** - CPU: 0.0 - Memoria: 14.9844 MB - Disk Write: 0.0 MB/s \[\text{Indice}_{demo} = 0.5 \times \left(\frac{0.0}{1}\right) + 0.25 \times \left(\frac{14.9844}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[\text{Indice}_{demo} \approx 0 + 0.25 \times 0.117 = 0.02925 \] 3. **worker-app** - CPU: 0.5265 - Memoria: 5.3281 MB - Disk Write: 0.0 MB/s \[\text{Indice}_{worker} = 0.5 \times \left(\frac{0.5265}{1}\right) + 0.25 \times \left(\frac{5.3281}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[\text{Indice}_{worker} \approx 0.26325 + 0.25 \times 0.0416 +	

						3. **worker-app** - CPU: 0.5265 - Memoria: 5.3281 MB - Disk Write: 0.0 MB/s \[\text{Indice}_{worker} = 0.5 \times \left(\frac{0.5265}{1}\right) + 0.25 \times \left(\frac{5.3281}{128}\right) + 0.25 \times \left(\frac{0.0}{1}\right) \] \[\text{Indice}_{worker} \approx 0.26325 + 0.25 \times 0.0416 + 0 \approx 0.26325 + 0.0104 = 0.27365 \] ### Risultati Finali - **cache-app**: 0.12415 - **demo-app**: 0.02925 - **worker-app**: 0.27365 ### Conclusione L'applicazione che ottiene il punteggio più alto è **worker-app** con un indice di interferenza normalizzato di **0.27365**.
GPT-4o	4.82 s	1355	546	1901	0.76	Per calcolare l'indice di interferenza normalizzato per ogni applicazione, utilizzeremo la formula fornita: - 50% del punteggio è basato sull'utilizzo della CPU. - 25% del punteggio è basato sull'utilizzo della memoria, normalizzato su 128 MB. - 25% del punteggio è basato sulla scrittura su disco, normalizzato su 1 MB/s (1000 KB/s). Calcoliamo l'indice per ciascuna applicazione: 1. **Cache-app**: - CPU: \(0.018407413123207 09 \times 0.5 = 0.009203706561603545\) - Memoria: \(\frac{2.01953125}{128} \times 0.25 = 0.003936767578125\) - Disco: \(\frac{466431.46083735797}{1000000} \times 0.25 = 0.11660786520933949\) - Indice totale: \(0.009203706561603545 + 0.003936767578125 + 0.11660786520933949 = 0.12974833934906803\) 2. **Demo-app**: - CPU: \(0.0 \times 0.5 = 0.0\) - Memoria: \(\frac{14.984375}{128} \times 0.25 = 0.029296875\) - Disco: \(0.0 \times 0.25 = 0.0\) - Indice totale: \(0.0 + 0.029296875 + 0.0 = 0.029296875\) 3. **Worker-app**: - CPU: \(0.5265049996160318 \times 0.5 = 0.2632524998080159\) - Memoria: \(\frac{5.328125}{128} \times 0.25 = 0.010400390625\) - Disco: \(0.0 \times 0.25 = 0.0\) - Indice totale: \(0.2632524998080159 + 0.010400390625 + 0.0 = 0.2736528904330159\) **Conclusione**: L'applicazione "worker-app" ottiene il punteggio di interferenza più alto con un indice totale di 0.2736528904330159.

DOMANDA 6 A

```
--------------------------------------------------------------------------------
ESECUZIONE DOMANDA [6A]: Applica la formula di interferenza (peso 0.5 per 1 core CPU, 0.25 per la RAM normalizzata su 128 MB e 0.25 per l'I/O disco parametrato su 1 MB/s): quale servizio evidenzia l'indice complessivo maggiore?
--------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.77 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2737) | Tutti: {'cache-app': 0.1244, 'demo-app': 0.0293, 'worker-app': 0.2737}
Gemini 2.5 Flash	6.61 s	1672	1483	3155	0.74	Calcolando l'indice di interferenza per ciascun servizio: * **cache-app**: * CPU: 0.5 * 0.0184 = 0.0092 * RAM: 0.25 * (2.02 / 128) = 0.0039 * I/O Disco: 0.25 * (466431.46 / 1000000) = 0.1166 * **Indice totale cache-app**: 0.0092 + 0.0039 + 0.1166 = **0.1297** * **demo-app**: * CPU: 0.5 * 0.0 = 0.0 * RAM: 0.25 * (14.98 / 128) = 0.0293 * I/O Disco: 0.25 * (0.0 / 1000000) = 0.0 * **Indice totale demo-app**: 0.0 + 0.0293 + 0.0 = **0.0293** * **worker-app**: * CPU: 0.5 * 0.5265 = 0.2633 * RAM: 0.25 * (5.33 / 128) = 0.0104 * I/O Disco: 0.25 * (0.0 / 1000000) = 0.0 * **Indice totale worker-app**: 0.2633 + 0.0104 + 0.0 = **0.2737** Il servizio che evidenzia l'indice complessivo maggiore è **worker-app** con un valore di 0.2737.

Gemini 3.6 Flash	41.81 s	1686	2812	4498	0.76	Sulla base della formula dell'indice di interferenza fornita: $$\text{Indice} = 0.5 \times \frac{\text{CPU (core)}}{1.0} + 0.25 \times \frac{\text{RAM (MB)}}{128} + 0.25 \times \frac{\text{I/O Disco (B/s)}}{1.000.000}$$ Ecco il calcolo dettagliato per ciascun servizio: --- ### 1. `worker-app` * **CPU (0.5265 core)**: $0.5 \times \frac{0.526505}{1.0} = 0.2632525$ * **RAM (5.33 MB)**: $0.25 \times \frac{5.328125}{128} = 0.0104065$ * **I/O Disco (0 B/s)**: $0.25 \times \frac{0}{1.000.000} = 0$ * **Indice Complessivo**: **`0.27366`** --- ### 2. `cache-app` * **CPU (0.0184 core)**: $0.5 \times \frac{0.018407}{1.0} = 0.0092037$ * **RAM (2.02 MB)**: $0.25 \times \frac{2.019531}{128} = 0.0039444$ * **I/O Disco (466.431,46 B/s)**: $0.25 \times \frac{466.431,46}{1.000.000} = 0.1166079$ * **Indice Complessivo**: **`0.12976`** *(usando 1 MB/s = 1.000.000 B/s; con 1 MiB/s = 1.048.576 B/s l'indice sarebbe `0.12435`)* --- ### 3. `demo-app` * **CPU (0.0 core)**: $0.5 \times \frac{0}{1.0} = 0$ * **RAM (14.98 MB)**: $0.25 \times \frac{14.984375}{128} = 0.0292664$ * **I/O Disco (0 B/s)**: $0.25 \times \frac{0}{1.000.000} = 0$ * **Indice Complessivo**: **`0.02927`** --- ### Risultato Finale Il servizio che evidenzia l'**indice di interferenza complessivo maggiore** è **`worker-app`** (con un punteggio di **~0.2737**), trainato principalmente dall'elevato consumo di CPU (~0.526 core su 2 pod).

Claude Opus 5.5	17.79 s	2829	1658	4487	0.95	# Indice di interferenza: il valore più alto è di **worker-app** Ho usato solo lo snapshot che mi hai fornito. **Formula:** `I = 0,5 · (CPU / 1 core) + 0,25 · (RAM / 128 MB) + 0,25 · (Disk / 1 MB/s)`, con 1 MB/s = 1.048.576 B/s. | Servizio | CPU (0,5) | RAM (0,25) | Disk I/O (0,25) | **Indice totale** | |---|---|---|---|---| | **worker-app** (2 pod) | 0,5 × 0,5265 = 0,2633 | 0,25 × 0,0416 = 0,0104 | 0 | **0,2737** | | cache-app (1 pod) | 0,5 × 0,0184 = 0,0092 | 0,25 × 0,0158 = 0,0039 | 0,25 × 0,4448 = 0,1112 | **0,1244** | | demo-app (2 pod) | 0 | 0,25 × 0,1171 = 0,0293 | 0 | **0,0293** | Se invece 1 MB/s vale 1.000.000 B/s, l'indice di cache-app sale a circa 0,1298. La classifica non cambia. ## Lettura dei risultati 1. **worker-app (0,274)** è il servizio con l'indice più alto. Quasi tutto il valore (circa il 96%) viene dalla CPU: 0,53 core in totale. Una nota importante: il valore è la somma di 2 repliche. **Calcolato per singolo pod** (circa 0,26 core e 2,7 MB a pod) l'indice è circa **0,137**, quindi di poco sopra cache-app. Il primato resta, ma con un margine piccolo. 2. **cache-app (0,124)** ha un profilo diverso. Quasi tutto l'indice (circa l'89%) viene dalla scrittura su disco: circa 455 KiB/s da un solo pod. Questo carico disturba soprattutto lo storage del nodo, non la CPU. 3. **demo-app (0,029)** è di fatto inattivo: CPU, disco e rete sono tutti a 0, e conta solo la memoria (circa 15 MB su 2 pod). ## Raccomandazioni - **worker-app:** se i 2 pod girano sullo stesso nodo, la contesa sulla CPU è il rischio principale. Conviene usare una `podAntiAffinity` o `topologySpreadConstraints` per distribuirli su nodi diversi. Conviene anche impostare `requests` e `limits` di CPU coerenti (almeno 300m per pod) e valutare un HPA basato sulla CPU se il carico cresce. - **cache-app:** conviene tenerlo lontano da carichi che fanno molto I/O. Bisogna anche capire se le scritture continue sono previste (persistenza o snapshot della cache) o sono un'anomalia, perché per una cache in memoria questo volume è insolito. - **demo-app:** è un buon candidato per ridurre le repliche (da 2 a 1) o per lo scale-to-zero, se non deve servire traffico. Questa analisi si basa su un solo campione. Prima di modificare HPA o repliche conviene controllare l'andamento nel tempo con lo storico delle metriche (per esempio le ultime 6–24 ore). Posso farlo io se vuoi.

Claude Sonnet 5.5	11.47 s	2829	1349	4178	0.90	**Il servizio con l'indice di interferenza maggiore è `worker-app`, con circa 0,274.** Ho usato la formula `indice = 0,5·(CPU/1 core) + 0,25·(RAM_MB/128) + 0,25·(disk_write_bps/1 MB/s)`. I valori sono quelli dello snapshot, quindi non ho invocato tool. | Servizio | CPU (0,5 × cores) | RAM (0,25 × MB/128) | Disco (0,25 × bps/1 MB/s) | **Indice** | |---|---|---|---|---| | **worker-app** | 0,5 × 0,5265 = 0,2633 | 0,25 × 5,33/128 = 0,0104 | 0 | **≈ 0,274** | | cache-app | 0,5 × 0,0184 = 0,0092 | 0,25 × 2,02/128 = 0,0039 | 0,25 × 0,466 = ~0,111–0,117* | ≈ 0,124–0,130 | | demo-app | 0 | 0,25 × 14,98/128 = 0,0293 | 0 | ≈ 0,029 | *Il disco dipende dalla definizione di 1 MB/s. Con 1.048.576 B/s il contributo è 0,111. Con 1.000.000 B/s è 0,117. Il ranking non cambia. **Interpretazione e raccomandazioni** - **worker-app**: l'indice è quasi tutto dovuto alla CPU, circa 0,53 core sommati sui 2 pod (~0,26 core a pod). La RAM è trascurabile (5,3 MB) e non c'è I/O disco. Se la CPU cresce, conviene un HPA su CPU. Prima di impostarlo, controlla requests e limits per scegliere una soglia sensata. - **cache-app**: è secondo grazie alle scritture su disco (~466 KB/s), mentre CPU e RAM sono quasi nulle. Il carico è tutto I/O. Verifica se la persistenza, per esempio snapshot o AOF, è necessaria con questa frequenza. Con una sola replica è anche un single point of failure. - **demo-app**: è praticamente idle, con CPU 0, nessun I/O e nessun traffico di rete. L'unico contributo è la RAM (~15 MB). Puoi ridurla a 1 replica se non serve ridondanza. Questo è un confronto istantaneo. Non dice nulla su picchi o trend. Se vuoi, posso recuperare lo storico con `get_historical_metrics` per verificare se il ranking è stabile.
GPT-4o Mini	8.50 s	1357	704	2061	0.72	Per calcolare l'indice complessivo di interferenza per ciascuna applicazione, utilizziamo la formula fornita: \[\text{Indice} = 0.5 \times \text{CPU (core)} + 0.25 \times \left(\frac{\text{Memory (MB)}}{128}\right) + 0.25 \times \left(\frac{\text{Disk Write (bps)}}{1 \text{ MB/s}}\right) \] Convertiamo i valori di I/O disco da bps a MB/s (1 MB = 1,000,000 bytes): 1. **cache-app**: - CPU: 0.018407413123207 09 core - Memory: 2.01953125 MB - Disk Write: 466431.46083735797 bps = 0.46643146083735797 MB/s \[\text{Indice}_{cache} = 0.5 \times 0.018407413123207 09 + 0.25 \times \left(\frac{2.01953125}{128}\right) + 0.25 \times 0.46643146083735797 \] \[\text{Indice}_{cache} \approx 0.009203706561603545 + 0.00394921875 + 0.11660786520933949 \approx 0.129760790520943 \] 2. **demo-app**: - CPU: 0.0 core - Memory: 14.984375 MB - Disk Write: 0.0 bps = 0.0 MB/s \[\text{Indice}_{demo} = 0.5 \times 0.0 + 0.25 \times \left(\frac{14.984375}{128}\right) + 0.25 \times 0.0 \] \[\text{Indice}_{demo} \approx 0 + 0.029296875 + 0 \approx 0.029296875 \] 3. **worker-app**: - CPU: 0.5265049996160318 core - Memory: 5.328125 MB - Disk Write: 0.0 bps = 0.0 MB/s \[\text{Indice}_{worker} = 0.5 \times 0.5265049996160318 + 0.25 \times \left(\frac{5.328125}{128}\right) + 0.25 \times 0.0 \] \[\text{Indice}_{worker} \approx 0.2632524998080159 + 0.0104140625 + 0 \approx 0.2736665623080159 \] ### Risultati Finali: - **cache-app**: 0.129760790520943 - **demo-app**: 0.029296875 - **worker-app**: 0.2736665623080159 ### Conclusione: Il servizio che evidenzia l'indice complessivo maggiore è **worker-app** con un indice di **0.2737**.

GPT-4o	5.64 s	1357	595	1952	0.76	Per calcolare l'indice di interferenza complessivo per ciascun servizio, utilizzeremo la formula fornita: \[\text{Indice di Interferenza} = 0.5 \times \text{CPU Cores} + 0.25 \times \left(\frac{\text{Memory MB}}{128}\right) + 0.25 \times \left(\frac{\text{Disk Write Bps}}{1,048,576}\right) \] Calcoliamo l'indice per ciascuna applicazione: 1. **cache-app**: - CPU: \(0.5 \times 0.018407413123207 09 = 0.009203706561603545\) - Memory: \(0.25 \times \left(\frac{2.01953125}{128}\right) = 0.003945703125\) - Disk I/O: \(0.25 \times \left(\frac{466431.46083735797}{1,048,576}\right) = 0.111086865209\) - Indice complessivo: \(0.009203706561603545 + 0.003945703125 + 0.111086865209 = 0.12423627489560354\) 2. **demo-app**: - CPU: \(0.5 \times 0.0 = 0.0\) - Memory: \(0.25 \times \left(\frac{14.984375}{128}\right) = 0.029296875\) - Disk I/O: \(0.25 \times \left(\frac{0.0}{1,048,576}\right) = 0.0\) - Indice complessivo: \(0.0 + 0.029296875 + 0.0 = 0.029296875\) 3. **worker-app**: - CPU: \(0.5 \times 0.5265049996160318 = 0.2632524998080159\) - Memory: \(0.25 \times \left(\frac{5.328125}{128}\right) = 0.010417480468 75\) - Disk I/O: \(0.25 \times \left(\frac{0.0}{1,048,576}\right) = 0.0\) - Indice complessivo: \(0.2632524998080159 + 0.010417480468 75 + 0.0 = 0.2736699802767659\) **Conclusione**: L'applicazione "worker-app" evidenzia l'indice complessivo maggiore con un valore di 0.2736699802767659.

DOMANDA 6 B

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [6B]: Ponderando le risorse con 50% CPU, 25% memoria (benchmark 128MB) e 25% scrittura disco (benchmark 1MB/s), determina quale applicazione risulta la più interferente in termini di score normalizzato.
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature will be ignored.
  request = self._build_request_config(
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	2.78 ms	0	0	0	N/A	Punteggio più alto: worker-app (Score: 0.2737) | Tutti: {'cache-app': 0.1244, 'demo-app': 0.0293, 'worker-app': 0.2737}
Gemini 2.5 Flash	8.96 s	1660	2028	3688	0.76	Calcoliamo lo score normalizzato per ciascuna applicazione: **'cache-app':** * CPU Score: 0.018407413123207 09 / 1 = 0.0184 * Memory Score: 2.01953125 MB / 128 MB = 0.0158 * Disk Write Score: 466431.46083735797 B/s / 1,000,000 B/s = 0.4664 * Score Ponderato: (0.0184 * 0.50) + (0.0158 * 0.25) + (0.4664 * 0.25) = 0.0092 + 0.00395 + 0.1166 = **0.12975** **'demo-app':** * CPU Score: 0.0 / 1 = 0.0 * Memory Score: 14.984375 MB / 128 MB = 0.1171 * Disk Write Score: 0.0 B/s / 1,000,000 B/s = 0.0 * Score Ponderato: (0.0 * 0.50) + (0.1171 * 0.25) + (0.0 * 0.25) = 0.0 + 0.029275 + 0.0 = **0.029275** **'worker-app':** * CPU Score: 0.5265049996160318 / 1 = 0.5265 * Memory Score: 5.328125 MB / 128 MB = 0.0416 * Disk Write Score: 0.0 B/s / 1,000,000 B/s = 0.0 * Score Ponderato: (0.5265 * 0.50) + (0.0416 * 0.25) + (0.0 * 0.25) = 0.26325 + 0.0104 + 0.0 = **0.27365** Confrontando gli score ponderati: * 'cache-app': 0.12975 * 'demo-app': 0.029275 * 'worker-app': 0.27365 L'applicazione che risulta la più interferente in termini di score normalizzato è **'worker-app'** con uno score di 0.27365.

Claude Opus 5.5	17.63 s	2812	1763	4575	0.90	# Score di interferenza normalizzato **L'applicazione più interferente è worker-app, con uno score di circa 0,274.** È circa il doppio di cache-app (0,124) e circa 9 volte demo-app (0,029). Quasi tutto lo score viene dalla CPU. ## Come ho calcolato lo score Per ogni risorsa divido il valore misurato per il suo riferimento, poi sommo i risultati pesati: **Score = 0,50 × (CPU ÷ 1 core) + 0,25 × (memoria ÷ 128 MB) + 0,25 × (scrittura disco ÷ 1 MB/s)** - **Riferimento CPU:** nella richiesta non c'è, quindi ho usato **1 core**. - **1 MB/s** = 1.048.576 B/s. - Ho usato i valori totali di ogni deployment. ## Risultati | App | CPU normalizzata (×0,50) | Memoria normalizzata (×0,25) | Disco normalizzato (×0,25) | **Score** | |---|---|---|---|---| | **worker-app** | 0,5265 → 0,2633 | 5,33/128 = 0,0416 → 0,0104 | 0 → 0 | **0,2737** 🥇 | | cache-app | 0,0184 → 0,0092 | 2,02/128 = 0,0158 → 0,0039 | 466.431/1.048.576 = 0,4448 → 0,1112 | **0,1243** | | demo-app | 0 → 0 | 14,98/128 = 0,1171 → 0,0293 | 0 → 0 | **0,0293** | ## Quanto è affidabile la classifica - **Score per pod:** worker-app resta prima, con circa 0,137 per pod contro 0,124 di cache-app, che ha un solo pod. Lo scarto però si riduce molto. - **Se 1 MB/s vale 1.000.000 B/s:** cache-app sale a circa 0,130 e la classifica non cambia. - **Riferimento CPU:** worker-app resta prima finché il riferimento è sotto circa 2,3 core. Con un riferimento più alto (per esempio 4 core) cache-app diventerebbe la più interferente. ## Cosa significa per ogni applicazione 1. **worker-app:** il carico è dovuto alla CPU, circa 0,26 core per pod. Rete e disco sono a zero, quindi sembra un lavoro di solo calcolo. - Verifica i `requests/limits` di CPU e se ci sono rallentamenti imposti dal limite CPU (throttling), con `container_cpu_cfs_throttled_periods_total`. - Se il carico continua, valuta un autoscaling automatico (HPA) sulla CPU con obiettivo al 70% del request. 2. **cache-app:** è la seconda fonte di interferenza, per la scrittura su disco (circa 455 KiB/s con un solo pod). Può rallentare il disco per gli altri pod sullo stesso nodo. - Verifica se queste scritture sono previste (salvataggi o log di una cache non dovrebbero essere continui). - Valuta un volume dedicato o tenerla su un nodo diverso dagli altri carichi che scrivono molto su disco. 3. **demo-app:** è praticamente inattiva, con CPU, rete e disco a zero. Puoi valutare di ridurla a 1 replica per liberare risorse. Questi risultati si basano su una sola istantanea. Prima di decidere modifiche alle repliche o all'autoscaling, conviene controllare l'andamento nelle ultime ore (con `get_historical_metrics`), così da escludere un picco momentaneo.

Claude Sonnet 5.5	12.39 s	2812	1534	4346	0.76	**Risultato: `worker-app` è l'applicazione più interferente, con score ≈ 0,27.** **Assunzione:** per la CPU non hai indicato un benchmark, quindi ho usato 1 core = 1,0. Per memoria e disco ho usato i benchmark che hai dato (128 MB e 1 MB/s). Ho preso le metriche così come sono nello snapshot, cioè aggregate per applicazione. **Formula:** score = 0,5·(CPU/1 core) + 0,25·(MEM/128 MB) + 0,25·(DiskWrite/1 MB/s) | App | CPU norm. | MEM norm. | Disk norm. | Score | |---|---|---|---|---| | **worker-app** | 0,5265 → 0,2633 | 5,33/128 = 0,0416 → 0,0104 | 0 → 0 | **≈ 0,274** | | cache-app | 0,0184 → 0,0092 | 2,02/128 = 0,0158 → 0,0039 | 0,466 → 0,1166 | ≈ 0,130 | | demo-app | 0 → 0 | 14,98/128 = 0,117 → 0,0293 | 0 → 0 | ≈ 0,029 | **Interpretazione** - **worker-app** pesa soprattutto per la CPU: usa circa 0,53 core in totale, cioè circa 0,26 core per pod su 2 repliche. La memoria è trascurabile e non scrive su disco. Lo score è guidato quasi solo dalla CPU, quindi è il candidato per un HPA basato su CPU. Prima di scalare, verifica requests e limits CPU. - **cache-app** è al secondo posto per le scritture su disco (~0,47 MB/s, circa il 47% del benchmark). La CPU e la memoria sono minime. Il suo impatto è di I/O, non di calcolo. Controlla se la persistenza o lo snapshotting della cache è troppo aggressivo. - **demo-app** è praticamente idle: CPU, disco e rete sono a 0, e solo la memoria (~15 MB) pesa un po'. Con 2 repliche si può valutare di scendere a 1. **Robustezza:** il ranking non cambia con un benchmark CPU diverso, purché sia inferiore a circa 2,3 core. Con un benchmark più alto, il peso del disco di `cache-app` supererebbe quello della CPU di `worker-app`. Se normalizzi la CPU rispetto al valore massimo osservato, `worker-app` resta al primo posto con ancora più margine.

GPT-4o Mini	9.69 s	1347	807	2154	0.00	Per calcolare lo score normalizzato delle applicazioni in base alle risorse utilizzate (CPU, memoria e scrittura su disco), utilizzeremo i benchmark forniti: - **CPU**: 50% del punteggio - **Memoria**: 25% del punteggio - **Scrittura Disco**: 25% del punteggio I benchmark sono: - **CPU**: 50% di 1 core = 0.5 core - **Memoria**: 25% di 128 MB = 32 MB - **Scrittura Disco**: 25% di 1 MB/s = 0.25 MB/s Ora calcoliamo lo score normalizzato per ciascuna applicazione: 1. **cache-app**: - CPU: 0.0184 / 0.5 = 0.0368 - Memoria: 2.0195 / 32 = 0.0637 - Scrittura Disco: 466431.46 / 1 = 466431.46 (questo valore è molto alto e influenzerà notevolmente il punteggio) Score totale = \(0.0368 \times 0.5 + 0.0637 \times 0.25 + 466431.46 \times 0.25\) 2. **demo-app**: - CPU: 0.0 / 0.5 = 0.0 - Memoria: 14.9844 / 32 = 0.468 - Scrittura Disco: 0.0 / 1 = 0.0 Score totale = \(0.0 \times 0.5 + 0.468 \times 0.25 + 0.0 \times 0.25\) 3. **worker-app**: - CPU: 0.5265 / 0.5 = 1.053 - Memoria: 5.3281 / 32 = 0.166 - Scrittura Disco: 0.0 / 1 = 0.0 Score totale = \(1.053 \times 0.5 + 0.166 \times 0.25 + 0.0 \times 0.25\) Ora calcoliamo i punteggi: ### Calcolo dei punteggi 1. **cache-app**: \[Score_{cache} = 0.0368 \times 0.5 + 0.0637 \times 0.25 + 466431.46 \times 0.25 = 0.0184 + 0.015925 + 116607.865 = 116607.8993 \] 2. **demo-app**: \[Score_{demo} = 0.0 \times 0.5 + 0.468 \times 0.25 + 0.0 \times 0.25 = 0.0 + 0.117 = 0.117 \] 3. **worker-app**: \[Score_{worker} = 1.053 \times 0.5 + 0.166 \times 0.25 + 0.0 \times 0.25 = 0.5265 + 0.0415 = 0.568 \]	

						1. **cache-app**: \[Score_{cache} = 0.0368 \times 0.5 + 0.0637 \times 0.25 + 466431.46 \times 0.25 = 0.0184 + 0.015925 + 116607.865 = 116607.8993 \] 2. **demo-app**: \[Score_{demo} = 0.0 \times 0.5 + 0.468 \times 0.25 + 0.0 \times 0.25 = 0.0 + 0.117 = 0.117 \] 3. **worker-app**: \[Score_{worker} = 1.053 \times 0.5 + 0.166 \times 0.25 + 0.0 \times 0.25 = 0.5265 + 0.0415 = 0.568 \] ### Risultati Finali - **cache-app**: 116607.8993 - **demo-app**: 0.117 - **worker-app**: 0.568 ### Conclusione L'applicazione **cache-app** risulta la più interferente in termini di score normalizzato, principalmente a causa dell'elevato utilizzo di scrittura su disco.

GPT-4o	4.60 s	1347	572	1919	0.72	Per determinare quale applicazione risulta la più interferente in termini di score normalizzato, utilizzeremo i pesi forniti: 50% per la CPU, 25% per la memoria e 25% per la scrittura su disco. I benchmark di riferimento sono 1 core CPU, 128 MB di memoria e 1 MB/s di scrittura su disco. Calcoliamo lo score normalizzato per ciascuna applicazione: 1. **Cache-app:** - CPU: \(\frac{0.018407413123207 09}{1} \times 0.5 = 0.0092037065616035 45 \) - Memoria: \(\frac{2.01953125}{128} \times 0.25 = 0.0039367675781 25 \) - Scrittura Disco: \(\frac{466431.46083735797}{1000000} \times 0.25 = 0.116607865209339 49 \) - Score totale: \(0.009203706561603545 + 0.003936767578125 + 0.11660786520933949 = 0.12974833934906803 \) 2. **Demo-app:** - CPU: \(\frac{0.0}{1} \times 0.5 = 0.0 \) - Memoria: \(\frac{14.984375}{128} \times 0.25 = 0.029296875 \) - Scrittura Disco: \(\frac{0.0}{1000000} \times 0.25 = 0.0 \) - Score totale: \(0.0 + 0.029296875 + 0.0 = 0.029296875 \) 3. **Worker-app:** - CPU: \(\frac{0.5265049996160318}{1} \times 0.5 = 0.2632524998080159 \) - Memoria: \(\frac{5.328125}{128} \times 0.25 = 0.010400390625 \) - Scrittura Disco: \(\frac{0.0}{1000000} \times 0.25 = 0.0 \) - Score totale: \(0.2632524998080159 + 0.010400390625 + 0.0 = 0.2736528904330159 \) **Conclusione:** La "worker-app" risulta essere l'applicazione più interferente in termini di score normalizzato, con un valore di 0.2736528904330159.

DOMANDA 7

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	117.71 ms	0	0	0	N/A	App: demo-app | Delta: -14.00% (da 17.36 a 14.93 MB)
Gemini 2.5 Flash	30.11 s	5921	1027	6948	0.70	L'applicazione `demo-app` consuma attualmente 14.93 MB di memoria. Analizzando lo storico delle metriche delle ultime 2 ore, la memoria di `demo-app` era di 17.36 MB. La variazione percentuale della memoria di `demo-app` nelle ultime 2 ore è stata del **-14.00%**.
Gemini 3.6 Flash	30.72 s	5544	1678	7222	0.80	### Analisi della Memoria e Variazione Storica #### 1. Selezione dell'Applicazione con Maggior Consumo di Memoria Dall'analisi delle metriche istantanee attuali: - **`cache-app`**: 1,04 MB (`memory_mb`) - **`worker-app`**: 5,25 MB (`memory_mb`) - **`demo-app`**: **14,93 MB** (`memory_mb`) L'applicazione che consuma più memoria è **`demo-app`**. --- #### 2. Calcolo della Variazione Percentuale nelle Ultime 2 Ore Interrogando lo storico delle metriche persostite su database per `demo-app`: - **Valore iniziale (circa 1 ora e 00 minuti fa - 16:03:39)**: - Memoria in MB: **17,36 MB** - Memoria in Bytes: **18.206.720 bytes** - **Valore finale/attuale (17:03:45)**: - Memoria in MB: **14,93 MB** (14,9296875 MB) - Memoria in Bytes: **15.654.912 bytes** ##### Calcolo della Variazione: $$\text{Variazione \%} = \frac{\text{Valore Finale} - \text{Valore Iniziale}}{\text{Valore Iniziale}} \times 100$$ - **In Megabyte (MB):** $$\frac{14,93 - 17,36}{17,36} \times 100 \approx -14,00\%$$ - **In Bytes:** $$\frac{15.654.912 - 18.206.720}{18.206.720} \times 100 \approx -14,02\%$$ --- ### Esito e Considerazioni SRE - **Risultato**: La memoria dell'applicazione **`demo-app`** ha subito una **riduzione di circa il 14,00%** nelle ultime due ore. - **Stato Carico**: Il carico di memoria per `demo-app` è attualmente stabile e ridotto rispetto al picco registrato ad inizio intervallo (~17,36 MB).

Claude Opus 5.5	36.64 s	8320	1447	9767	0.95	# Variazione della memoria di demo-app nelle ultime 2 ore **La memoria di demo-app è scesa del 14,02%, da 17,36 MB a 14,93 MB.** Non è un segnale di saturazione. ## Applicazione scelta Secondo lo snapshot attuale, **demo-app** è quella che usa più memoria: | App | Memoria | Pod | |---|---|---| | **demo-app** | **14,93 MB** | 2 | | worker-app | 5,25 MB | 2 | | cache-app | 1,04 MB | 1 | ## Calcolo | | Ora | Memoria | |---|---|---| | Primo campione | 16:03:39 | 18.206.720 B (**17,36 MB**) | | Ultimo campione | 17:03:45 | 15.654.912 B (**14,93 MB**) | **Variazione = (15.654.912 − 18.206.720) / 18.206.720 × 100 = −14,02%** L'ultimo campione salvato corrisponde esattamente allo snapshot attuale (14,93 MB), quindi i dati sono coerenti. ## Come è scesa la memoria - **Picco:** 17,36 MB (primo campione). **Minimo:** 14,93 MB (ultimo). - **Primi due minuti circa (16:03 → 16:05):** la memoria passa da 17,36 a 15,62 MB (circa −10%). Nello stesso momento la CPU va a 0 e le scritture su disco si fermano. Probabilmente l'app ha finito un'attività e ha liberato memoria. - **16:05 → 17:03:** la memoria resta quasi ferma (da 15,62 a 14,93 MB) e la CPU rimane a 0. **Attenzione:** anche se ho chiesto le ultime 2 ore, lo storico copre solo **circa 60 minuti** (dalle 16:03 alle 17:03). Non ci sono dati prima delle 16:03. Inoltre i campioni sono molto fitti all'inizio e poi molto radi (16:31, 17:01, 17:03). Il −14% vale quindi per l'ora coperta dai dati, non per 2 ore intere. ## Raccomandazioni 1. **Valutare di passare da 2 a 1 replica.** Usa meno di 15 MB su 2 pod, la CPU è a 0 da circa un'ora e non riceve traffico di rete. È di fatto inattiva. 2. **Ridimensionare le richieste di risorse** (`requests`/`limits`), che con circa 7,5 MB per pod sono probabilmente sovradimensionate rispetto all'uso reale. 3. **Raccogliere più dati con regolarità:** campionare ogni 5–15 minuti, per avere davvero 2 ore di storico prima di decisioni definitive sullo scaling.
Claude Sonnet 5.5	32.79 s	8309	1023	9332	0.90	**demo-app** ha la memoria più alta nello snapshot (14,93 MB totali su 2 pod), contro 5,25 MB di worker-app e 1,04 MB di cache-app. Nelle ultime 2 ore la sua memoria è scesa di circa **14,0%**. | | Timestamp | Memoria | |---|---|---| | Primo campione | 16:03:39 | 18.206.720 B (17,36 MB) | | Ultimo campione | 17:03:45 | 15.654.912 B (14,93 MB) | Variazione % = (15.654.912 − 18.206.720) / 18.206.720 × 100 = **−14,02%**, cioè circa −2,55 MB. - **Finestra dati:** lo storico parte alle 16:03, quindi copre circa 1 ora e non le 2 richieste. Il valore si riferisce a quell'intervallo. - **Andamento:** il calo è avvenuto quasi tutto nei primi minuti, da 17,36 a 15,62 MB entro le 16:05. Poi la memoria si è stabilizzata, con un lieve calo fino a 14,93 MB. Sembra un assestamento dopo l'avvio e non una crescita o una perdita di memoria. - **Carico:** la CPU è a 0 e non c'è traffico di rete. I 2 pod usano circa 7,5 MB ciascuno, quindi demo-app è sostanzialmente idle. - **Raccomandazione:** valuta di ridurre le repliche da 2 a 1, se non ti serve la ridondanza. Il consumo è molto basso e stabile. Verifica però prima i requisiti di disponibilità.
GPT-4o Mini	26.39 s	4661	250	4911	0.67	L'applicazione che consuma più memoria attualmente è **demo-app**, con un consumo di **14.93 MB**. Analizzando le metriche storiche delle ultime 2 ore, ecco i valori di memoria: - **Inizio (17:03:45)**: 14.93 MB - **Fine (16:03:39)**: 17.36 MB Ora calcoliamo la variazione percentuale della memoria: \[\text{Variazione Percentuale} = \left(\frac{\text{Memoria Finale} - \text{Memoria Iniziale}}{\text{Memoria Iniziale}} \right) \times 100 \] \[\text{Variazione Percentuale} = \left(\frac{14.93 - 17.36}{17.36} \right) \times 100 \approx -13.98\% \] Quindi, la variazione percentuale della memoria della **demo-app** nelle ultime 2 ore è di circa **-13.98%**.

GPT-4o	25.01 s	4661	246	4907	0.70	L'applicazione che consuma più memoria attualmente è la "demo-app" con un utilizzo di 14.93 MB. Per calcolare la variazione percentuale della memoria nelle ultime 2 ore, confrontiamo il valore attuale con il valore iniziale di 2 ore fa: - Memoria iniziale (2 ore fa): 17.36 MB - Memoria attuale: 14.93 MB La variazione percentuale è calcolata come: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria iniziale}}{\text{Memoria iniziale}}\right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{14.93 - 17.36}{17.36}\right) \times 100 \approx -13.99\% \] Quindi, la memoria della "demo-app" è diminuita di circa il 13.99% nelle ultime 2 ore.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.5731 (mini: $0.0743 | 4o: $1.4988)

DOMANDA 7 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	222.00 ms	0	0	0	N/A	App: demo-app | Delta: -14.00% (da 17.36 a 14.93 MB)
Gemini 2.5 Flash	25.20 s	5929	1190	7119	0.76	L'applicazione con il maggior footprint di RAM è **demo-app** con 14.93 MB. Analizzando la serie storica della memoria di `demo-app` nelle ultime 2 ore: * La memoria registrata all'inizio del periodo (circa 2 ore fa) era di 17.36 MB. * La memoria attuale è di 14.93 MB. Il delta percentuale di memoria registrato nelle ultime 2 ore per `demo-app` è: ((14.93 MB - 17.36 MB) / 17.36 MB) * 100 = **-14.01%** Questo indica una riduzione del consumo di memoria di circa il 14.01% nelle ultime due ore per l'applicazione `demo-app`.
Gemini 3.6 Flash	36.74 s	5552	1378	6930	0.80	### 1. Identificazione dell'Applicazione con maggior Footprint di RAM Dall'analisi delle metriche istantanee correnti: * **`cache-app`**: 1.04 MB * **`worker-app`**: 5.25 MB * **`demo-app`**: **14.93 MB** (14.9296875 MB) L'applicazione con il maggior consumo di memoria RAM è **`demo-app`** (distribuita su 2 pod). --- ### 2. Analisi della Serie Storica (Ultime 2 Ore su Database) Consultando lo storico su MongoDB per `demo-app` nell'intervallo delle ultime 2 ore: * **Primo rilevamento registrato** (`16:03:39`): **17.36 MB** (18.206.720 bytes) * **Ultimo rilevamento registrato** (`17:03:45`): **14.93 MB** (15.654.912 bytes) #### Calcolo del Delta Percentuale: $$\text{Delta} = \frac{14.93 \text{ MB} - 17.36 \text{ MB}}{17.36 \text{ MB}} \times 100 \approx -14.00\%$$ *(Su base byte: $\frac{15.654.912 - 18.206.720}{18.206.720} \times 100 \approx -14.02\%$)* --- ### Sintesi e Considerazioni SRE * **Esito**: L'applicazione `demo-app` presenta un footprint RAM di circa **14.93 MB**, con una **riduzione del ~14.00%** della memoria occupata nelle ultime 2 ore. * **Stato del carico**: L'applicazione risulta attualmente idle o a bassissimo utilizzo sia di CPU (0.0 cores) che di rete/disco, mantenendo un profilo di consumo stabile e contenuto.

Claude Opus 5.5	35.49 s	8316	1780	10096	0.90	# Analisi footprint RAM: demo-app ## 1. App con più RAM (snapshot attuale) | App | Pod | Memoria totale | Memoria per pod | |---|---|---|---| | **demo-app** | 2 | **14,93 MB** | ~7,46 MB | | worker-app | 2 | 5,25 MB | ~2,63 MB | | cache-app | 1 | 1,04 MB | 1,04 MB | **demo-app** è l'app che usa più RAM. Da sola consuma circa il 70% della memoria usata dalle tre app (14,93 MB su 21,22 MB). ## 2. Storico su MongoDB (ultime 2 ore) Ho trovato 23 campioni, tra le 16:03:39 e le 17:03:45 del 30/10/2026. Il numero di pod è rimasto a 2 per tutto il periodo. | Punto | Timestamp | memory_bytes | memory_mb | |---|---|---|---| | Primo campione (anche il valore più alto) | 16:03:39 | 18.206.720 | 17,36 | | Ultimo campione (anche il valore più basso) | 17:03:45 | 15.654.912 | 14,93 | L'ultimo campione coincide esattamente con lo snapshot attuale (14,9296875 MB = 15.654.912 byte), quindi i dati sono coerenti. ### Calcolo del delta - **Differenza assoluta:** 15.654.912 − 18.206.720 = **−2.551.808 byte (circa −2,43 MB)** - **Differenza percentuale:** −2.551.808 / 18.206.720 × 100 = **−14,02%** ## 3. Come è cambiata la memoria - **La memoria è sempre scesa o rimasta ferma:** in nessun campione risale. Il calo più forte avviene nei primi 2 minuti (da 17,36 a 15,62 MB tra 16:03 e 16:05). - **Dalle 16:31 in poi è quasi ferma:** passa da 14,98 a 14,93 MB, quindi praticamente stabile. - **Nello stesso periodo l'app smette di lavorare:** dalle 16:04:34 CPU e scritture su disco vanno a 0, e il traffico di rete in ingresso è sempre stato 0. Probabilmente il calo di memoria è un rilascio di cache o buffer dopo un breve periodo di attività, non una perdita di memoria. - **Manca un tratto di dati:** non ci sono campioni tra le 16:05:21 e le 16:31:29, e dopo i campioni sono pochi. Il valore iniziale e quello finale sono affidabili, ma non sappiamo cosa sia successo in quel buco di circa 26 minuti. ## 4. Raccomandazioni

						| Punto | Timestamp | memory_bytes | memory_mb | |---|---|---|---| | Primo campione (anche il valore più alto) | 16:03:39 | 18.206.720 | 17,36 | | Ultimo campione (anche il valore più basso) | 17:03:45 | 15.654.912 | 14,93 | L'ultimo campione coincide esattamente con lo snapshot attuale (14,9296875 MB = 15.654.912 byte), quindi i dati sono coerenti. ### Calcolo del delta - **Differenza assoluta:** 15.654.912 − 18.206.720 = **−2.551.808 byte (circa −2,43 MB)** - **Differenza percentuale:** −2.551.808 / 18.206.720 × 100 = **−14,02%** ## 3. Come è cambiata la memoria - **La memoria è sempre scesa o rimasta ferma:** in nessun campione risale. Il calo più forte avviene nei primi 2 minuti (da 17,36 a 15,62 MB tra 16:03 e 16:05). - **Dalle 16:31 in poi è quasi ferma:** passa da 14,98 a 14,93 MB, quindi praticamente stabile. - **Nello stesso periodo l'app smette di lavorare:** dalle 16:04:34 CPU e scritture su disco vanno a 0, e il traffico di rete in ingresso è sempre stato 0. Probabilmente il calo di memoria è un rilascio di cache o buffer dopo un breve periodo di attività, non una perdita di memoria. - **Manca un tratto di dati:** non ci sono campioni tra le 16:05:21 e le 16:31:29, e dopo i campioni sono pochi. Il valore iniziale e quello finale sono affidabili, ma non sappiamo cosa sia successo in quel buco di circa 26 minuti. ## 4. Raccomandazioni

1. **Nessun problema di memoria:** l'uso è basso (~7,5 MB per pod) e in calo. Non ci sono segni di una perdita di memoria.
2. **demo-app risulta inattiva:** con CPU, scritture su disco e traffico di rete in ingresso a zero, 2 repliche sembrano superflue.
   - Potete ridurla a **1 replica**.
   - Se l'app serve solo in certi momenti, potete configurare un **HPA con `minReplicas: 1`**, oppure lo scale-to-zero (es. con KEDA).
3. **Ridimensionare le risorse richieste:** se i `requests` e `limits` di memoria del pod sono molto più alti di circa 8–10 MB per pod, abbassateli per liberare spazio sui nodi. Lasciate un po' di margine, considerando il picco di 17,36 MB osservato per l'intera app.
4. **Migliorare la raccolta dati:** conviene campionare a intervalli regolari (es. ogni 5 minuti) per evitare buchi come quello delle 16:05–16:31 e rendere più affidabili le analisi future.

Claude Sonnet 5.5	28.77 s	8351	1303	9654	0.95	**Applicazione con più RAM: `demo-app`**, con 14,93 MB totali su 2 pod (circa 7,5 MB per pod). `worker-app` è a 5,25 MB e `cache-app` a 1,04 MB. **Delta di memoria nelle ultime 2 ore** | Punto | Timestamp | Memoria | |---|---|---| | Primo campione | 16:03:39 | 18.206.720 B (17,36 MB) | | Ultimo campione | 17:03:45 | 15.654.912 B (14,93 MB) | - **Delta assoluto:** −2.551.808 B, cioè circa −2,43 MB. - **Delta percentuale:** **−14,0%**. - Se si esclude il primo campione, che sembra un picco isolato (17,36 MB, poi subito 16,17 MB), il delta è da 16.957.440 B a 15.654.912 B, cioè **−7,7%**. **Limiti dei dati** - Lo storico copre circa 1 ora (16:03–17:03), non 2. Prima delle 16:03 non ci sono campioni, quindi il delta si riferisce a questa finestra. - I campioni sono densi nei primi 90 secondi e poi rari (16:05, 16:31, 17:01, 17:03), quindi la curva intermedia è poco risolta. **Interpretazione e raccomandazioni** - La memoria scende in modo graduale e senza picchi, e non indica leak. Può essere un assestamento dopo l'avvio, con rilascio di cache o buffer iniziali. Il valore si sta stabilizzando: tra 16:31 e 17:03 la variazione è di soli 0,05 MB. - Il carico è praticamente idle. La CPU è a 0 core, e il disco e la rete sono a 0, mentre a inizio finestra c'era un'attività minima (~0,0016 core). Con 2 repliche e 15 MB totali, **`demo-app` si può ridurre a 1 replica** se non servono ridondanza o alta disponibilità. Questo libera circa 7,5 MB senza impatto sul carico attuale. - Non c'è saturazione di RAM su nessuna app. `worker-app` è invece quella con CPU più alta, 0,57 core su 2 pod, e andrebbe valutata per un HPA basato su CPU.
GPT-4o Mini	32.13 s	8844	307	9151	0.80	L'applicazione con il maggior footprint di RAM attualmente è **demo-app**, con un utilizzo di **14.93 MB** di memoria. ### Analisi della serie storica della memoria per demo-app nelle ultime 2 ore: 1. **Memoria all'inizio del periodo (16:03:39)**: 17.36 MB 2. **Memoria alla fine del periodo (17:03:45)**: 14.93 MB ### Calcolo del delta percentuale di memoria: - **Delta** = Memoria finale − Memoria iniziale = 14.93 MB − 17.36 MB = −2.43 MB - **Delta percentuale** = (Delta / Memoria iniziale) * 100 = (−2.43 / 17.36) * 100 ≈ −14.0% ### Risultato: Il delta percentuale di memoria registrato per **demo-app** nelle ultime 2 ore è di circa **−14.0%**. Questo indica una riduzione dell'utilizzo della memoria nel tempo.

GPT-4o Mini	32.13 s	8844	307	9151	0.80	L'applicazione con il maggior footprint di RAM attualmente è **demo-app**, con un utilizzo di **14.93 MB** di memoria. ### Analisi della serie storica della memoria per demo-app nelle ultime 2 ore: 1. **Memoria all'inizio del periodo (16:03:39)**: 17.36 MB 2. **Memoria alla fine del periodo (17:03:45)**: 14.93 MB ### Calcolo del delta percentuale di memoria: - **Delta** = Memoria finale − Memoria iniziale = 14.93 MB − 17.36 MB = −2.43 MB - **Delta percentuale** = (Delta / Memoria iniziale) * 100 = (−2.43 / 17.36) * 100 ≈ −14.0% ### Risultato: Il delta percentuale di memoria registrato per **demo-app** nelle ultime 2 ore è di circa **−14.0%**. Questo indica una riduzione dell'utilizzo della memoria nel tempo.

| GPT-4o | 22.88 s | 4671 | 230 | 4901 | 0.67 | L'applicazione con il maggior footprint di RAM attualmente è la "demo-app", che utilizza 14.93 MB di memoria.

Analizzando la serie storica delle metriche di memoria per la "demo-app" nelle ultime 2 ore, possiamo osservare il seguente delta percentuale:

- Memoria iniziale (2 ore fa): 17.36 MB
- Memoria attuale: 14.93 MB

Calcolo del delta percentuale:
\[
\text{Delta percentuale} = \left(\frac{\text{Memoria attuale} − \text{Memoria iniziale}}{\text{Memoria iniziale}}\right) \times 100 = \left(\frac{14.93 − 17.36}{17.36}\right) \times 100 \approx −13.99\%
\]

Quindi, la memoria utilizzata dalla "demo-app" è diminuita di circa il 13.99% nelle ultime 2 ore. |

[BILLING MONITOR] Spesa cumulata OpenAI: $1.5886 (mini: $0.0758 | 4o: $1.5128)

DOMANDA 7 B

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	142.71 ms	0	0	0	N/A	App: demo-app | Delta: -14.00% (da 17.36 a 14.93 MB)
Gemini 2.5 Flash	19.08 s	5938	808	6746	0.70	L'applicazione "demo-app" è il top consumer di memoria. * **Utilizzo attuale di RAM per "demo-app"**: 14.93 MB * **Utilizzo di RAM per "demo-app" due ore fa**: 17.36 MB (basato sulla prima metrica disponibile nello storico) La variazione percentuale dell'utilizzo di RAM per "demo-app" è stata di circa **-14.01%**, indicando una diminuzione rispetto a due ore fa.
Gemini 3.6 Flash	25.79 s	5558	1300	6858	0.76	### Analisi del Top Consumer di Memoria 1. **Identificazione del Top Consumer:** Dalle metriche istantanee correnti, l'applicazione con il maggior consumo di memoria RAM è **`demo-app`**: - **`demo-app`**: ~14.93 MB (15.654.912 byte) - `worker-app`: 5.25 MB - `cache-app`: ~1.04 MB --- 2. **Confronto Storico (2 ore fa vs Attuale):** Interrogando lo storico delle metriche salvato su database per **`demo-app`**: - **Utilizzo di RAM circa 2 ore fa** (~16:03:39): **17.36 MB** (18.206.720 byte) - **Utilizzo di RAM attuale** (~17:03:45 / Corrente): **14.93 MB** (15.654.912 byte) --- 3. **Variazione Percentuale:** $$\text{Variazione \%} = \frac{14.93 \text{ MB} - 17.36 \text{ MB}}{17.36 \text{ MB}} \times 100 \approx -13.99\% \quad (\approx -14.02\% \text{ calcolato sui byte})$$ **Esito:** L'utilizzo di RAM per **`demo-app`** è **diminuito di circa il 14.0%** nell'arco delle ultime due ore.
Claude Opus 5.5	32.84 s	8342	1574	9916	0.95	Ho verificato lo storico di **demo-app**: tra il dato più vecchio disponibile e quello attuale la RAM è **scesa di circa il 14%**. I dati però coprono solo circa 1 ora, non 2. **App analizzata:** nello snapshot demo-app è quella che usa più RAM (14,93 MB su 2 pod), davanti a worker-app (5,25 MB) e cache-app (1,04 MB). **Calcolo** | Momento | Ora | Memoria | |---|---|---| | Inizio (dato più vecchio disponibile) | 16:03:39 | 17,36 MB (18.206.720 byte) | | Adesso | 17:03:45 | 14,93 MB (15.654.912 byte) | **Variazione = (14,93 − 17,36) / 17,36 × 100 ≈ −14,0%** (−2,43 MB) ⚠️ **Dati incompleti:** ho chiesto le ultime 2 ore, ma il primo dato salvato è delle 16:03:39. Non c'è nulla prima, quindi non ho un valore vero di "due ore fa". Il −14% riguarda circa 1 ora. Per il confronto su 2 ore bisognerà ripetere la verifica quando lo storico coprirà tutto il periodo. **Andamento** - **16:03–16:05:** la memoria scende in fretta, da 17,36 a 15,62 MB. - **16:05–17:03:** cala ancora pochissimo e poi resta ferma intorno a 14,93–14,98 MB. Non c'è nessuna crescita, quindi niente fa pensare a una perdita di memoria. - **CPU:** è a 0 dalle 16:04:34 in poi. Anche scritture su disco e traffico di rete in entrata sono a zero. **Raccomandazioni** 1. **L'app sembra inattiva (idle):** CPU, disco e rete sono a zero da circa un'ora e ogni pod usa in media solo ~7,5 MB. 2. **Ridurre le repliche:** se demo-app non deve garantire alta disponibilità, si può scendere da 2 a 1 replica (`kubectl scale deploy demo-app --replicas=1`). In alternativa si può configurare un HPA con `minReplicas: 1`. 3. **Ridurre requests/limits:** se le richieste di memoria sono molto sopra i ~8 MB per pod usati davvero, conviene abbassarle per liberare risorse sul nodo. 4. **Attenzione al vero carico CPU:** la pressione sulla CPU viene da **worker-app** (~0,57 core su 2 pod), non dall'app che consuma più memoria. Ha senso valutare un HPA basato sulla CPU per worker-app.

Claude Sonnet 5.5	26.18 s	8276	1306	9582	0.95	**Il top consumer di memoria è demo-app, e la sua RAM è scesa di circa il 14,0% rispetto a due ore fa.** Nello snapshot corrente demo-app usa 14,93 MB. cache-app ne usa 1,04 e worker-app 5,25. | Momento | Memoria | |---|---| | Due ore fa (campione più vecchio della finestra, 16:03:39) | 18.206.720 byte (17,36 MB) | | Attuale (14,93 MB, uguale all'ultimo campione delle 17:03:45) | 15.654.912 byte (14,93 MB) | Variazione = (15.654.912 − 18.206.720) / 18.206.720 ≈ **−14,0%**, cioè circa −2,4 MB. **Andamento:** - La RAM è calata in modo quasi monotono, da 17,36 MB a 16,17 MB nei primi secondi. - Alle 16:05 era a 15,62 MB e alle 17:03 a 14,93 MB. - Non ci sono picchi né segni di memory leak. - Il calo più forte è all'inizio, poi la curva si stabilizza. **Limite del dato:** l'ultimo campione è delle 17:03:45, mentre il più vecchio nella finestra di 2 ore è delle 16:03:39. Quindi il confronto copre circa 1 ora di storico effettivo. Se l'ora attuale è più avanti dell'ultimo campione, il valore "due ore fa" è solo il più vecchio disponibile nella finestra. **Raccomandazioni:** - demo-app ha CPU a 0 e nessun traffico di rete o disco. Il carico è idle e la RAM è stabile. - Con 2 repliche a ~7,5 MB l'una, puoi valutare di scendere a 1 replica, se non servono ridondanza o alta disponibilità. - I valori di memoria sono molto bassi, quindi controlla che requests e limits non siano sovradimensionati.
GPT-4o Mini	19.92 s	4677	246	4923	0.70	Per calcolare la variazione percentuale dell'utilizzo della RAM della `demo-app` confrontando il dato attuale con quello di due ore fa, utilizziamo i seguenti valori: - **Utilizzo di RAM attuale**: 14.93 MB - **Utilizzo di RAM due ore fa**: 17.36 MB La formula per calcolare la variazione percentuale è: \[\text{Variazione Percentuale} = \left(\frac{\text{Valore Attuale} - \text{Valore Precedente}}{\text{Valore Precedente}} \right) \times 100 \] Sostituendo i valori: \[\text{Variazione Percentuale} = \left(\frac{14.93 - 17.36}{17.36} \right) \times 100 \approx -13.98\% \] Quindi, l'utilizzo della RAM della `demo-app` è diminuito di circa **13.98%** rispetto a due ore fa.

GPT-4o	17.80 s	4677	201	4878	0.20	Due ore fa, l'applicazione "demo-app" utilizzava 15.78 MB di memoria. Attualmente, l'utilizzo di memoria è di 14.93 MB. Per calcolare la variazione percentuale nell'utilizzo della memoria: \[\text{Variazione percentuale} = \left(\frac{\text{Memoria attuale} - \text{Memoria passata}}{\text{Memoria passata}}\right) \times 100 \] \[\text{Variazione percentuale} = \left(\frac{14.93 - 15.78}{15.78}\right) \times 100 \approx -5.39\% \] Quindi, l'utilizzo di memoria della "demo-app" è diminuito di circa il 5.39% rispetto a due ore fa.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6031 (mini: $0.0766 | 4o: $1.5265)

DOMANDA 8

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [8]: Assumendo che l'applicazione a maggior carico di CPU abbia un limit di 300m per pod (600m totali su 2 repliche), calcola la percentuale di saturazione e valuta il r
ischio di throttling.
--------------------------------------------------------------------------------------------------------------

Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio
Prometheus MCP server running on stdio
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling paramet
er(s) temperature will be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\
033[00m\\]\\$ '
[10/30/26 21:17:00] INFO     Processing request of type CallToolRequest                                                                                    server.py:733
[10/30/26 21:17:01] INFO     Processing request of type ListToolsRequest                                                                                   server.py:733
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	4.72 ms	0	0	0	N/A	App: worker-app | Uso: 0.2749 cores | Saturazione: 45.82% | Rischio: BASSO
Gemini 2.5 Flash	4.82 s	1701	570	2271	0.70	L'applicazione con il maggior carico di CPU è `worker-app`, con un consumo di 0.275 core. Assumendo un limite di 300m (0.3 core) per pod e 2 repliche, il limite totale di CPU per `worker-app` è di 600m (0.6 core). Calcolo della saturazione CPU: Saturazione = (CPU utilizzata / Limite CPU totale) * 100 Saturazione = (0.275 / 0.6) * 100 = 45.83% La `worker-app` sta utilizzando circa il 45.83% del suo limite CPU totale. A questo livello, il rischio di throttling è basso.
Gemini 3.6 Flash	30.58 s	24725	2550	27275	0.90	Ecco l'analisi dettagliata relativa all'applicazione a maggior carico di CPU: --- ### 1. Identificazione dell'Applicazione Dall'analisi delle metriche istantanee correnti: * **`worker-app`**: **0.27495 CPU cores** (~274.95 mCPU totali su 2 repliche) $\rightarrow$ **Maggior carico di CPU** * `cache-app`: 0.00883 CPU cores (~8.83 mCPU) * `demo-app`: 0.00234 CPU cores (~2.34 mCPU) --- ### 2. Calcolo della Percentuale di Saturazione * **Consumo attuale complessivo:** ~274.95 mCPU (0.27495 cores) * **Limit totale disponibile:** 600 mCPU (0.600 cores su 2 repliche da 300m ciascuna) $$\text{Saturazione Totale} = \left(\frac{274.95 \text{ mCPU}}{600 \text{ mCPU}} \right) \times 100 \approx \mathbf{45.83\%}$$ * **Consumo medio per Pod:** ~137.47 mCPU su un limit individuale di 300m ($\mathbf{45.83\%}$ di saturazione per singolo pod, assumendo un carico equamente bilanciato). --- ### 3. Valutazione del Rischio di CPU Throttling * **Rischio Globale: BASSO** * Con una saturazione complessiva del **45.83%**, l'applicazione sta utilizzando meno della metà delle risorse limite assegnate. * **Considerazioni e Raccomandazioni SRE:** * **Carichi costanti:** Se l'applicazione elabora un carico costante/fluido, non sussiste alcun rischio imminente di CPU throttling o degrado delle prestazioni. * **Carichi a spunti (Bursty):** Se `worker-app` esegue elaborazioni con spike improvvisi di durata ridotta, i singoli pod potrebbero momentaneamente saturare la quota del limit individuale (300m) all'interno del periodo di controllo del CFS kernel (di default 100ms). * **Azione consigliata:** Verificare nei grafici di monitoraggio la metrica `container_cpu_cfs_throttled_periods_total` per confermare l'assenza di micro-throttling durante i picchi di elaborazione.

Claude Opus 5.5	18.85 s	2819	1655	4474	0.86	# Saturazione CPU di worker-app rispetto al limit (300m/pod) worker-app è l'applicazione che usa più CPU: **0,2749 core (≈275m)** su 2 repliche. È circa 30 volte demo-app e cache-app messe insieme. ## Calcolo della saturazione | Livello | Consumo | Limit | Saturazione | |---|---|---|---| | **Aggregato (2 pod)** | 274,9m | 600m | **≈ 45,8%** | | **Per pod (media)** | ≈ 137,5m | 300m | **≈ 45,8%** | | Margine residuo | ≈ 325m totali (≈ 162m/pod) | | | Formula: 0,27495 / 0,600 × 100 = **45,82%** ## Valutazione del rischio di throttling **Rischio attuale: da basso a moderato.** 1. **Nella media c'è margine.** Al 46% del limit il throttling dovrebbe essere raro. 2. **La media può nascondere picchi.** Kubernetes misura il limit su finestre di 100ms. Un carico a raffiche, tipico di un worker, può superare i 300m in alcune finestre e subire throttling anche se la media resta sotto il 50%. Lo snapshot non mostra questi picchi. 3. **Il carico tra i due pod potrebbe non essere bilanciato.** Il dato è aggregato. Se un pod facesse, per esempio, 200m e l'altro 75m, il primo sarebbe al 67% del limit, con un rischio molto più alto. 4. **Se si perde una replica (scenario N-1):** il pod rimasto dovrebbe gestire circa 275m su 300m, cioè **circa il 92%**. A quel punto il throttling sarebbe quasi certo. È il rischio principale. 5. **Possibile ambiguità del dato:** se il valore `cpu_cores` fosse già una media per pod e non una somma, ogni pod sarebbe a 275m su 300m (**≈ 91,6%**), con un rischio alto. Il campo `pods: 2` fa pensare a una somma, ma conviene verificarlo. ## Raccomandazioni - **Verificare il throttling reale con Prometheus:** - quota di periodi in throttling: `rate(container_cpu_cfs_throttled_periods_total{pod=~"worker-app.*"}[5m]) / rate(container_cpu_cfs_periods_total{pod=~"worker-app.*"}[5m])`. Sopra il 5–10% c'è un problema. - consumo per singolo pod, per controllare lo squilibrio: `sum by (pod) (rate(container_cpu_usage_seconds_total{pod=~"worker-app.*"}[5m]))` - **Configurare un HPA** su worker-app, con minimo 2 e massimo 4 repliche. L'HPA calcola l'utilizzo sulle **requests**, non sui limits: va impostata una request coerente (per esempio 150–200m) e un target intorno al 70%, così scala prima di arrivare al limit. - **Mantenere almeno 2 repliche** e valutare un PodDisruptionBudget (`minAvailable: 1`). Anche così, nello scenario N-1 il pod rimasto sarebbe al 92%: conviene alzare il limit a 400–500m oppure tenere 3 repliche minime. - **Controllare lo storico** con `get_historical_metrics` sulle ultime 24h/7gg per capire se 275m è un valore normale o un picco. Se vuoi, eseguo subito le query di throttling e l'analisi dello storico per confermare la valutazione.

Claude Sonnet 5.5	29.66 s	9242	2170	11412	0.68	## Risultato **Il worker-app è il più carico in CPU. Con il limit ipotizzato di 300m per pod è al ~46% di saturazione, ma le metriche reali indicano un rischio di throttling alto.** ### Saturazione con il limit ipotizzato (300m per pod, 600m totali) | Misura | Uso CPU | Saturazione | |---|---|---| | Aggregato (snapshot) | 0,275 core | 0,275 / 0,6 = **45,8%** | | Pod `…ftlln` (Prometheus, 5m) | 0,150 core | 0,150 / 0,3 = **50,0%** | | Pod `…9kj75` (Prometheus, 5m) | 0,129 core | 0,129 / 0,3 = **43,2%** | | Picco storico (16:04–17:03) | ~0,59 core | ~0,59 / 0,6 = **~98%** | ### Rischio di throttling - **Sulla base dell'uso attuale** il rischio sarebbe basso, perché c'è circa il 54% di margine. - **Il throttling misurato contraddice questa lettura.** `container_cpu_cfs_throttled_periods_total` / `container_cpu_cfs_periods_total` dà **~94% di periodi throttled** su entrambi i pod (0,941 e 0,937). Con un uso di 130–150m, questo non è compatibile con un limit di 300m. - **Il limit reale è probabilmente più basso di 300m.** Non l'ho verificato, ad esempio con `kube_pod_container_resource_limits`. Una stima, solo indicativa, è che sia nell'ordine di 100–150m, e l'uso attuale sarebbe quindi un valore compresso dal limite. - **Nello storico il carico era più alto.** La CPU aggregata stava a ~0,59 core, cioè ~0,295 per pod, quasi al tetto dei 300m. Il valore resta piatto e quasi identico per molti campioni consecutivi, un pattern tipico di un carico tagliato da un limite. - **Il calo recente non è un recupero.** Alle 20:15 la CPU è scesa a 0,275 e il traffico di rete in ingresso è basso (~4,6 B/s). Il basso utilizzo potrebbe dipendere da un calo del carico, ma anche da throttling o rallentamento del lavoro.

						### Raccomandazioni 1. **Verifica il limit effettivo:** `kubectl get deploy worker-app -o jsonpath='{..resources}'`. Se è inferiore a 300m, il throttling è spiegato. 2. **Alza il limit CPU** a 500–600m per pod, oppure rimuovilo e tieni solo la request. Il picco osservato è ~300m per pod, quindi 300m non lascia margine. 3. **Imposta una request realistica**, intorno a 250–300m, in base al carico storico. 4. **Valuta un HPA** sul worker-app con target CPU ~60–70%, min 2 repliche. Il carico ha oscillato tra 0,27 e 0,59 core, quindi lo scaling orizzontale ha senso. 5. **Monitora il rapporto di throttling** con un alert sopra il 25%. ### Altre applicazioni - **cache-app:** 8,8m di CPU e 0,85 MB di memoria, con 131 KB/s di scritture su disco. È quasi idle in CPU, ma ha un'attività di I/O costante. - **demo-app:** 2,3m di CPU su 2 repliche e 17 MB di memoria. È idle e si può ridurre a 1 replica se non serve alta disponibilità.
GPT-4o Mini	5.77 s	1359	391	1750	0.95	Per calcolare la percentuale di saturazione della CPU per l'applicazione "worker-app", utilizziamo i dati forniti: - **Limite di CPU per pod**: 300m (0.3 CPU) - **Numero di pod**: 2 - **Carico attuale di CPU**: 0.27494928595663526 CPU ### Calcolo della saturazione della CPU 1. **Limite totale di CPU**: \[\text{Limite totale} = \text{Limite per pod} \times \text{Numero di pod} = 0.3 \, \text{CPU} \times 2 = 0.6 \, \text{CPU} \] 2. **Percentuale di saturazione**: \[\text{Percentuale di saturazione} = \left(\frac{\text{Carico attuale}}{\text{Limite totale}} \right) \times 100 = \left(\frac{0.27494928595663526}{0.6} \right) \times 100 \approx 45.82\% \] ### Valutazione del rischio di throttling Con una percentuale di saturazione della CPU di circa 45.82%, l'applicazione "worker-app" non è attualmente a rischio di throttling, poiché il carico è ben al di sotto del limite massimo di 600m. ### Raccomandazione Non ci sono azioni immediate necessarie per l'applicazione "worker-app" in termini di scaling o ottimizzazione delle risorse, poiché il carico di CPU è gestibile e non si avvicina al limite. Tuttavia, è sempre consigliabile monitorare le metriche nel tempo per eventuali cambiamenti nel carico di lavoro.

						nel tempo per eventuali cambiamenti nel carico di lavoro.
GPT-4o	4.54 s	1359	392	1751	0.80	Per valutare la percentuale di saturazione della CPU e il rischio di throttling per l'applicazione con il maggior carico di CPU, seguiamo questi passaggi: 1. **Identificazione dell'applicazione con il maggior carico di CPU:** - Dalle metriche fornite, l'applicazione "worker-app" ha il maggior carico di CPU con 0.27494928595663526 core utilizzati. 2. **Calcolo della percentuale di saturazione:** - Il limite di CPU per pod è 300m (0.3 core), quindi per 2 repliche è 600m (0.6 core). - La percentuale di saturazione è calcolata come: \((\text{CPU utilizzata} / \text{CPU limite}) \times 100\). 3. **Calcolo:** - CPU utilizzata totale: 0.27494928595663526 core. - CPU limite totale: 0.6 core. - Percentuale di saturazione: \((0.27494928595663526 / 0.6) \times 100 \approx 45.82\%\). 4. **Valutazione del rischio di throttling:** - La percentuale di saturazione è circa il 45.82%, il che indica che l'applicazione sta utilizzando meno della metà del limite disponibile. - Non c'è un rischio immediato di throttling, poiché l'utilizzo è ben al di sotto del limite. **Conclusione:** L'applicazione "worker-app" utilizza circa il 45.82% della CPU disponibile, quindi attualmente non c'è rischio di throttling. Tuttavia, è sempre consigliabile monitorare regolarmente le metriche per rilevare eventuali aumenti improvvisi nel consumo di risorse.

DOMANDA 8 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.78 ms	0	0	0	N/A	App: worker-app | Uso: 0.2749 cores | Saturazione: 45.82% | Rischio: BASSO
Gemini 2.5 Flash	6.28 s	1700	1072	2772	0.28	L'applicazione più intensiva su CPU è **worker-app**, con un consumo di 0.2749 core. Ipotizzando una quota limite di 300 millicore (0.3 core) per pod, l'utilizzo attuale di `worker-app` è del **91.65%** del budget CPU per pod (0.2749 / 0.3 * 100). Considerando questa percentuale di utilizzo molto elevata, il pericolo di CPU throttling è **concreto e imminente**. Anche piccoli picchi di carico potrebbero causare throttling, impattando negativamente le performance dell'applicazione.
Gemini 3.6 Flash	10.58 s	1714	1845	3559	0.95	### Analisi delle Metriche e Valutazione del Budget CPU #### 1. Identificazione dell'applicazione più intensiva Dall'analisi delle metriche istantanee correnti, l'applicazione a maggior consumo di CPU è **`worker-app`**: * **Pod attivi:** 2 (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`) * **Utilizzo CPU totale corrente:** `0.27495 core` (**~274.95 millicore**) * *(Per confronto: `cache-app` utilizza ~8.83 mCPU, `demo-app` utilizza ~2.34 mCPU)*. --- #### 2. Calcolo della Percentuale del Budget Utilizzata Con le ipotesi concordate: * **Limite per pod:** `300 millicore` (0.3 core) * **Limite totale deployment (2 pod):** `600 millicore` (0.6 core) **Calcolo:** $$ \text{Percentuale del budget} = \left(\frac{274.95 \text{ mCPU}}{600 \text{ mCPU}} \right) \times 100 \approx 45.82\% $$ Assumendo un carico ben distribuito sui 2 pod, ciascun pod sta consumando mediamente **~137.47 mCPU**, pari a circa il **45.8% del limite individuale di 300 mCPU**. --- #### 3. Valutazione del Rischio di CPU Throttling * **Pericolo Immediato:** **BASSO**. * **Motivazione:** Il deployment sta attualmente consumando meno della metà del budget complessivo assegnato (ha ancora un margine libero di oltre il 54%, ovvero ~325 mCPU disponibili prima di raggiungere la quota di throttling). #### Raccomandazioni SRE: 1. **Margine di Sicurezza:** L'attuale utilizzo lascia ampio margine per sostenere incrementi moderati di carico senza incorrere in saturazione. 2. **Bilanciamento del Carico:** Assicurarsi che il traffico/lavoro sia distribuito equamente tra i due pod. Se uno dei due pod stesse assorbendo l'intero carico (es. 275 mCPU su 300 mCPU limite), quel singolo pod sarebbe vicino alla soglia di throttling (~91.6%). 3. **Micro-burst e Throttling:** I limiti di CPU in Kubernetes utilizzano il meccanismo Completely Fair Scheduler (CFS). Se l'applicazione genera picchi rapidi di CPU (burst millisecondari), potrebbe verificarsi throttling anche con un consumo medio basso. In tal caso, si consiglia di monitorare tramite Prometheus la metrica `container_cpu_cfs_throttled_periods_total`.

Claude Opus 5.5	48.70 s	11316	2506	13822	0.63	## worker-app: il throttling c'è già, non è solo un rischio L'app che usa più CPU è **worker-app** (2 pod). Ho verificato su Prometheus che il limite di **300m per pod non è un'ipotesi**: è già configurato (`kube_pod_container_resource_limits` = 0.3 core sul container `cpu-worker` di entrambi i pod). ### 1. Percentuale del budget usata | Fonte | CPU misurata | Budget (600m) | |---|---|---| | Snapshot fornito | 0,275 core (275m) | **≈ 45,8%** | | Prometheus, media 5 minuti per pod | ftlln 295,1m + 9kj75 294,5m = **≈ 590m** | **≈ 98,3%** | | Prometheus, picco dell'ultima ora (finestre da 1 minuto) | ≈ 295,6m + 295,4m ≈ 591m | **≈ 98,5%** | **Attenzione:** lo snapshot e Prometheus non coincidono. Il valore di 275m quasi certamente è la **media per pod**, non la somma del deployment, oppure viene da un campionamento diverso. Letto come media per pod sarebbe circa il 91,6% del limite, coerente con Prometheus. Il dato reale è che **ogni pod usa circa il 98% del suo limite di 300m**. ### 2. Quanto è concreto il pericolo di throttling Non è un pericolo, sta già succedendo. Ho misurato la quota di periodi CFS (le finestre in cui il kernel controlla il limite di CPU) in cui il pod è stato bloccato, negli ultimi 5 minuti: - worker-app-84cf4dd446-ftlln: **95,7%** - worker-app-84cf4dd446-9kj75: **95,1%** Quindi in circa 95 finestre su 100 il pod esaurisce la quota e il kernel lo ferma fino alla finestra successiva. L'uso fermo a ~295m per tutta l'ora vuol dire che **il limite fa da tetto rigido**: l'app vorrebbe più CPU di quella che le viene data. Il vero fabbisogno quindi **non si vede dalle metriche, perché viene tagliato dal limite**. La metrica sui secondi di throttling (`container_cpu_cfs_throttled_seconds_total`) non è disponibile, quindi non posso dire di quanto. Effetti probabili: latenza più alta, job più lenti e code che si allungano. La memoria è trascurabile (~5,6 MB) e non c'entra. ### 3. Cosa consiglio 1. **Subito:** alzare il limite CPU, ad esempio a 500m–1000m per pod, oppure toglierlo e tenere solo la `request`. Poi guardare dove si stabilizza l'uso reale senza il tetto. 2. **Scaling orizzontale (HPA):** se il carico si può dividere tra più pod, configurare un HPA sulla CPU con target 70% della request e min 2 / max 4–6 repliche. Con 2 repliche sature non c'è margine per i picchi. 3. **Dopo la modifica**, tenere d'occhio questi due valori: - `rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m])`: obiettivo sotto il 5–10% - l'uso di CPU per pod, per dimensionare request e limit sul fabbisogno reale. 4. **Altri candidati a ridimensionamento:** cache-app (~9m) e demo-app (~2m su 2 pod) sono quasi a riposo. demo-app potrebbe scendere a 1 replica, se non serve per l'alta disponibilità.

Claude Sonnet 5.5	35.49 s	9214	2105	11319	0.29	## Risposta **L'app più intensiva su CPU è `worker-app`.** Con un limite ipotetico di 300m per pod (600m sul deployment), il budget è consumato così: | Fonte | CPU per pod | % del limite di 300m | |---|---|---| | Snapshot istantaneo (0,275 core totali su 2 pod) | ~0,137 core | **~46%** | | Prometheus, `rate` su 5 minuti | 0,295 / 0,294 core | **~98%** | | Storico (16:04–17:03) | ~0,26–0,30 core a pod (0,52–0,59 totali) | ~88–99% | Lo snapshot dà una fotografia fuorviante. Il valore di 0,275 core è un minimo isolato: l'ultimo punto storico (20:15) è circa la metà dei precedenti, quindi probabilmente cattura un calo momentaneo o una finestra di campionamento diversa. Il dato più affidabile è quello a 5 minuti. Secondo quel dato **i pod sono praticamente al limite, intorno al 98% del budget**. ## Pericolo di CPU throttling: alto - `container_cpu_cfs_throttled_periods_total` mostra circa **19 periodi/s throttlati per pod** (19,1 e 18,2). Il throttling è già in corso, non solo possibile. - Con un uso medio a ~98% del limite non c'è margine per i picchi. Il CFS applica il limite a finestre brevi (di solito 100 ms), quindi anche picchi istantanei vengono tagliati. - Lo storico mostra che l'uso è rimasto vicino alla saturazione per ore. L'uso totale si è stabilizzato a ~0,59 core, vicino al tetto di 0,6 del deployment, cioè un comportamento da CPU-bound saturo. ## Cautele sui dati - Il valore di ~19 periodi/s è superiore a quanto ci si aspetta con un periodo CFS da 100 ms (massimo 10/s). Può dipendere da un periodo diverso, da più container nel pod o dalla finestra di calcolo. Non ho interrogato `container_cpu_cfs_periods_total`, quindi **non posso darti la percentuale esatta di periodi throttlati**. - Se nel cluster esiste già un limite reale, potrebbe non essere 300m. Verifica il valore effettivo nel manifest. - Se il limite di 300m è solo ipotetico, il throttling osservato indica comunque che un limite è già applicato. ## Raccomandazioni 1. **Alza il limite CPU** a 500–600m per pod, oppure rimuovilo e tieni solo la request. Con ~300m di uso medio, 300m di limite è troppo stretto. 2. **Configura un HPA** su `worker-app` con target CPU intorno al 60–70% della request. Oggi le 2 repliche sono sature, quindi servirebbe scalare a 3–4 repliche. 3. **Memoria non critica:** `worker-app` usa ~5,6 MB totali, quindi puoi dimensionare la memoria in modo molto contenuto. 4. **Altre app non a rischio:** `demo-app` usa 0,0023 core e `cache-app` 0,009 core. Sono quasi idle e si potrebbe valutare di ridurre le repliche di `demo-app`.

GPT-4o Mini	5.55 s	1362	321	1683	0.24	Analizzando le metriche fornite, l'applicazione più intensiva su CPU è la **worker-app**, che attualmente utilizza circa **0.2749 core** di CPU per i suoi 2 pod. Calcoliamo la percentuale di utilizzo della CPU rispetto al budget di 600 millicore (0.6 core) per il deployment: 1. **Utilizzo totale della CPU per la worker-app**: - Utilizzo per pod: 0.2749 core - Totale per 2 pod: 0.2749 * 2 = 0.5498 core 2. **Percentuale di utilizzo rispetto al budget**: \[\text{Percentuale} = \left(\frac{0.5498}{0.6} \right) \times 100 \approx 91.63\% \] Quindi, l'applicazione **worker-app** sta utilizzando circa **91.63%** del budget di CPU disponibile. ### Rischio di CPU Throttling Con un utilizzo della CPU del 91.63%, c'è un rischio concreto di CPU throttling, specialmente se ci sono picchi di carico o se l'applicazione richiede più risorse in determinati momenti. È consigliabile monitorare attentamente l'utilizzo della CPU e considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per gestire meglio i picchi di carico e prevenire il throttling.
GPT-4o	4.16 s	1362	389	1751	0.90	Per determinare quale applicazione è la più intensiva su CPU, possiamo confrontare l'utilizzo attuale della CPU per ciascuna applicazione. Dalle metriche istantanee fornite: - **cache-app**: 0.0088272375 core - **demo-app**: 0.0023352025628008916 core - **worker-app**: 0.27494928595663526 core L'applicazione **worker-app** è quella che utilizza più CPU, con un totale di 0.27494928595663526 core. Ora, considerando che il limite di CPU è di 300 millicore per pod (0.3 core), e che il deployment ha due pod, il limite totale per il deployment è di 600 millicore (0.6 core). Calcoliamo la percentuale di utilizzo rispetto al limite totale: \[\text{Percentuale di utilizzo CPU} = \left(\frac{\text{Utilizzo CPU attuale}}{\text{Limite CPU totale}}\right) \times 100 = \left(\frac{0.27494928595663526}{0.6}\right) \times 100 \approx 45.82\% \] L'utilizzo attuale della CPU per il deployment "worker-app" è circa il 45.82% del limite totale disponibile. ### Conclusione Attualmente, l'utilizzo della CPU è ben al di sotto del limite totale di 600 millicore, quindi il pericolo di CPU throttling è basso. Tuttavia, è importante monitorare l'andamento nel tempo per assicurarsi che l'utilizzo non aumenti significativamente, avvicinandosi al limite. Se l'utilizzo della CPU dovesse aumentare, potrebbe essere necessario considerare lo scaling orizzontale per evitare il throttling.

DOMANDA 8 B

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.94 ms	0	0	0	N/A	App: worker-app | Uso: 0.2749 cores | Saturazione: 45.82% | Rischio: BASSO
Gemini 2.5 Flash	4.68 s	1704	813	2517	0.76	Il servizio più caricato in termini di CPU è `worker-app`, con un consumo attuale di 0.275 core. Considerando un limite teorico di 0.3 core per pod, e dato che `worker-app` ha 2 pod, il limite totale per questo servizio sarebbe di 0.6 core. La percentuale di utilizzo della CPU per `worker-app` rispetto a questo limite è: (0.275 core / 0.6 core) * 100 = **45.83%** Dato che l'utilizzo attuale (0.275 core) è ben al di sotto del limite teorico (0.6 core), **non sussistono i presupposti per tagli di cicli da parte dello scheduler CFS**. L'applicazione sta operando entro i limiti definiti.

Gemini 3.6 Flash	21.09 s	4714	2191	6905	0.63	### Analisi del Servizio Più Caricato Il servizio con il consumo di CPU più elevato è **`worker-app`**, distribuito su 2 pod (`worker-app-84cf4dd446-9kj75` e `worker-app-84cf4dd446-ftlln`). --- ### 1. Valutazione della Percentuale di Utilizzo della CPU * **Consumo istantaneo totale:** **~0.275 core** (`0.27495` core totali su 2 pod, ossia circa `0.1375` core per pod). * **Limite teorico complessivo:** **0.6 core** (0.3 core/pod × 2 pod). #### Calcolo Percentuale: * **Rispetto al totale del servizio:** $$\frac{0.27495 \text{ core}}{0.60000 \text{ core}} \times 100 \approx \mathbf{45.83\%}$$ * **Rispetto al limite per singolo pod:** $$\frac{0.13747 \text{ core}}{0.30000 \text{ core}} \times 100 \approx \mathbf{45.83\%}$$ --- ### 2. Presupposti ed Evidenze di Tagli di Cicli (CFS CPU Throttling) **Sì, sussistono ampiamente i presupposti per il taglio dei cicli CPU da parte dello scheduler Linux CFS.** Dall'ispezione diretta delle metriche dello scheduler cgroup (`container_cpu_cfs_throttled_periods_total` / `container_cpu_cfs_periods_total`): * **Tasso di Throttling Rilevato:** Circa il **95% dei periodi CFS** (`~95.0%`) relativi ai container `cpu-worker` subiscono il taglio dei cicli di clock. #### Motivazione Tecnica: Anche se l'utilizzo medio cumulativo appare ben al di sotto del limite teorico (~45.8%), lo scheduler CFS (Completely Fair Scheduler) valuta le quote di CPU su finestre temporali estremamente brevi (tipicamente 100ms). Il processo dell'applicazione genera brevi e intensi picchi di calcolo (*bursts*) che saturano la quota consentita all'interno della singola finestra di 100ms, causando il blocco dei thread fino alla finestra successiva (*CFS CPU throttling*). --- ### Raccomandazioni SRE: 1. **Aumento del CPU Limit o rimozione dei Limiti CPU:** Valutare la rimozione o l'incremento del limite di CPU per `worker-app` per prevenire la latenza indotta dal throttling CFS nei periodi di burst. 2. **Scaling Orizzontale (HPA):** Se il carico non può essere spalmato con rimozione dei limiti, considerare l'aumento delle repliche per distribuire meglio i burst di lavoro.
Claude Opus 5.5	34.85 s	7028	2288	9316	0.27	# Il CFS sta già tagliando i cicli a worker-app **Risposta breve:** sì. Il servizio più caricato è **worker-app**, e non c'è solo il rischio di tagli: il CFS li sta già facendo. Lo snapshot fa sembrare la situazione tranquilla, ma le metriche dirette di Prometheus mostrano il contrario. ## 1. Cosa dice lo snapshot | Parametro | Valore | |---|---| | CPU totale dichiarata | 0,2749 core | | Limite totale teorico | 0,6 core (2 pod × 0,3) | | **Utilizzo calcolato** | **circa 45,8%** | | Media per pod | circa 0,137 core, cioè il 45,8% di 0,3 | Con questi numeri sembrerebbe esserci molto margine e nessun taglio. ## 2. Cosa dice Prometheus (finestra di 5 minuti, controllo diretto) | Pod | CPU usata | Limite configurato | % del limite | Periodi CFS con taglio | |---|---|---|---|---| | worker-app-…-ftlln | 0,2973 core | 0,3 core | **99,1%** | **95,1%** | | worker-app-…-9kj75 | 0,2971 core | 0,3 core | **99,0%** | **95,0%** | | **Totale** | **0,594 core** | **0,6 core** | **99,1%** | – | - Ho verificato tramite `kube_pod_container_resource_limits` che il limite di 0,3 core per container (`cpu-worker`) è davvero configurato, quindi non è solo un'ipotesi. - I due pod si comportano in modo identico. Il carico è distribuito in modo uniforme e consuma tutta la CPU disponibile, senza squilibri tra le repliche. - In circa **95 periodi CFS su 100** il container esaurisce la sua quota prima della fine del periodo e resta fermo fino al periodo successivo. Questo è un taglio continuo, non un picco occasionale. ## 3. Perché i due dati non coincidono Lo snapshot riporta circa 0,275 core in totale, mentre la misura diretta ne dà circa 0,594. Le cause più probabili sono una finestra di campionamento diversa (la media potrebbe includere un momento di carico più basso o un pod appena avviato) oppure un errore nella somma tra i pod. **Per valutare la saturazione i dati più affidabili sono quelli di Prometheus**, perché sono misurati per pod e includono i contatori CFS. Il secondo è quello che conta di più.

						| worker-app-…-9kj75 | 0,2971 core | 0,3 core | **99,0%** | **95,0%** | | **Totale** | **0,594 core** | **0,6 core** | **99,1%** | - | - Ho verificato tramite `kube_pod_container_resource_limits` che il limite di 0,3 core per container (`cpu-worker`) è davvero configurato, quindi non è solo un'ipotesi. - I due pod si comportano in modo identico. Il carico è distribuito in modo uniforme e consuma tutta la CPU disponibile, senza squilibri tra le repliche. - In circa **95 periodi CFS su 100** il container esaurisce la sua quota prima della fine del periodo e resta fermo fino al periodo successivo. Questo è un taglio continuo, non un picco occasionale. ## 3. Perché i due dati non coincidono Lo snapshot riporta circa 0,275 core in totale, mentre la misura diretta ne dà circa 0,594. Le cause più probabili sono una finestra di campionamento diversa (la media potrebbe includere un momento di carico più basso o un pod appena avviato) oppure un errore nella somma tra i pod. **Per valutare la saturazione i dati più affidabili sono quelli di Prometheus**, perché sono misurati per pod e includono i contatori CFS. Il secondo è quello che conta di più. ## 4. Effetti pratici - I tempi di risposta e di elaborazione si allungano: il Worker resta fermo per parte di ogni periodo di 100 ms. - Se c'è una coda, i lavori in attesa si accumulano. - La CPU media sembra "sotto il limite" ma in realtà è bloccata al tetto. Il consumo reale probabilmente supererebbe 0,3 core per pod se il limite non lo fermasse. ## 5. Cosa consiglio 1. **Subito:** alzare il limite CPU di `cpu-worker` a **0,5–0,6 core** per pod. Poi controllare di nuovo la percentuale di periodi tagliati (obiettivo: sotto il 5–10%) e il consumo reale senza limite. 2. **Scaling orizzontale (HPA):** il carico è uniforme e si distribuisce bene, quindi worker-app è adatta. Configurazione suggerita: `minReplicas: 2`, `maxReplicas: 4–5`, obiettivo sul 70% delle `requests` CPU. Attenzione: se le `requests` sono basse rispetto ai limiti, l'HPA potrebbe scalare in modo sbagliato, quindi vanno impostate su valori realistici (circa 0,25–0,3 core). 3. **Alternativa:** valutare di togliere il limite CPU e tenere solo le `requests`, se il nodo `thesis-cluster-control-plane` ha capacità libera. Lo scheduler continua a garantire le `requests`, ma il limite non frena più il pod. 4. **Monitoraggio:** impostare un allarme su `throttled_periods / periods > 0.25` per 10 minuti. Per le altre app non serve fare nulla su questo fronte: cache-app (circa 0,009 core) e demo-app (circa 0,002 core) sono praticamente ferme e lontane da qualsiasi rischio di taglio.

Claude Sonnet 5.5	39.50 s	10917	2724	13641	0.00	## Risposta breve Sì, i presupposti per il throttling CFS ci sono, e il throttling è già in corso. Il servizio più caricato è **worker-app**. ## Dati | Pod | CPU (rate 5m) | Limite | Utilizzo | Periodi CFS throttled | |---|---|---|---|---| | worker-app-84cf4dd446-9kj75 | 0,2971 core | 0,3 | **~99,0%** | **~94,7%** | | worker-app-84cf4dd446-ftlln | 0,2973 core | 0,3 | **~99,1%** | **~94,8%** | | **Totale** | **~0,594 core** | 0,6 | **~99,1%** | - | Il limite di 0,3 core per pod risulta effettivamente impostato (`kube_pod_container_resource_limits`). Il throttling è di circa 18,7–19,0 periodi al secondo per pod. ## Interpretazione - Entrambi i pod consumano quasi esattamente il limite. Un utilizzo piatto al 99% di un limite rigido è tipico di un carico che vorrebbe usare più CPU di quella concessa. - In quasi 19 periodi CFS su 20 il container esaurisce la quota e viene sospeso fino al periodo successivo. - Il consumo reale del workload è probabilmente superiore a 0,3 core per pod. Il valore misurato è tagliato dal limite, quindi non è la domanda reale. - La memoria è irrilevante: 5,6 MB totali per il servizio, quindi il collo di bottiglia è solo la CPU. ## Discrepanza con lo snapshot Lo snapshot indica per worker-app 0,275 core totali, cioè circa 46% di 0,6 core. Le query live danno invece ~0,594 core totali, cioè circa 99%. Non ho verificato da dove venga la differenza. Lo snapshot potrebbe essere stato campionato su una finestra diversa o in un momento di minor carico. Per la valutazione del throttling mi baso sui dati live, perché il ratio dei periodi throttled è una misura diretta e coerente con il ~99% di utilizzo. Se vuoi, posso controllare lo storico su MongoDB per capire quale dei due valori sia anomalo. Anche `container_cpu_cfs_throttled_seconds_total` ha restituito un risultato vuoto, quindi non posso quantificare il tempo totale di sospensione. Il throttling è comunque confermato dal conteggio dei periodi.

						## Raccomandazioni 1. **Alza subito il limite CPU** di worker-app, ad esempio a 0,5–0,6 core per pod. Poi osserva dove si stabilizza il consumo, per trovare la domanda reale. 2. **Configura un HPA su CPU** (target ~60–70% della request) con min 2 repliche. Così il carico si distribuisce su più pod invece di restare saturo a 2 pod con limite fisso. 3. **Controlla che le request siano coerenti con i limiti.** Un HPA calcola l'utilizzo rispetto alle request, quindi servono request realistiche. 4. **Non ridurre le repliche** di worker-app. 5. Per il resto, cache-app (0,009 core) e demo-app (0,002 core totali) sono praticamente idle. Hanno un ampio margine, e demo-app potrebbe in teoria scendere a 1 replica se non servono ridondanza o alta disponibilità.
GPT-4o Mini	5.81 s	1369	478	1847	0.32	Per valutare la percentuale di utilizzo della CPU rispetto a un limite teorico di 0.3 core per pod, calcoliamo l'utilizzo della CPU per ciascuna applicazione e confrontiamolo con il limite. ### Dati delle metriche attuali: - **cache-app**: - CPU utilizzata: 0.0088272375 core - Limite teorico: 0.3 core - Percentuale di utilizzo: \(\frac{0.0088272375}{0.3} \times 100 \approx 2.94\% \) - **demo-app**: - CPU utilizzata: 0.0023352025628008916 core (per 2 pod) - Limite teorico: 0.3 core - Percentuale di utilizzo: \(\frac{0.0023352025628008916}{0.3} \times 100 \approx 0.78\% \) - **worker-app**: - CPU utilizzata: 0.27494928595663526 core (per 2 pod) - Limite teorico: 0.3 core - Percentuale di utilizzo: \(\frac{0.27494928595663526}{0.3} \times 100 \approx 91.65\% \) ### Risultati: - **cache-app**: 2.94% - **demo-app**: 0.78% - **worker-app**: 91.65% ### Conclusione: Il servizio più caricato è **worker-app**, con un utilizzo della CPU del 91.65%. Questo valore è molto vicino al limite teorico di 0.3 core per pod. ### Raccomandazione: Sì, sussistono i presupposti per tagli di cicli da parte dello scheduler CFS, poiché l'utilizzo della CPU è molto elevato e potrebbe portare a una saturazione delle risorse. Si consiglia di monitorare attentamente il carico della CPU e considerare l'implementazione di un Horizontal Pod Autoscaler (HPA) per gestire meglio il carico di lavoro e garantire prestazioni ottimali.

GPT-4o	4.08 s	1369	446	1815	0.80	Per valutare se ci sono i presupposti per tagli di cicli da parte dello scheduler CFS, dobbiamo confrontare l'utilizzo della CPU con il limite teorico di 0.3 core per pod. ### Analisi delle metriche istantanee: 1. **cache-app**: - CPU utilizzata: 0.0088 core - Limite teorico: 0.3 core - Percentuale di utilizzo: \(\frac{0.0088}{0.3} \times 100 \approx 2.93\% \) 2. **demo-app**: - CPU utilizzata: 0.0023 core - Limite teorico per pod: 0.3 core - Limite teorico totale: 0.6 core (per 2 pod) - Percentuale di utilizzo: \(\frac{0.0023}{0.6} \times 100 \approx 0.39\% \) 3. **worker-app**: - CPU utilizzata: 0.2749 core - Limite teorico per pod: 0.3 core - Limite teorico totale: 0.6 core (per 2 pod) - Percentuale di utilizzo: \(\frac{0.2749}{0.6} \times 100 \approx 45.82\% \) ### Conclusione: - **worker-app** è il servizio più caricato con un utilizzo del 45.82% rispetto al limite teorico totale di 0.6 core. - Nonostante sia il più caricato, l'utilizzo della CPU è ancora ben al di sotto del limite teorico di 0.6 core totali. Pertanto, non ci sono attualmente i presupposti per tagli di cicli da parte dello scheduler CFS, poiché l'utilizzo non supera il limite disponibile. In sintesi, nessuna delle applicazioni sta saturando la CPU al punto da richiedere interventi dello scheduler CFS per limitare l'uso della CPU.

DOMANDA 9

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	28.27 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	24.21 s	4053	903	4956	0.70	Non ci sono stati riavvii imprevisti di container nel namespace `default` nell'ultima ora. Tutte le metriche `kube_pod_container_status_restarts_total` mostrano un incremento di 0 nell'ultima ora.
Gemini 3.6 Flash	34.00 s	7379	991	8370	0.80	In base all'analisi effettuata tramite query Prometheus sul namespace `default`: ### Esito Verifiche Riavvii (Ultima Ora) Dalla query `increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])`, **non si sono verificati riavvii di container nell'ultima ora** per nessuno dei pod attivi. Dettaglio per Pod/Container: * **`worker-app-84cf4dd446-ftlln`** (`cpu-worker`): **0** riavvii nell'ultima ora (Totale storico: 53) * **`worker-app-84cf4dd446-9kj75`** (`cpu-worker`): **0** riavvii nell'ultima ora (Totale storico: 53) * **`cache-app-5c6fd4f5cf-xxsz4`** (`cache-service`): **0** riavvii nell'ultima ora (Totale storico: 53) * **`demo-app-7958c64697-2qg9s`** (`nginx`): **0** riavvii nell'ultima ora (Totale storico: 53) * **`demo-app-7958c64697-4rzjl`** (`nginx`): **0** riavvii nell'ultima ora (Totale storico: 53) ### Considerazioni SRE I pod mostrano un contatore totale accumulato in passato, ma lo stato del workload nel namespace `default` negli ultimi 60 minuti risulta completamente stabile e privo di crash o riavvii imprevisti.
Claude Opus 5.5	56.65 s	19058	1876	20934	0.95	**Nell'ultima ora non c'è stato nessun riavvio imprevisto di container nel namespace `default`.** Però ogni container ha accumulato 53 riavvii da quando è stato creato. ### Riavvii nell'ultima ora Ho calcolato l'aumento del contatore dei riavvii (`kube_pod_container_status_restarts_total`) nell'ultima ora: | Deployment | Pod | Container | Riavvii nell'ultima ora | |---|---|---|---| | cache-app | cache-app-5c6fd4f5cf-xxsz4 | cache-service | **0** | | demo-app | demo-app-7958c64697-2qg9s | nginx | **0** | | demo-app | demo-app-7958c64697-4rzjl | nginx | **0** | | worker-app | worker-app-84cf4dd446-9kj75 | cpu-worker | **0** | | worker-app | worker-app-84cf4dd446-ftlln | cpu-worker | **0** | Ho controllato anche le ultime 24 ore: anche lì **0 riavvii** per tutti i container. ### I 53 riavvii totali - **Tutti i 5 container hanno lo stesso numero di riavvii: 53.** - **I pod esistono da circa 38 giorni** (3.320.835 secondi). - **Per tutti, il motivo dell'ultima terminazione è `Unknown`.** - **Il pod di raccolta metriche ha cambiato indirizzo IP più volte** (10.244.0.2, .3, .5, .14), quindi anche lui è stato riavviato. **Interpretazione:** il fatto che i numeri siano identici fa pensare che i riavvii siano dovuti al riavvio del nodo o del cluster, non a crash delle singole applicazioni. Il motivo `Unknown` compare di solito quando un container si ferma perché si ferma il nodo, non per un errore dell'applicazione. Un crash per memoria esaurita (OOMKilled) o un errore dell'applicazione (Error) avrebbero un motivo esplicito e numeri diversi tra un container e l'altro. È una situazione tipica dei cluster di laboratorio, come minikube o kind, che vengono spenti e riaccesi. ### Collegamento con le metriche attuali - **worker-app** usa 0,57 core in totale, cioè circa 0,28 core per pod. È il carico più alto, ma non provoca riavvii, quindi per ora la CPU non sembra causare instabilità. - **cache-app** e **demo-app** sono quasi inattive (CPU 0,008 e 0,0 core). Questo esclude che siano riavviate per un sovraccarico. ### Cosa consiglio 1. **Non serve intervenire subito**: i workload sono stabili da almeno 24 ore. 2. **Verifica la causa dei 53 riavvii** con `kubectl get events -A` e `kubectl describe node`. Se sono riavvii del nodo programmati o fatti a mano, si possono ignorare. 3. **Imposta un allarme** sulla query `increase(kube_pod_container_status_restarts_total{namespace="default"}[15m]) > 0`. Così un crash vero viene notato subito e non si confonde con il totale accumulato. 4. **Tieni d'occhio worker-app**: è l'unico con un consumo di CPU significativo. Se compaiono riavvii con motivo `OOMKilled` o `Error`, valuta di alzare i limiti di risorse o di attivare la scalabilità automatica (HPA) basata sulla CPU.

Claude Sonnet 5.5	33.92 s	7808	833	8641	0.95	**Nell'ultima ora non ci sono stati riavvii imprevisti** nel namespace `default`. Per tutti e 5 i container `increase(kube_pod_container_status_restarts_total[1h])` vale **0**. | Pod | Container | Riavvii nell'ultima ora | Contatore cumulativo | |---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 53 | | demo-app-7958c64697-2qg9s | nginx | 0 | 53 | | demo-app-7958c64697-4rzjl | nginx | 0 | 53 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 53 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 53 | **Anomalia sul contatore cumulativo.** Tutti i container mostrano esattamente 53 riavvii totali, anche se appartengono ad applicazioni e pod diversi. Non ho dati per spiegarlo, quindi ti indico solo le ipotesi: - Potrebbe essere un artefatto della raccolta o dell'ambiente di test, ad esempio un riavvio del nodo o del cluster che ha riavviato tutti i container insieme. Un contatore identico su workload indipendenti è insolito. - Dato che `increase` è 0, i riavvii sono avvenuti prima dell'ultima ora. **Verifiche che ti suggerisco:** - Controlla un intervallo più lungo, ad esempio `increase(...[24h])` o `[7d]`, per capire quando sono avvenuti i 53 riavvii. - Controlla `kube_pod_container_status_last_terminated_reason` per vedere il motivo dell'ultima terminazione (OOMKilled, Error e così via). - Verifica con `kubectl describe pod` gli eventi del nodo. Se vuoi, eseguo io queste query.
GPT-4o Mini	31.35 s	4149	163	4312	0.00	Non sono riuscito a eseguire la query per verificare i riavvii imprevisti dei container nel namespace default nell'ultima ora a causa di un errore nella richiesta. Ti consiglio di sottomettere questa richiesta a un modello più avanzato o a un amministratore di sistema con accesso diretto a Prometheus per ulteriori indagini.

GPT-4o	25.26 s	2718	399	3117	0.70	Non ci sono stati riavvii imprevisti di container nel namespace "default" nell'ultima ora. Ecco alcune raccomandazioni basate sulle metriche istantanee attuali: 1. **Cache-app**: - **CPU e Memoria**: L'utilizzo della CPU è molto basso (0.0078 core) e la memoria utilizzata è minima (1.29 MB). - **Raccomandazione**: Considerando l'uso minimo delle risorse, potresti valutare la possibilità di ridurre ulteriormente le risorse allocate o mantenere la configurazione attuale se è già ottimizzata per il carico previsto. 2. **Demo-app**: - **CPU e Memoria**: Non ci sono utilizzi di CPU registrati e la memoria utilizzata è di 14.95 MB. - **Raccomandazione**: L'applicazione sembra essere inattiva o in uno stato di idle. Potresti considerare di ridurre il numero di repliche se non è necessario mantenere due pod attivi. 3. **Worker-app**: - **CPU e Memoria**: L'utilizzo della CPU è significativo (0.5678 core) rispetto alle altre applicazioni, mentre la memoria utilizzata è di 5.82 MB. - **Raccomandazione**: Se l'utilizzo della CPU è costante, potresti considerare di scalare orizzontalmente aggiungendo più repliche per distribuire il carico e migliorare la resilienza. Queste raccomandazioni sono basate sulle metriche attuali e potrebbero necessitare di ulteriori analisi storiche per decisioni a lungo termine.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6385 (mini: $0.0787 | 4o: $1.5598)

DOMANDA 9 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	211.73 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	21.63 s	4090	778	4868	0.50	Nell'arco degli ultimi 60 minuti, non si sono verificati crash o restart anomali dei container nel namespace `default`. Tutte le metriche di riavvio (`kube_pod_container_status_restarts_total`) mostrano un incremento pari a zero.
Gemini 3.6 Flash	30.41 s	7435	1297	8732	0.80	Dall'analisi effettuata tramite Prometheus sulle metriche di riavvio dei container (`kube_pod_container_status_restarts_total`) nel namespace **`default`**: ### Risultati del controllo (ultimi 60 minuti): * **Incremento dei riavvii negli ultimi 60 min (`increase(...[60m])`)**: **0** per tutti i pod e container attivi. * **Stato dei singoli pod/container**: * **`cache-app`** (`cache-app-5c6fd4f5cf-xxsz4` / container `cache-service`): **0 nuovi riavvii** (totale cumulativo storico: 53) * **`demo-app`** (`demo-app-7958c64697-2qg9s` / container `nginx`): **0 nuovi riavvii** (totale cumulativo storico: 53) * **`demo-app`** (`demo-app-7958c64697-4rzjl` / container `nginx`): **0 nuovi riavvii** (totale cumulativo storico: 53) * **`worker-app`** (`worker-app-84cf4dd446-9kj75` / container `cpu-worker`): **0 nuovi riavvii** (totale cumulativo storico: 53) * **`worker-app`** (`worker-app-84cf4dd446-ftlln` / container `cpu-worker`): **0 nuovi riavvii** (totale cumulativo storico: 53) --- ### Esito e Considerazioni **Non si è verificato alcun crash o riavvio anomalo negli ultimi 60 minuti** per i carichi di lavoro presenti nel namespace `default`. Il sistema risulta stabile nell'arco temporale richiesto.
Claude Opus 5.5	73.13 s	32421	3235	35656	0.00	**Sì, nell'ultima ora c'è stato un riavvio: tutti e 5 i container sono ripartiti insieme circa 20 minuti fa.** Il conteggio diretto dei restart sull'ultima ora indica 0, ma è un falso negativo. Il riavvio è avvenuto a livello del nodo. Non risulta che una singola applicazione sia andata in crash. ### Cosa mostrano i dati | Verifica (PromQL) | Risultato | |---|---| | Restart nell'ultima ora (`increase(...[60m])`) | 0 su tutti i container | | Restart totali (`kube_pod_container_status_restarts_total`) | **53 su tutti e 5 i container**, valore identico | | Tempo dall'ultimo avvio dei container (`kube_pod_container_state_started`) | **~1218–1223 s (~20 min)** per tutti | | Tempo dall'avvio del kubelet | ~1254 s (~21 min) | | Motivo dell'ultima terminazione | `Unknown` per tutti | | Container bloccati in attesa (es. CrashLoopBackOff) | Nessuno | | Valori del contatore nelle 24h | 50, 51, 52, 53, uno per ciascuno dei 4 IP da cui Prometheus ha raccolto le metriche di stato del cluster | ### Perché lo zero è sbagliato 1. **Il valore viene da fonti diverse.** Il servizio che pubblica questi contatori (kube-state-metrics) cambia IP a ogni riavvio del cluster: finora ne ha usati 4. Prometheus quindi tratta ogni IP come una serie separata e piatta, e il calcolo sull'ultima ora non vede il passaggio da 52 a 53. 2. **Mancano dati.** Nell'ultima ora ci sono solo ~40 campioni, quindi un periodo non è stato registrato, quello in cui il cluster era fermo. 3. **I tempi coincidono.** Il kubelet è ripartito circa 21 minuti fa e tutti i container circa 20 minuti fa, uno o due secondi dopo l'altro. ### Come interpreto i dati - Tutti i container hanno lo stesso numero di restart (53), con motivo `Unknown` e ripartenze simultanee. È il tipico effetto di un riavvio del nodo o del runtime (`thesis-cluster-control-plane`, probabilmente un cluster kind/Docker fermato e riavviato). Non è un crash applicativo. - Non ci sono segnali di crash dovuti all'esaurimento della memoria (`OOMKilled`), di errori dell'applicazione (`Error`) o di riavvii a ripetizione (`CrashLoopBackOff`). Il consumo attuale lo conferma: la memoria va da 1,3 a 15 MB e la CPU di worker-app è di circa 0,57 core in totale. - I 53 restart sono il totale accumulato dai riavvii del cluster nel tempo, non un problema dei singoli workload.

						3. **I tempi coincidono.** Il kubelet è ripartito circa 21 minuti fa e tutti i container circa 20 minuti fa, uno o due secondi dopo l'altro.
						
						### Come interpreto i dati
						- Tutti i container hanno lo stesso numero di restart (53), con motivo `Unknown` e ripartenze simultanee. È il tipico effetto di un riavvio del nodo o del runtime (`thesis-cluster-control-plane`, probabilmente un cluster kind/Docker fermato e riavviato). Non è un crash applicativo.
						- Non ci sono segnali di crash dovuti all'esaurimento della memoria (`OOMKilled`), di errori dell'applicazione (`Error`) o di riavvii a ripetizione (`CrashLoopBackOff`). Il consumo attuale lo conferma: la memoria va da 1,3 a 15 MB e la CPU di worker-app è di circa 0,57 core in totale.
						- I 53 restart sono il totale accumulato dai riavvii del cluster nel tempo, non un problema dei singoli workload.
						
						### Raccomandazioni
						1. **Rendere affidabili gli alert sui restart.** Raggruppa per pod e container, così il cambio di IP non conta:
						``promql
						sum by (namespace, pod, container) (increase(kube_pod_container_status_restarts_total{namespace="default"}[1h])) > 0
						``
						Aggiungi anche un controllo sui container avviati da poco, che non dipende dal contatore:
						``promql
						(time() - kube_pod_container_state_started{namespace="default"}) < 3600
						``
						2. **Controllare il nodo.** Usa `kubectl get events -A --sort-by=.lastTimestamp` e `kubectl describe node thesis-cluster-control-plane`, e verifica se il nodo è stato riavviato di proposito.
						3. **Al momento non serve intervenire su HPA o repliche:** il riavvio non è legato al carico.
						
						Un limite: Prometheus non ha l'orario preciso di ogni riavvio e l'ultima ora ha dei buchi. Per capire con certezza perché è ripartito il nodo servono gli eventi Kubernetes o i log del nodo, che con questi strumenti non posso leggere.

Claude Sonnet 5.5	37.15 s	7939	1490	9429	0.90	**Negli ultimi 60 minuti non risultano crash né restart dei container nel namespace `default`.** C'è però un'anomalia nei contatori cumulativi, descritta sotto. **Restart nell'ultima ora** Ho usato `increase(kube_pod_container_status_restarts_total[60m])`. | Pod | Container | Restart (60 min) | |---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | | demo-app-7958c64697-2qg9s | nginx | 0 | | demo-app-7958c64697-4rzjl | nginx | 0 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | **Anomalia nei contatori cumulativi** - **Valore identico:** tutti e 5 i container hanno `kube_pod_container_status_restarts_total = 53`. Pod di tre Deployment diversi, con ReplicaSet hash diversi, non dovrebbero avere lo stesso numero di restart. - **Motivo dell'ultima terminazione:** `kube_pod_container_status_last_terminated_reason` è `Unknown` per tutti. Non c'è quindi nessun `OOMKilled` o `Error` che indichi un problema specifico dell'applicazione. **Interpretazione** Il valore identico fa pensare a un evento comune a tutto il cluster, ad esempio riavvii ripetuti del nodo o dell'ambiente (per esempio un cluster locale tipo minikube/kind), oppure a un artefatto di kube-state-metrics. Non fa pensare a un problema dei singoli workload. L'evento è comunque precedente all'ultima ora, perché in questo intervallo il contatore non è cresciuto. Questo è un mio sospetto: dai dati raccolti non posso confermarlo. **Raccomandazioni** 1. Il carico attuale è stabile: `worker-app` usa 0,57 core e la memoria è bassa su tutte le app. Per ora non serve nessuna azione sui restart. 2. Controlla con `kubectl describe pod` l'ultimo `State/Reason` e l'`Exit Code`. Controlla anche gli eventi del nodo (`kubectl get events`, `kubectl describe node`) per capire quando e perché si sono accumulati i 53 restart. 3. Se vuoi, posso estendere l'analisi a 24 ore o 7 giorni con `increase(...[24h])` e con `kube_pod_start_time` e `kube_pod_container_status_last_terminated_timestamp`. Così si individua quando è avvenuto l'evento. 4. Se i restart continuano a salire, imposta un alert come `increase(kube_pod_container_status_restarts_total[15m]) > 0`.
GPT-4o Mini	27.97 s	4179	197	4376	0.00	Non sono riuscito a eseguire la query per controllare i contatori di riavvio dei container in Prometheus a causa di un errore nella richiesta. Questo potrebbe essere dovuto a un problema con la sintassi della query o con la disponibilità delle metriche. Ti consiglio di sottomettere questa richiesta a un modello più avanzato o a un amministratore di sistema che possa eseguire direttamente le query su Prometheus per verificare se ci sono stati crash o riavvii anomali dei container nel namespace default negli ultimi 60 minuti.
GPT-4o	20.42 s	2777	109	2886	0.50	Nell'arco degli ultimi 60 minuti, non si sono verificati crash o restart anomali dei container nel namespace "default". Non ci sono stati incrementi nei contatori di riavvio dei container.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6472 (mini: $0.0794 | 4o: $1.5678)

DOMANDA 9B

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	311.67 ms	0	0	0	N/A	0 riavvii imprevisti rilevati nell'ultima ora
Gemini 2.5 Flash	17.26 s	4069	644	4713	0.70	Nessun container nel namespace `default` ha registrato riavvii improvvisi nell'ultima ora. Tutte le metriche di restart (`kube_pod_container_status_restarts_total`) mostrano un incremento di 0 nell'intervallo specificato.
Gemini 3.6 Flash	27.59 s	7403	1196	8599	0.80	È stata eseguita l'analisi degli incrementi dei riavvii nell'ultima ora tramite la metrica Prometheus `kube_pod_container_status_re starts_total` per tutti i container nel namespace `default`. --- ### 📊 Esito dell'Analisi (`increase(...[1h])`) | Pod | Container | Conteggio totale riavvii (corrente) | Incrementi nell'ultima ora | | :--- | :--- | :---: | :---: | | `cache-app-5c6fd4f5cf-xxsz4` | `cache-service` | 53 | **0** | | `demo-app-7958c64697-2qg9s` | `nginx` | 53 | **0** | | `demo-app-7958c64697-4rzjl` | `nginx` | 53 | **0** | | `worker-app-84cf4dd446-9kj75` | `cpu-worker` | 53 | **0** | | `worker-app-84cf4dd446-ftlln` | `cpu-worker` | 53 | **0** | --- ### 🔍 Considerazioni SRE - **Nessun riavvio recente**: Nessun container nel namespace `default` ha registrato riavvii o crash nell'ultima ora (`increase = 0`). - **Storico cumulativo**: Sebbene i pod presentino un valore cumulativo storico di 53 riavvii ciascuno, tale contatore è rimasto stabile nell'intervallo temporale richiesto. I carichi di lavoro sono attualmente stabili.
Claude Opus 5.5	66.19 s	31620	3132	34752	0.00	**Sì: tutti e 5 i container del namespace `default` sono ripartiti circa 24 minuti fa, quindi dentro l'ultima ora.** La query `increase(kube_pod_container_status_restarts_total[1h])` restituisce **0** per tutti, ma è un **falso negativo**: Prometheus non ha dati che coprano il momento del riavvio, quindi non può contarlo. ### Cosa ho trovato | Pod | Container | Riavvii totali | In esecuzione da | Ultimo motivo di terminazione | |---|---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 53 | ~1452 s (~24 min) | Unknown | | demo-app-7958c64697-2qg9s | nginx | 53 | ~1457 s | Unknown | | demo-app-7958c64697-4rzjl | nginx | 53 | ~1457 s | Unknown | | worker-app-84cf4dd446-9kj75 | cpu-worker | 53 | ~1457 s | Unknown | | worker-app-84cf4dd446-ftlln | cpu-worker | 53 | ~1453 s | Unknown | ### Perché `increase()` dà 0 - Nell'ultima ora Prometheus ha campioni solo degli ultimi ~18 minuti, mentre i container sono ripartiti ~24 minuti fa. Il riavvio è avvenuto prima del primo campione, quindi il contatore risulta fermo a 53. - Il componente che espone il conteggio dei riavvii (`kube-state-metrics`) ha cambiato indirizzo IP più volte (10.244.0.3, .5, .14 e ora .2). Ogni cambio crea serie nuove che partono da zero, e `increase()` non vede il salto nemmeno su 24 ore. ### Cosa indicano i dati - **Riavvio di tutto il nodo o del cluster, non un crash delle singole applicazioni.** Tre indizi: 1. Tutti i container sono ripartiti entro **5 secondi** l'uno dall'altro. 2. Hanno **tutti lo stesso conteggio (53)**, anche se fanno lavori molto diversi (nginx inattivo, cache con scritture su disco, worker che usa CPU). 3. Il motivo di terminazione è sempre **`Unknown`**, tipico di un nodo o di un runtime riavviato. Non risulta `OOMKilled` né `Error`. - **Nessun riavvio dei container in corso.** Da quando ci sono campioni (~18 min) il contatore è fermo a 53 su tutti i pod. Le metriche attuali sono coerenti con questo: worker-app usa ~0,57 core, la memoria è bassa ovunque, non c'è pressione di memoria. - **53 riavvii a testa sono comunque tanti.** È probabile che il nodo, o l'ambiente locale (kind/minikube), venga riavviato spesso.

						nodo, o l'ambiente locale (kind/minikube), venga riavviato spesso. ### Raccomandazioni 1. **Controllare il nodo:** `kubectl get nodes`, `kubectl describe node`, `journalctl -u kubelet` e i log del container runtime intorno a 24 minuti fa. Se è un ambiente locale, verificare se l'host o Docker sono stati riavviati o sospesi. 2. **Rendere Prometheus persistente:** senza storage persistente, ogni riavvio cancella lo storico e le query `increase()`/`rate()` sui riavvii perdono valore. 3. **Per gli alert sui riavvii, affiancare a `increase()` un controllo sull'età del container**, ad esempio `(time() - kube_pod_container_state_started) < 3600`. Così si rilevano i riavvii anche quando lo storico si interrompe. 4. **Non serve agire su scaling o risorse:** non risultano crash per mancanza di memoria né saturazione legata ai riavvii. **Punto da confermare:** la causa "riavvio del nodo" è dedotta dai dati, non verificata. Per esserne certi servono gli eventi e i log del nodo, che non posso consultare con gli strumenti che ho.
Claude Sonnet 5.5	27.56 s	7830	1003	8833	0.90	**Nell'ultima ora nessun container del namespace `default` ha avuto riavvii.** `increase(kube_pod_container_status_restarts_total[1h])` vale 0 per tutti e 5 i pod. | Pod | Container | Incremento 1h | Contatore totale | |---|---|---|---| | cache-app-5c6fd4f5cf-xxsz4 | cache-service | 0 | 53 | | demo-app-7958c64697-2qg9s | nginx | 0 | 53 | | demo-app-7958c64697-4rzjl | nginx | 0 | 53 | | worker-app-84cf4dd446-9kj75 | cpu-worker | 0 | 53 | | worker-app-84cf4dd446-ftlln | cpu-worker | 0 | 53 | **Contatore a 53 su tutti i pod.** Ogni container ha lo stesso valore, anche se appartengono a Deployment diversi e hanno carichi molto diversi (nginx idle, worker a circa 0,57 core in totale, cache). È difficile che 53 crash indipendenti coincidano esattamente. Può dipendere da un evento comune, come il riavvio del nodo o del runtime, oppure da un artefatto dell'ambiente. Può anche dipendere da come kube-state-metrics espone il dato. Dalle metriche non posso stabilire quale sia la causa. **Cosa verificare** - Controlla `kubectl describe pod` e gli eventi del namespace, per vedere `Last State`, `Reason` ed `Exit Code`. - Interroga una finestra più lunga, ad esempio `increase(...[24h])` o `[7d]`, per capire quando sono avvenuti i 53 riavvii. Se il valore è fermo da tempo, il problema è passato e al momento non c'è instabilità. - Le metriche istantanee non mostrano segni di crash loop. Se ne vedi altri sintomi, controlla anche i log precedenti con `kubectl logs --previous`.

GPT-4o Mini	17.77 s	6175	223	6398	0.80	Dall'analisi delle metriche di riavvio dei container negli ultimi 60 minuti, non ci sono stati riavvii registrati per nessuno dei pod nel namespace `default`. Ecco un riepilogo dei pod attivi e dei loro stati: - **cache-app**: `cache-app-5c6fd4f5cf-xxsz4` - 0 riavvii - **demo-app**: `demo-app-7958c64697-2qg9s` e `demo-app-7958c64697-4rzjl` - 0 riavvii - **worker-app**: `worker-app-84cf4dd446-9kj75` e `worker-app-84cf4dd446-ftlln` - 0 riavvii Se hai bisogno di ulteriori informazioni o di un'analisi più approfondita, fammi sapere!
GPT-4o	15.94 s	2727	66	2793	0.50	Nell'ultima ora, non ci sono stati riavvii improvvisi dei container nel namespace "default". Le metriche di restart non mostrano alcun incremento.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6558 (mini: $0.0805 | 4o: $1.5753)

DOMANDA 10

```
--------------------------------------------------------------------------------------------------------------
ESECUZIONE DOMANDA [10]: Dobbiamo sottomettere un nuovo batch da 12 pod con vincolo critico di CPU. In base alle risorse attuali, accanto a quale applicazione è più sicuro collocarli e da quale devono essere isolati?
--------------------------------------------------------------------------------------------------------------
Invocazione modelli in corso...
/home/filip/AgentLab/.venv/lib/python3.14/site-packages/langchain_google_genai/chat_models.py:3908: UserWarning: Model 'gemini-3.6-flash' uses fixed sampling defaults; the sampling parameter(s) temperature wi
ll be ignored.
  request = self._build_request_config(
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
env['PS1'] contains unexpanded variable reference: '(.venv) \\[\\e]0;\\u@\\h: \\w\\a\\]${debian_chroot:+($debian_chroot)}\\[\\033[01;32m\\]\\u@\\h\\[\\033[00m\\]:\\[\\033[01;34m\\]\\w\\[\\033[00m\\]\\$ '
Prometheus MCP server running on stdio
```

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	3.18 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5678 cores)
Gemini 2.5 Flash	3.30 s	1652	460	2112	0.70	In base alle metriche istantanee correnti: * **Collocazione più sicura:** I nuovi 12 pod dovrebbero essere collocati accanto a **`demo-app`**. Attualmente, `demo-app` ha un utilizzo della CPU pari a **0.0 core**, indicando che ha ampie risorse disponibili e non dovrebbe competere per la CPU con i nuovi pod. * **Isolamento consigliato:** I nuovi pod dovrebbero essere isolati da **`worker-app`**. `worker-app` ha l'utilizzo della CPU più elevato tra le applicazioni attuali, pari a **0.5678 core**, e potrebbe causare contesa di risorse con un batch di 12 nuovi pod con vincolo critico di CPU.

Gemini 3.6 Flash	12.20 s	1666	2057	3723	0.90	In base all'analisi dello snapshot delle metriche correnti, ecco la valutazione e le raccomandazioni per la collocazione del nuovo batch da 12 pod con **vincolo critico di CPU**: --- ### 📊 Analisi dell'Utilizzo della CPU Attuale 1. **`demo-app`** - **Pod attivi:** 2 (`demo-app-7958c64697-2qg9s`, `demo-app-7958c64697-4rzjl`) - **Consumo CPU:** **`0.0 cores`** - **Memoria:** `14.95 MB` - **Stato:** Completamente **idle** / nessun consumo computazionale attivo. 2. **`cache-app`** - **Pod attivi:** 1 (`cache-app-5c6fd4f5cf-xxsz4`) - **Consumo CPU:** **`0.0078 cores`** (~7.8 mCPU) - **Memoria:** `1.285 MB` - **Stato:** Consumo di CPU trascurabile (presenta solo attività I/O su disco a 468 KB/s). 3. **`worker-app`** - **Pod attivi:** 2 (`worker-app-84cf4dd446-9kj75`, `worker-app-84cf4dd446-ftlln`) - **Consumo CPU:** **`0.5678 cores`** (~567.8 mCPU) - **Memoria:** `5.82 MB` - **Stato:** **CPU-Bound / Carico elevato**, assorbe quasi la totalità della CPU utilizzata nell'intero cluster. --- ### 💡 Raccomandazioni di Collocazione e Isolamento * **✅ Dove collocare il batch (Vicino sicuro): `demo-app`** * `demo-app` registra un utilizzo della CPU pari a **0.0 cores** (completamente idle). Collocare il batch da 12 pod in affiancamento a `demo-app` (tramite regole di *podAffinity* o *topologySpreadConstraints*) garantisce la totale assenza di contesa per i cicli di calcolo CPU. * *(Nota: Anche `cache-app` rappresenta un'alternativa valida per la CPU, ma presenta attività di scrittura su disco).* * **❌ Da quale applicazione isolare il batch: `worker-app`** * `worker-app` è l'unica applicazione attualmente con un carico CPU significativo (~0.568 cores). * Collocare un batch intensivo in termini di CPU sugli stessi nodi di `worker-app` genererebbe un'elevata contesa delle risorse (*CPU throttling* / *starvation*), compromettendo sia la latency di `worker-app` che l'esecuzione del batch critico. Si consiglia vivamente di impostare regole di **`podAntiAffinity`** rispetto ai pod di `worker-app`.

Claude Opus 5.5	48.45 s	18969	2739	21708	0.86	**Risposta breve:** mettete il batch accanto a **demo-app** e tenetelo **lontano da worker-app**. C'è però un limite importante: oggi il cluster ha **un solo nodo**, quindi non potete ancora isolarli fisicamente. ## Cosa ho verificato Lo snapshot non bastava, quindi ho interrogato Prometheus su nodi, requests/limits e throttling CFS. | App | CPU reale | Request / Limit per pod | Throttling CFS | Profilo | |---|---|---|---|---| | **worker-app** (2 pod) | ~0,58 core (0,295 + 0,287) | 0,15 / **0,3** | **~95%** (0,96 e 0,94) | Usa intensamente la CPU ed è sempre al limite | | **cache-app** (1 pod) | ~0,01 core | 0,05 / 0,1 | ~48% | Brevi picchi di CPU; scrive molto su disco (~468 KB/s) | | **demo-app** (2 pod) | **0 core** | 0,05 / 0,1 | nessun dato (non usa CPU) | Ferma | Nota: lo throttling misura quanto spesso un pod viene bloccato perché ha raggiunto il proprio limite CPU. ## 1. Il vicino più sicuro: demo-app - Usa 0 CPU e non viene mai bloccato, quindi non toglie cicli CPU al batch. - Ha anche poca RAM (~15 MB) e non usa né disco né rete. ## 2. Da isolare: worker-app - I suoi due pod sono bloccati dal limite nel 94–96% dei casi. Significa che consumerebbero più CPU se potessero: oggi solo il limite di 0,3 core li ferma. - Se qualcuno alza o rimuove quel limite, o se worker-app viene scalata, competerà direttamente con i vostri 12 pod. Rischio concreto di rallentamenti e latenza imprevedibile. **cache-app** è una via di mezzo: consuma poca CPU media, ma ha picchi brevi e un I/O su disco costante. Meglio non metterle accanto il batch se il carico è sensibile anche ai tempi di risposta o al disco.

						risposta o al disco. ## ⚠️ Limite di topologia Tutti e 5 i pod girano su **`thesis-cluster-control-plane`** (8 core allocabili, 1,4 core già prenotati). Quindi: - **Oggi "isolare" non è possibile fisicamente.** Le regole di anti-affinity non hanno un altro nodo dove spostare i pod. Con `required` i pod resterebbero `Pending`; con `preferred` non cambierebbe nulla. - **Spazio disponibile:** restano circa 6,6 core da prenotare, cioè **al massimo ~0,55 core di request per pod** per 12 pod. Valori più alti lasciano dei pod `Pending`. ## Raccomandazioni pratiche 1. **Request = limit su ogni pod del batch** (classe QoS Guaranteed). Con CPU Manager in modalità `static` e valori interi potete anche riservare core dedicati: è il vero isolamento possibile su un nodo solo. 2. **Bloccare worker-app** con un `LimitRange` o una `ResourceQuota` nel namespace, così nessuno ne alza i limiti mentre il batch gira. 3. **Isolamento vero:** aggiungete un nodo dedicato con label e taint (es. `workload=batch-cpu:NoSchedule`) e fate puntare lì il batch con nodeSelector e toleration. Poi aggiungete una regola anti-affinity verso `app=worker-app`. 4. **Nel frattempo:** se il vincolo CPU è stretto, valutate di ridurre worker-app a 1 replica mentre il batch è in esecuzione. Questi dati descrivono la situazione attuale, non la storia passata. Se volete, posso controllare i dati storici su MongoDB (`get_historical_metrics`) per verificare che il consumo di worker-app sia costante e non un picco momentaneo.

Claude Sonnet 5.5	10.84 s	2811	1231	4042	0.95	**Raccomandazione:** metti il batch da 12 pod accanto a **demo-app**. Tienilo isolato da **worker-app**. **Dati dello snapshot** | App | Pod | CPU (core) | CPU per pod | Altro | |---|---|---|---|---| | worker-app | 2 | 0,568 | circa 0,28 | Memoria 5,8 MB | | cache-app | 1 | 0,0078 | 0,0078 | Scrittura disco circa 468 KB/s | | demo-app | 2 | 0,0 | 0 | Memoria 15 MB, nessun I/O né rete | **Perché demo-app** - Ha CPU a 0 e nessun traffico disco o rete, quindi è di fatto idle. Non compete per la CPU con i pod critici. - Ha 2 repliche, quindi è la più facile da spostare o ridurre se serve altro spazio. **Perché isolare worker-app** - È l'unico carico che usa CPU in modo significativo, circa 0,57 core in totale, cioè circa 70 volte la CPU di cache-app. - Con un vincolo critico sulla CPU, il rischio di contesa, throttling e noisy neighbour è massimo accanto a lui. Il suo carico potrebbe anche salire se scala. - Usa pochissima memoria (5,8 MB), quindi la CPU è il suo vero collo di bottiglia. Se il nodo è condiviso, i suoi picchi sottrarrebbero core ai 12 pod. **Su cache-app** - Usa pochissima CPU (0,0078 core), quindi per la CPU è un vicino accettabile. - Scrive su disco a circa 468 KB/s. Se i 12 pod fanno I/O o condividono lo stesso volume o nodo, c'è un rischio di contesa sul disco. Come seconda scelta va bene, ma dopo demo-app. **Limiti dell'analisi** - Lo snapshot non contiene i dati di capacità dei nodi (CPU allocabile), né le `requests`/`limits` dei pod, né il nodo su cui gira ciascun pod. Prima di schedulare, verifica che il nodo scelto abbia abbastanza core allocabili per 12 pod. - Il valore 0 di demo-app è istantaneo. Se vuoi essere sicuro che non abbia picchi periodici, posso controllare lo storico con `get_historical_metrics`. - Per garantire l'isolamento in modo vincolante, usa `nodeAffinity`/`podAntiAffinity` contro `worker-app`, oppure un node pool dedicato con taint e toleration. Imposta anche `requests` CPU uguali ai `limits`, così i pod hanno la classe QoS Guaranteed.

GPT-4o Mini	4.92 s	1346	327	1673	0.00	Analizzando le metriche attuali delle applicazioni: 1. **cache-app**: - CPU: 0.0078 cores - Memoria: 1.29 MB - Pod attivi: 1 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.95 MB - Pod attivi: 2 3. **worker-app**: - CPU: 0.5678 cores - Memoria: 5.82 MB - Pod attivi: 2 ### Raccomandazioni: - **Applicazione più sicura per collocare i nuovi pod**: **cache-app** - Ha un utilizzo di CPU molto basso (0.0078 cores) e una memoria molto bassa (1.29 MB). Questo la rende un candidato ideale per l'aggiunta di nuovi pod, poiché non sta attualmente saturando le risorse. - **Applicazione da cui isolarsi**: **worker-app** - Ha un utilizzo di CPU significativamente più alto (0.5678 cores) rispetto alle altre applicazioni. Collocare nuovi pod accanto a questa applicazione potrebbe portare a saturazione delle risorse CPU, specialmente considerando che si stanno aggiungendo 12 nuovi pod con un vincolo critico di CPU. ### Conclusione: È consigliabile collocare i nuovi pod nella **cache-app** e isolarsi dalla **worker-app** per evitare problemi di saturazione delle risorse.

GPT-4o	4.31 s	1346	335	1681	0.72	Per determinare dove collocare i nuovi pod con vincolo critico di CPU, analizziamo le metriche istantanee delle applicazioni attuali: 1. **cache-app**: - CPU: 0.0078 cores - Memoria: 1.29 MB - Disco e rete: attività minima 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.95 MB - Disco e rete: nessuna attività 3. **worker-app**: - CPU: 0.5678 cores - Memoria: 5.82 MB - Disco e rete: nessuna attività **Raccomandazioni:** - **Isolamento**: I nuovi pod dovrebbero essere isolati dalla **worker-app**, poiché sta utilizzando una quantità significativa di CPU (0.5678 cores). Questo potrebbe portare a saturazione della CPU se ulteriori pod con vincolo critico di CPU vengono collocati accanto ad essa. - **Collocazione Sicura**: I nuovi pod possono essere collocati accanto a **cache-app** o **demo-app**. Entrambe le applicazioni hanno un utilizzo di CPU molto basso o nullo, quindi è più sicuro collocare i nuovi pod accanto a queste applicazioni per evitare problemi di saturazione della CPU. In sintesi, per evitare problemi di saturazione della CPU, colloca i nuovi pod accanto a **cache-app** o **demo-app** e isolali dalla **worker-app**.

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6629 (mini: $0.0809 | 4o: $1.5820)

DOMANDA 10 A

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.77 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5678 cores)
Gemini 2.5 Flash	4.63 s	1655	685	2340	0.76	Per la pianificazione del batch computazionale da 12 pod, che è sensibile ai cicli CPU, ecco le raccomandazioni basate sulle metriche attuali: **Regole di Affinità (conviene definire):** * **`cache-app`**: Con un consumo di CPU di soli `0.00777` core, `cache-app` è un ottimo candidato per definire regole di affinità. I pod del batch computazionale possono essere co-locati con `cache-app` senza causare contesa significativa per le risorse CPU. * **`demo-app`**: Attualmente con `0.0` core di CPU utilizzati, `demo-app` è anch'esso un candidato ideale per regole di affinità. **Regole di Anti-Affinità (indispensabile applicare):** * **`worker-app`**: Con un consumo di CPU di `0.5678` core, `worker-app` è l'applicazione da cui è indispensabile applicare regole di anti-affinità. Co-locare i pod del batch computazionale con `worker-app` causerebbe una significativa contesa per le risorse CPU, compromettendo le prestazioni di entrambi i carichi di lavoro.

Gemini 3.6 Flash	12.48 s	1669	1867	3536	0.76	Sulla base delle metriche istantanee e della topologia attuale dei carichi di lavoro, ecco l'analisi SRE e le raccomandazioni per il placement del nuovo batch computazionale da 12 pod (sensibile ai cicli CPU): --- ### 1. Profilo di Carico dei Servizi Correnti * **`worker-app` (2 pod):** * **CPU:** **~0.568 core** (Consumo elevato, carico CPU-bound attivo) * **I/O Disco / Rete:** 0.0 bps * **`demo-app` (2 pod):** * **CPU:** **0.0 core** (Completamente Idle / Inattivo) * **Memoria:** 14.95 MB * **I/O Disco / Rete:** 0.0 bps * **`cache-app` (1 pod):** * **CPU:** ~0.008 core (Negligibile) * **Disco:** ~468 KB/s (In scrittura costante) --- ### 2. Raccomandazioni per le Regole di Affinità e Anti-Affinità #### A. Anti-Affinità Indispensabile: `worker-app` * **Motivazione:** `worker-app` è l'unico carico che registra un consumo significativo di calcolo (~0.57 core complessivi). Co-locare i pod del nuovo batch (anch'esso *CPU-intensive* e sensibile alla latenza/cicli di calcolo) sugli stessi nodi fisici/virtuali di `worker-app` causerebbe un'immediata **contesa sulla CPU** (*CPU starvation / throttling*). * **Azione da intraprendere:** Applicare regole di **`podAntiAffinity`** (preferibilmente `requiredDuringSchedulingIgnoredDuringExecution`) nei confronti delle etichette di `worker-app`, oppure configurare *taints/tolerations* per isolare i nodi dedicati a `worker-app`. #### B. Affinità Consigliata: `demo-app` (o nodi ospitanti carichi Idle) * **Motivazione:** `demo-app` si trova in uno stato di sostanziale inattività dal punto di vista computazionale (0.0 core di CPU impiegati). Posizionare i pod del batch sui nodi che ospitano `demo-app` consente di effettuare un **bin-packing efficiente**, sfruttando i cicli CPU inutilizzati lasciati liberi da `demo-app` senza rischiare contese di calcolo. * **Nota su `cache-app`:** Sebbene anche `cache-app` utilizzi pochissima CPU (~0.008 core), genera un traffico continuo di scrittura su disco (~468 KB/s). Se il batch effettua anche operazioni di I/O, è preferibile privilegiare l'affinità verso `demo-app`. * **Azione da intraprendere:** Definire regole di **`podAffinity`** soft (`preferredDuringSchedulingIgnoredDuringExecution`) verso `demo-app` o utilizzare `nodeAffinity` per indirizzare i pod verso i nodi a basso utilizzo complessivo.
Claude Opus 5.5	55.16 s	20823	2667	23490	0.90	Regole di **affinità** verso **demo-app** e di **anti-affinity** verso **worker-app**. Prima però c'è un problema: il cluster ha **un solo nodo**, quindi una anti-affinity obbligatoria oggi bloccherebbe il batch. ## Cosa ho verificato oltre ai dati attuali - **Nodi:** uno solo, `thesis-cluster-control-plane`, con **8 core** allocabili. Tutti e 5 i pod attuali girano lì. - **CPU usata da tutti i pod:** circa **1,18 core**. **CPU prenotata (requests):** **1,4 core**. - **Storico di worker-app (ultime 6 ore, 25 campioni):** CPU costante tra **0,27 e 0,59 core**, quasi sempre tra 0,5 e 0,59. Il carico è continuo, non un picco isolato. ## Affinità: con demo-app - 2 pod, **0 core di CPU**, circa 15 MB di RAM, nessun traffico disco o rete. - È di fatto inattivo, quindi è il vicino ideale per un carico che ha bisogno di cicli CPU: non gli contende nulla. - Puoi usare una regola `podAffinity` "preferita" (`preferredDuringScheduling…`) sull'etichetta `app=demo-app`. ## Neutro o affinità debole: cache-app - Usa pochissima CPU (circa 0,008 core), quindi sul fronte CPU non dà fastidio. - Però scrive circa **468 KB/s su disco** in modo continuo. Se anche il batch scrive su disco (log, file temporanei, checkpoint), può nascere contesa su I/O e page cache. - Va bene stargli vicino solo se il batch è puramente di calcolo. ## Anti-affinity indispensabile: worker-app - È l'unico carico che consuma davvero CPU: **circa 0,57 core**, cioè circa **0,28 core per pod**, in modo stabile. - Pod di calcolo messi accanto a lui si contenderebbero il tempo di CPU dello scheduler Linux, con più cambi di contesto, cache del processore meno efficace e il rischio di rallentamenti forzati se sono impostati limiti di CPU. - Regola: `podAntiAffinity` sull'etichetta `app=worker-app` con `topologyKey: kubernetes.io/hostname`.

						## Anti-affinity indispensabile: worker-app - È l'unico carico che consuma davvero CPU: **circa 0,57 core**, cioè circa **0,28 core per pod**, in modo stabile. - Pod di calcolo messi accanto a lui si contenderebbero il tempo di CPU dello scheduler Linux, con più cambi di contesto, cache del processore meno efficace e il rischio di rallentamenti forzati se sono impostati limiti di CPU. - Regola: `podAntiAffinity` sull'etichetta `app=worker-app` con `topologyKey: kubernetes.io/hostname`. ## Il vincolo da cui dipende tutto: un solo nodo - Una anti-affinity **obbligatoria** (`requiredDuringScheduling…`) oggi lascerebbe **tutti e 12 i pod in Pending**, perché non c'è un altro nodo dove metterli. - Le strade possibili: 1. **Aggiungere almeno un nodo worker** dedicato al batch, magari con taint e toleration oltre all'anti-affinity. È l'unico modo per evitare davvero la contesa. 2. **Restare su un solo nodo:** usare una anti-affinity solo "preferita" (non cambia nulla in pratica) e ottenere l'isolamento in un altro modo: - classe QoS **Guaranteed**, con requests uguali ai limits e core interi; - **CPU Manager con policy `static`** sul kubelet, così ogni pod riceve core esclusivi e non condivisi con worker-app. ## Quanta CPU resta per il batch - Restano circa **6,6 core prenotabili** (8 meno 1,4), cioè circa **0,55 core per pod** se i pod sono 12. - Con core interi esclusivi (policy `static`) ce ne stanno solo **6**: per tutti e 12 servono più nodi oppure un'esecuzione a ondate (ad esempio `parallelism: 6` nel Job). - Ridurre worker-app o fissargli dei limiti libera poco (circa 0,6 core), ma rende il nodo più prevedibile. In breve: affinità verso demo-app, anti-affinity verso worker-app. Su un solo nodo però l'anti-affinity da sola non basta: servono un nodo in più oppure core dedicati con QoS Guaranteed e CPU Manager `static`.

Claude Sonnet 5.5	12.37 s	2812	1414	4226	0.90	## Raccomandazione **Affinity (co-locazione): demo-app, con cache-app come seconda scelta** **Anti-affinity (obbligatoria): worker-app** ### Dati dello snapshot | App | Pod | CPU (core) | CPU per pod | Memoria | Disco/Rete | Giudizio per il batch | |---|---|---|---|---|---|---| | worker-app | 2 | **0,568** | ~0,28 | 5,8 MB | 0 | Unico carico CPU-bound: va tenuto separato | | cache-app | 1 | 0,0078 | 0,0078 | 1,3 MB | **~468 KB/s in scrittura** | CPU trascurabile, ma con I/O disco | | demo-app | 2 | 0,0 | 0 | 15 MB | 0 | Idle: nessuna contesa di CPU | ### Motivazioni - **worker-app: anti-affinity indispensabile.** Consuma circa 0,57 core, cioè il 99% della CPU usata dai tre workload. Se un pod del batch finisse sullo stesso nodo, ci sarebbe contesa diretta sui cicli CPU: throttling o rallentamenti per il batch, che è sensibile a questo, e anche per il worker. - Usa `podAntiAffinity` con `requiredDuringSchedulingIgnoredDuringExecution` su `app=worker-app` e `topologyKey: kubernetes.io/hostname`. - Se i nodi sono pochi, usa la versione `preferred`, altrimenti alcuni pod restano Pending. - **demo-app: affinity possibile.** Ha CPU pari a 0 e rete e disco nulli, quindi non sottrae cicli al batch. Il batch può stare sugli stessi nodi. - **cache-app: affinity accettabile per la CPU, con una cautela.** Usa 0,008 core, trascurabili. Scrive però ~468 KB/s su disco. Se il batch fa I/O su disco, quei nodi potrebbero avere contesa di I/O. Per un batch puramente CPU non è un problema. - **Batch da 12 pod.** Se i pod del batch sono CPU-bound tra loro, conviene anche una `podAntiAffinity` o un `topologySpreadConstraints` tra i pod del batch stesso. Altrimenti si saturano a vicenda. Imposta `requests` e `limits` CPU coerenti, così lo scheduler tiene conto del carico reale. ### Limiti dell'analisi - È un solo campione istantaneo. Per confermare che demo-app e cache-app restino idle, controlla lo storico con `get_historical_metrics`. - Non ho dati sul nodo di ciascun pod, sulle CPU allocabili dei nodi, né su `requests`/`limits`. Non posso quindi verificare quanti nodi servano per i 12 pod né se worker-app sia già in throttling. Servono anche la capacità dei nodi e i `requests` del batch. Posso interrogare lo storico di worker-app e cache-app per verificare se i picchi di CPU e disco sono ricorrenti.

GPT-4o Mini	6.34 s	1348	449	1797	0.32	Per pianificare un batch computazionale da 12 pod sensibile ai cicli CPU, è importante analizzare i carichi di lavoro attuali delle applicazioni esistenti per identificare eventuali contese e stabilire regole di affinità e anti-affinity. ### Analisi dei Carichi di Lavoro Attuali 1. **cache-app**: - Pod: 1 - CPU: 0.0078 cores - Memoria: 1.29 MB 2. **demo-app**: - Pod: 2 - CPU: 0.0 cores - Memoria: 14.95 MB 3. **worker-app**: - Pod: 2 - CPU: 0.568 cores - Memoria: 5.82 MB ### Raccomandazioni #### Affinità - **worker-app**: Questa applicazione sta utilizzando una quantità significativa di CPU (0.568 cores) rispetto alle altre applicazioni. Se il batch computazionale richiede un uso intensivo della CPU, potrebbe essere vantaggioso pianificare i pod di questo batch su nodi che non eseguono `worker-app` per evitare contese. #### Anti-affinity - **worker-app**: Dato che `worker-app` sta già utilizzando una quantità considerevole di CPU, è indispensabile applicare regole di anti-affinity per i pod del batch computazionale. Questo garantirà che i nuovi pod non vengano pianificati sugli stessi nodi dei pod `worker-app`, riducendo il rischio di contesa per le risorse CPU. ### Conclusione - **Definire regole di affinità** per i pod del batch computazionale su nodi non occupati da `worker-app`. - **Applicare regole di anti-affinity** per evitare che i pod del batch vengano pianificati sugli stessi nodi dei pod `worker-app`. Queste misure aiuteranno a garantire che il batch computazionale possa operare in modo efficiente senza interferenze da parte di altre applicazioni.

GPT-4o	4.64 s	1348	498	1846	0.28	Per pianificare un batch computazionale sensibile ai cicli CPU, è importante analizzare le metriche attuali per identificare quali applicazioni potrebbero causare contesa delle risorse CPU e quali potrebbero essere più adatte per l'affinità. ### Analisi delle Metriche Istantanee: 1. **cache-app**: - CPU: 0.0078 cores - Memoria: 1.29 MB - Attività disco: 468000.74 Bps - Attività di rete: 0.0 Bps 2. **demo-app**: - CPU: 0.0 cores - Memoria: 14.95 MB - Attività disco: 0.0 Bps - Attività di rete: 0.0 Bps 3. **worker-app**: - CPU: 0.5678 cores - Memoria: 5.82 MB - Attività disco: 0.0 Bps - Attività di rete: 0.0 Bps ### Raccomandazioni: - **Regole di Affinità**: - **cache-app**: Questa applicazione utilizza pochissima CPU e memoria, quindi potrebbe essere un buon candidato per l'affinità con il batch computazionale. La sua bassa attività di rete e disco suggerisce che non dovrebbe interferire significativamente con il batch. - **Regole di Anti-Affinity**: - **worker-app**: Questa applicazione utilizza una quantità significativa di CPU (0.5678 cores), il che potrebbe portare a contesa delle risorse CPU se pianificata insieme al batch computazionale. È consigliabile applicare regole di anti-affinity per evitare che i pod del batch vengano pianificati sullo stesso nodo dei pod di worker-app. - **demo-app**: Non utilizza CPU e ha un utilizzo di memoria moderato. Non è necessario applicare regole di affinità o anti-affinity specifiche per questa applicazione, a meno che non ci siano altre considerazioni di rete o disco. In sintesi, per evitare contese CPU, è indispensabile applicare regole di anti-affinity con **worker-app**, mentre **cache-app** potrebbe essere un buon candidato per l'affinità con il batch computazionale.

DOMANDA 10B

Sistema	Latenza	In Tok	Out Tok	Tot Tok	Accuratezza	Risposta Completa
Script Deterministico	1.90 ms	0	0	0	N/A	Co-locare preferibilmente con 'demo-app' (minor CPU: 0.0000 cores) ed ISOLARE da 'worker-app' (maggior CPU: 0.5678 cores)
Gemini 2.5 Flash	5.66 s	1658	886	2544	0.42	Basandomi sulle metriche istantanee fornite, ecco le mie raccomandazioni: **Applicazione da evitare assolutamente per l'allocazione delle risorse:** * **cache-app**: Sebbene l'utilizzo di CPU e memoria sia basso, `cache-app` mostra un'attività di scrittura su disco significativa (468 KB/s). Affiancare un carico batch critico di 12 pod potrebbe portare a contesa I/O sul disco, causando un degrado delle prestazioni per entrambi i carichi di lavoro a causa dell'effetto "noisy neighbor". **Applicazione che offre l'affiancamento più affidabile:** * **demo-app**: Questa applicazione è la candidata più affidabile. Attualmente utilizza 0 CPU e solo 14.95 MB di memoria, senza alcuna attività di scrittura su disco. Questo indica che ha ampie risorse disponibili e un impatto minimo sul sistema, rendendola ideale per co-locare un carico batch critico senza rischi di degrado. In sintesi, per schedulare il carico batch critico, consiglio di affiancarlo a `demo-app` e di evitare `cache-app`.
Gemini 3.6 Flash	29.19 s	4081	2320	6401	0.95	Sulla base dell'analisi delle metriche istantanee e dello storico temporale registrato, ecco la valutazione per il posizionamento del carico batch critico da 12 pod senza incorrere in fenomeni di *noisy neighbor*: --- ### 1. Applicazione per l'affiancamento più affidabile: **`demo-app`** * **Stato Risorse:** * **CPU:** `0.0 cores` (trascurabile nei rilevamenti storici, max ~0.002 cores). * **Memoria:** `14.95 MB` totali distribuiti su 2 pod. * **I/O Disco / Rete:** `0.0 B/s`. * **Motivazione:** `demo-app` è in uno stato di sostanziale **idle/inattività**. Non genererà contesa né sui cicli di CPU né su I/O o banda di rete, rendendola il candidato ideale su cui affiancare i pod del carico batch critico. --- ### 2. Applicazione da evitare assolutamente: **`worker-app`** * **Stato Risorse:** * **CPU:** `0.568 cores` (~568 millicores su 2 pod), registrando un forte incremento rispetto allo storico recente (`0.275 cores`). * **Memoria:** `5.82 MB`. * **Motivazione:** `worker-app` è un carico **CPU-bound in crescita attiva**. Co-locare un batch critico da 12 pod sui medesimi nodi o nello stesso dominio di risorse causerebbe un'elevata contesa sui cicli CPU (*CPU throttling* e *context switching*), degradando le prestazioni sia del batch che di `worker-app`. --- ### 3. Considerazione aggiuntiva (`cache-app`) * **Attività I/O:** `cache-app` mostra un consumo di CPU e memoria contenuto (`0.0078 cores`, `1.29 MB`), ma registra un throughput di **scrittura su disco significativo e in crescita** (`~468 KB/s`, rispetto ai `131 KB/s` precedenti). Se il carico batch effettua operazioni intensive su disco, è opportuno evitare anche la co-locazione con `cache-app` per prevenire la contesa di I/O (*disk I/O wait*). --- ### Raccomandazione Finale Schedules/alloca i **12 pod batch** preferibilmente nei nodi/pool in cui risiede **`demo-app`** e **evita tassativamente** i nodi ospitanti **`worker-app`**.

Claude Opus 5.5	64.94 s	53008	3606	56614	0.95	**Da preferire come vicino: demo-app. Da evitare assolutamente: worker-app.** C'è però un vincolo che conta più di questa scelta: **tutti i pod girano su un unico nodo**, `thesis-cluster-control-plane` (8 core, circa 3,7 GiB di memoria allocabile). Qualunque pod del batch condividerà quindi CPU, disco e memoria con tutte e tre le applicazioni. Ho verificato su storico MongoDB (ultime 24h) e Prometheus. ## Dati misurati | App | CPU (ultime 24h) | Limite CPU per pod | Throttling CPU | Scritture su disco | Memoria | |---|---|---|---|---|---| | **demo-app** (2 pod) | quasi sempre 0, picchi di ~0,0016 core | 0,1 | nessuno | 0 (picchi di circa 140 B/s) | 14–19 MB, stabile | | **cache-app** (1 pod) | 0,004–0,023 core | 0,1 | **~41%** | **costanti, 290–620 KB/s** | 0,8–2,4 MB | | **worker-app** (2 pod) | **costante, 0,36–0,59 core** | 0,3 | **~96%** | 0 | ~5 MB | ## Vicino più affidabile: demo-app - È inattiva in modo stabile: niente CPU, niente disco, niente rete per quasi tutta la finestra di 24h. - La memoria è piatta: non si vedono perdite di memoria né consumi improvvisi. - Non ci sono picchi che possano disturbare i pod del batch. ## Da evitare assolutamente: worker-app - Usa la CPU in modo continuo e senza pause: circa 0,57 core su 0,6 di limite totale, cioè il 95% del tetto. - Viene rallentata dal limite di CPU nel 96% degli intervalli. Questo indica che consumerebbe molta più CPU se potesse. Se le si alzano i limiti o la si scala, diventa subito un vicino molto rumoroso. - In valore assoluto pesa circa il 7% della CPU del nodo. Il problema non è la quantità ma la pressione costante sullo scheduler della CPU, che fa variare i tempi di risposta di chi le sta accanto.

Default: filip@DESKTOP-0UIODV3: ~/AgentLab
ctrl+alt+1

- Usa la CPU in modo continuo e senza pause: circa 0,57 core su 0,6 di limite totale, cioè il 95% del tetto.
- Viene rallentata dal limite di CPU nel 96% degli intervalli. Questo indica che consumerebbe molta più CPU se potesse. Se le si alzano i limiti o la si scala, diventa subito un vicino molto rumoroso.
- In valore assoluto pesa circa il 7% della CPU del nodo. Il problema non è la quantità ma la pressione costante sullo scheduler della CPU, che fa variare i tempi di risposta di chi le sta accanto.

Da tenere d'occhio: cache-app
- Usa poca CPU, ma il 41% di throttling mostra che lavora a raffiche brevi.
- Soprattutto **scrive su disco in continuazione** (~470 KB/s di media). Se il batch legge o scrive molto su disco, cache-app è il vicino peggiore per l'I/O, anche se non per la CPU.

Un dato da verificare prima del lancio
Tutti i pod hanno esattamente 53 riavvii. Un numero identico ovunque fa pensare a una causa a livello di nodo o di cluster (riavvii del nodo o del container runtime, tipici di un cluster di test come kind) più che a un problema delle singole applicazioni. Se il nodo si riavvia periodicamente, nessun vicino rende affidabile un batch critico. Conviene controllare `kubectl describe node` e gli eventi prima di lanciarlo.

Raccomandazioni per il batch da 12 pod
1. **Classe di priorità garantita:** impostare `requests = limits` per CPU e memoria sui pod del batch, così Kubernetes li tratta come prioritari e sono gli ultimi a essere sacrificati se manca memoria.
2. **Isolamento CPU:** se servono tempi di risposta prevedibili, valutare il CPU Manager in modalità `static` con richieste di CPU intere.
3. **Più nodi:** finché il cluster ha un solo nodo, regole come `podAntiAffinity` contro `app=worker-app` non servono, perché non c'è un altro nodo dove mettere i pod. Se il batch è critico, aggiungere un nodo dedicato con taint e toleration.
4. **Capacità:** oggi i limiti delle applicazioni esistenti sommano 0,9 core e 768 MiB. Con 12 pod restano circa 7 core e meno di 3 GiB da dividere: sotto i 200–250 MiB di memoria per pod non c'è margine. Il limite reale sarà la memoria, non la CPU.
5. **Classe di priorità:** assegnare al batch una `PriorityClass` più alta di worker-app, così in caso di contesa lo scheduler sposta worker-app e non il batch.

Claude Sonnet 5.5	33.10 s	17288	2058	19346	0.95	## Raccomandazione **Affiancamento più affidabile: `demo-app`.** **Da evitare: `worker-app`.** `cache-app` richiede cautela solo se il batch scrive molto su disco. Ho controllato le ultime 24 ore, circa 33 campioni per app, per verificare che i valori istantanei siano stabili. | App | Pod | CPU (core) | RAM | Disk write | Net RX | Comportamento nello storico | |---|---|---|---|---|---|---| | `demo-app` | 2 | 0,0 (max ~0,0017) | ~15 MB | ~0 B/s | 0 | Quasi idle, molto stabile | | `cache-app` | 1 | ~0,008 (max ~0,023) | ~1,3 MB | ~470 KB/s | 0 | CPU bassa, scrittura disco costante (290–620 KB/s) | | `worker-app` | 2 | ~0,57 | ~5,8 MB | ~0 | ~0 (picchi ~15 B/s) | CPU alta e costante (0,36–0,59) | ## Motivazioni - **`demo-app` è il vicino più sicuro.** Ha CPU, disco e rete praticamente nulli, e la memoria è stabile a 14–19 MB. Non ha picchi, quindi è improbabile che disturbi il batch. - **`worker-app` è da evitare.** - Consuma circa 0,57 core in modo continuo, cioè 50–400 volte più delle altre app. - Il carico è sostenuto e non a raffiche, quindi la competizione sulla CPU sarebbe permanente. - I valori si fermano spesso intorno a 0,59 core, un massimale che potrebbe indicare un limite CPU con throttling. È solo un'ipotesi: i limiti non sono nelle metriche. - Un batch da 12 pod sullo stesso nodo subirebbe contesa sulla CPU, con possibile throttling in entrambe le direzioni. - **`cache-app` è un rischio secondario sull'I/O.** Scrive su disco in modo costante (~0,47 MB/s, unico carico di questo tipo nel cluster). La CPU è trascurabile, quindi va bene se il batch è CPU-bound. Se il batch scrive molto su disco, è un potenziale vicino rumoroso. ## Limiti dell'analisi - Le metriche sono aggregate per applicazione. Non so su quali nodi girano i pod, quindi non posso dire chi condivide davvero un nodo con il batch. - Mancano requests/limits e la capacità dei nodi, quindi non posso dire se i nodi reggono 12 pod aggiuntivi. - Prima di schedulare, verifica la distribuzione dei pod sui nodi con `kubectl get pod -o wide`. Poi usa `podAntiAffinity` contro `app=worker-app`, o `nodeAffinity` o taint dedicati per isolare il batch. - Imposta requests e limits espliciti sui 12 pod del batch, con QoS Guaranteed (requests uguali ai limits). Questo protegge il carico critico anche in caso di contesa.

GPT-4o Mini	6.46 s	1351	378	1729	0.00	Analizzando le metriche attuali delle applicazioni: 1. **cache-app**: - Pod attivi: 1 - CPU utilizzata: 0.0078 cores - Memoria utilizzata: 1.29 MB - Questa applicazione ha un utilizzo di risorse molto basso. 2. **demo-app**: - Pod attivi: 2 - CPU utilizzata: 0.0 cores - Memoria utilizzata: 14.95 MB - Anche questa applicazione ha un utilizzo di CPU molto basso, ma una memoria relativamente più alta rispetto alla cache-app. 3. **worker-app**: - Pod attivi: 2 - CPU utilizzata: 0.568 cores - Memoria utilizzata: 5.82 MB - Questa applicazione ha un utilizzo di CPU significativamente più alto rispetto alle altre due. ### Raccomandazioni: - **Applicazione da affiancare**: **cache-app**. Ha un utilizzo di CPU e memoria molto basso, il che la rende un candidato ideale per l'affiancamento con il carico batch critico. Può gestire ulteriori carichi senza rischiare di influenzare negativamente le prestazioni. - **Applicazione da evitare**: **worker-app**. Con un utilizzo di CPU di 0.568 cores, è già più carica rispetto alle altre applicazioni e potrebbe causare degrado delle prestazioni per il carico batch critico a causa del rischio di 'noisy neighbor'. In sintesi, per un'affidabilità ottimale, si consiglia di affiancare il carico batch alla **cache-app** e di evitare di allocare risorse alla **worker-app**.

GPT-4o	5.16 s	1351	379	1730	0.00	Per determinare quale applicazione offre l'affiancamento più affidabile per schedulare un carico batch critico da 12 pod, analizziamo le metriche istantanee delle applicazioni attuali: 1. **Cache-app**: - CPU: 0.0078 cores - Memoria: 1.29 MB - Disco: 468,000 Bps - Rete: 0 Bps 2. **Demo-app**: - CPU: 0 cores - Memoria: 14.95 MB - Disco: 0 Bps - Rete: 0 Bps 3. **Worker-app**: - CPU: 0.5678 cores - Memoria: 5.82 MB - Disco: 0 Bps - Rete: 0 Bps **Raccomandazioni:** - **Affiancamento più affidabile**: - **Demo-app**: Questa applicazione attualmente non utilizza CPU e ha un utilizzo di memoria relativamente basso. Non ci sono operazioni di disco o rete in corso, il che la rende un candidato ideale per evitare il rischio di 'noisy neighbor'. - **Applicazione da evitare**: - **Worker-app**: Questa applicazione sta utilizzando una quantità significativa di CPU (0.5678 cores), il che potrebbe causare problemi di risorse se affiancata a un carico batch critico. È meglio evitare di schedulare i nuovi pod in questo contesto per prevenire il degrado delle prestazioni. In sintesi, per minimizzare il rischio di interferenze, è consigliabile affiancare il carico batch alla "demo-app" e evitare di utilizzare la "worker-app".

[BILLING MONITOR] Spesa cumulata OpenAI: $1.6793 (mini: $0.0818 | 4o: $1.5975)